

Deep Learning, Generative AI & Neural Networks

3-Week Intensive Learning Plan

4 hours/week · Hands-on · From scratch to deployment

Roadmap

WEEK 1

The Neuron & Backprop Engine

MILESTONE

Build a working autograd engine from scratch

Curriculum: ① ② ⑤



WEEK 2

Deep Architectures & Training Dynamics

MILESTONE

Train a CNN on real data with >90% acc and explain WHY

Curriculum: ③ ⑥ ⑦



WEEK 3

Generative AI & Full-Stack Deploy

MILESTONE

Deploy a GenAI app end-to-end

Curriculum: ④ + GenAI

7 Core Principles

The theoretical backbone of this course

① Universal Approximation

One hidden layer approximates any function — depth is about efficiency

② ReLU Changed Everything

Sparse activation + no vanishing gradients = trainable deep networks

③ Overparameterization Helps

More params than data, yet generalizes — implicit regularization of SGD

④ NNs Don't Understand

They minimize loss. They detect statistical structure, not meaning.

⑤ Backprop = Chain Rule

Efficient gradient computation via dynamic programming. Nothing mystical.

⑥ Geometry Matters

Flat minima → better generalization. Sharp minima → poor robustness.

⑦ Capacity $\neq$ Performance

Architecture, init, normalization, data quality dominate over depth.

The Neuron & Backprop Engine

LEARNING FOCUS

How a neural network actually computes — forward pass, loss, gradients, update.

One concept: automatic differentiation is the chain rule, implemented as a graph traversal.

RESOURCES

Karpathy — "Zero to Hero" Lecture 1: micrograd

Builds autograd from scratch in 2 hours

3Blue1Brown — Neural Networks (Ch. 1-3)

Unmatched geometric intuition (~40 min)

YOU KNOW YOU'VE GOT IT WHEN...

You can draw the computation graph for $L = (w \cdot x + b) \cdot \text{relu}()$ and manually compute $\partial L / \partial w$

Source: [Karpathy Zero to Hero](#), [3Blue1Brown Neural Networks](#)

BUILD EXERCISE

Implement micrograd yourself:

1. Scalar-valued autograd engine
2. Value class with +, *, ReLU, backward()
3. Neuron → Layer → MLP classes
4. Train on 2D moons dataset
5. Visualize decision boundary

Week 1 Key Concepts

① Universal Approximation

A single hidden layer can approximate any continuous function — but may need impractically many neurons.

Depth is about efficiency of representation, not just power.

② ReLU Changed Everything

Before: sigmoid/tanh → vanishing gradients.
ReLU: sparse activation + gradient is 0 or 1.

This made deep networks trainable.

⑤ Backpropagation = Chain Rule

Nothing mystical. Given $L = f(g(h(x)))$, the chain rule gives:

$$\partial L / \partial x = (\partial L / \partial f) \cdot (\partial f / \partial g) \cdot (\partial g / \partial h) \cdot (\partial h / \partial x)$$

Backprop traverses the computation graph in reverse, accumulating gradients. It is dynamic programming applied to differentiation.

Deep Architectures & Training Dynamics

LEARNING FOCUS

Going from toy to real. Architecture + initialization + normalization matter more than raw depth.

RESOURCES

PyTorch tutorials — Training a Classifier

Canonical intro, short and focused

Karpathy — makemore Part 2

BatchNorm, init, training dynamics

d2l.ai Chapter 7 — CNNs

Runnable code + geometric explanations

YOU KNOW YOU'VE GOT IT WHEN...

You can look at a training curve and diagnose: learning rate too high? Vanishing gradients? Overfitting?

Source: [PyTorch Tutorials](#), [Dive into Deep Learning](#)

BUILD EXERCISE

CNN Ablation Study on CIFAR-10:

1. Start with bad arch (sigmoid, no BN)
2. Switch to ReLU → measure gain
3. Add He initialization → measure
4. Add BatchNorm → measure
5. Add data augmentation → measure
6. Log all experiments, plot curves

Target: >90% test accuracy

Week 2 Key Concepts

③

Overparameterization Helps

Networks with more parameters than data points generalize well. SGD acts as implicit regularizer — it finds simple solutions in weight space.

⑥

Geometry Matters

Training navigates a high-dimensional loss landscape. Flat minima generalize better. Sharp minima break under distribution shift.

⑦

Capacity $\neq$ Performance

More layers $\neq$ better. He/Xavier init, BatchNorm, data quality, and architecture design dominate raw capacity.

Why Initialization Matters

Same architecture, same data, same optimizer — only the initial weights change:

Method	Formula	Best For	Result
Random $N(0,1)$	$W \sim N(0, 1)$	Nothing	Exploding activations
Xavier / Glorot	$W \sim N(0, 2/(n_{in}+n_{out}))$	Sigmoid, Tanh	Stable gradients
He / Kaiming	$W \sim N(0, 2/n_{in})$	ReLU	Optimal gradient flow

The lesson: same capacity, vastly different outcomes. Architecture choices are first-order.

Generative AI & Full-Stack Deployment

LEARNING FOCUS

From discriminative to generative. Generative models learn the data distribution. Quality depends on what they saw, not what they know.

RESOURCES

HuggingFace NLP Course — Ch. 1-3

Fastest path to using pretrained transformers

fast.ai — Lesson 9: Stable Diffusion

Diffusion models without drowning in math

Simon Willison's blog

Practical, opinionated LLM building guides

YOU KNOW YOU'VE GOT IT WHEN...

You can explain the difference between a model that 'understands' and one that learned statistical patterns

Source: [HuggingFace Course](#), [fast.ai](#)

BUILD EXERCISE

Full-Stack GenAI Application:

1. Fine-tune / prompt-engineer a model
2. Domain: medical procedure summaries
3. Build Gradio interface with controls
4. Deploy to HuggingFace Spaces
5. Explain every layer of the stack

④ Neural Networks Don't "Understand"

What they DO

- Minimize a loss function
- Detect statistical regularities
- Approximate $P(\text{output} \mid \text{input})$
- Interpolate between training examples
- Compress information into weights

What they DON'T

- Reason about truth or falsehood
- Grasp causation (only correlation)
- Verify their own outputs
- Know what they don't know
- Have intentions or beliefs

This distinction is critical for safety-sensitive domains like medical AI.
A model that has "seen" medical text is not a model that "knows" medicine.

Projects Ladder

P1 micrograd: Autograd from Scratch

Week 1 | ★★☆☆☆

"I built backpropagation. Here's gradient flowing backward."

P2 CIFAR-10 CNN Training Lab

Week 2 | ★★☆☆☆

"I trained an image classifier to 92% accuracy. Here's why BatchNorm + He + ReLU won."

P3 Generative Text Application

Week 3 | ★★★★★

"I deployed an app that generates medical summaries. Try it."

P4 NN Myths Blog Post (Bonus)

Bonus | ★★☆☆☆

Debunk 5 common NN misconceptions with code demos.

5 Common Traps

1 Tutorial Hell

Never watch >30 min without coding. Pause, implement, continue.

2 Skipping Math That Matters

Hand-compute one backward pass. You don't need measure theory, but you DO need gradients.

3 Blaming the Model

Inspect data first: visualize samples, check balance, find label noise. 10 min of EDA saves 10 hours.

4 Overcomplicating Architecture

Start with the simplest model. Add complexity only when you can measure the improvement.

5 Confusing 'It Works' with 'I Understand'

After every experiment, write one paragraph explaining WHY. If you can't, go back.

Proving You Learned This

PORTFOLIO PIECES

- GitHub repo with micrograd + training notebooks
- Experiment tracking via Weights & Biases (free tier)
- Deployed Gradio app on HuggingFace Spaces

CERTIFICATIONS

- DeepLearning.AI Specialization (Coursera)
- HuggingFace Certification

COMMUNITIES

fast.ai forums

Most helpful DL learning community

r/MachineLearning & r/learnmachinelearning

Discussion, papers, project feedback

HuggingFace Discord

Active, welcoming, GenAI-focused

Today's Assignment

30 minutes to build momentum

Build a Single Neuron in Pure Python

1. Implement a Value class (data + grad)
2. Support `__add__`, `__mul__`, `__neg__`
3. Implement sigmoid: $1 / (1 + \exp(-x))$
4. Compute: $y = \text{sigmoid}(w_1 * x_1 + w_2 * x_2 + b)$
 $w_1=0.5, w_2=-0.3, b=0.1, x_1=2.0, x_2=3.0$
5. Call `y.backward()` and inspect gradients

This is the seed of your Week 1 project.

Accompanying Notebooks

SERIES 1: PURE PYTHON

Only Python + NumPy + matplotlib. No frameworks.

NB 01 The Neuron & Backprop Engine

Value class, autograd, MLP, moons training

NB 02 Training Dynamics

Matrix MLP, MNIST, init experiments, loss landscape

NB 03 Generative Models from Scratch

Char-level LM, autoencoder, temperature sampling

SERIES 2: PYTORCH

Professional framework. Build on pure-Python intuition.

NB 04 PyTorch Fundamentals & Autograd

Tensors, autograd, nn.Module, moons training

NB 05 CNNs & Overparameterization

CIFAR-10, ablation study, feature visualization

NB 06 Generative AI with Transformers

Attention, mini-GPT, HuggingFace, Gradio deploy

Start Building. Not Watching.

"Every concept introduced by building something, not reading about it."

3 weeks · 6 notebooks · 4 projects · 1 deployed app

Notebook 1: The Neuron & Backprop Engine

What We'll Build and Why

In this notebook we build a **complete automatic differentiation engine** from scratch -- no framework, no magic, just Python. We will implement a `Value` class that tracks every arithmetic operation and compute gradients automatically using the chain rule. On top of this engine we build neurons, lay out a full MLP, then train it to classify 2D data and approximate complex functions.

Three big ideas thread through everything we do:

#	Curriculum Point	Where We See It
①	Universal Approximation Theorem	Section 7 -- wide vs deep networks
②	ReLU changed everything	Section 4 -- vanishing gradients
⑤	Backpropagation = chain rule	Sections 2-3 -- the <code>Value</code> engine

Requirements: Python 3, NumPy, matplotlib, scikit-learn (for data generation only).

Part 1 -- The `Value` Class: A Scalar Autograd Engine

The core idea: wrap every scalar number in a `Value` object that remembers:

- **what operation** created it (+, *, ReLU, ...)
- **which inputs** (children) fed into that operation
- **how to propagate gradients** backward through that operation

This is *exactly* what PyTorch does with tensors, but we do it for scalars so every step is transparent.

⑤ **Curriculum Point -- Backpropagation is just the chain rule.** If $z = f(y)$ and $y = g(x)$, then $\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$. Our `_backward` lambda implements exactly this: each operation knows its local derivative and multiplies by the incoming gradient.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import math
import random

# Reproducibility
np.random.seed(42)
random.seed(42)

%matplotlib inline
```

```
plt.rcParams["figure.dpi"] = 100
plt.rcParams["font.size"] = 11
```

```
In [2]: class Value:
    """A scalar value with automatic gradient tracking.

    Stores the numerical value, its gradient (accumulated during backprop
    a backward function (local chain-rule step), and pointers to children
    """

    def __init__(self, data, _children=(), _op="", label=""):
        self.data = float(data)
        self.grad = 0.0          # dL/d(self), accumulated during backwa
        self._backward = lambda: None  # local chain-rule step
        self._prev = set(_children)
        self._op = _op
        self.label = label

    # ---- Arithmetic operations ----

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), "+")
        def _backward():
            # d(a+b)/da = 1, d(a+b)/db = 1
            self.grad += 1.0 * out.grad
            other.grad += 1.0 * out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), "*")
        def _backward():
            # d(a*b)/da = b, d(a*b)/db = a
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def __pow__(self, other):
        assert isinstance(other, (int, float)), "only int/float powers"
        out = Value(self.data ** other, (self,), f"**{other}")
        def _backward():
            # d(a^n)/da = n * a^(n-1)
            self.grad += other * (self.data ** (other - 1)) * out.grad
        out._backward = _backward
        return out

    def __neg__(self):
        return self * -1

    def __sub__(self, other):
        return self + (-other)

    def __truediv__(self, other):
        return self * (other ** -1) if isinstance(other, Value) else self

    def __radd__(self, other):
        return self + other
```

```

def __rmul__(self, other):
    return self * other

def __rsub__(self, other):
    return Value(other) + (-self)

def __rtruediv__(self, other):
    return Value(other) * (self ** -1)

# ---- Activation functions ----

def relu(self):
    out = Value(max(0, self.data), (self,), "relu")
    def _backward():
        #  $d(\text{relu})/dx = 1$  if  $x > 0$  else  $0$ 
        self.grad += (1.0 if self.data > 0 else 0.0) * out.grad
    out._backward = _backward
    return out

def sigmoid(self):
    # Numerically stable sigmoid
    if self.data >= 0:
        s = 1.0 / (1.0 + math.exp(-self.data))
    else:
        e = math.exp(self.data)
        s = e / (1.0 + e)
    out = Value(s, (self,), "sigmoid")
    def _backward():
        #  $d(\text{sigmoid})/dx = \text{sigmoid} * (1 - \text{sigmoid})$ 
        self.grad += s * (1 - s) * out.grad
    out._backward = _backward
    return out

def tanh(self):
    t = math.tanh(self.data)
    out = Value(t, (self,), "tanh")
    def _backward():
        #  $d(\text{tanh})/dx = 1 - \text{tanh}^2$ 
        self.grad += (1 - t ** 2) * out.grad
    out._backward = _backward
    return out

def exp(self):
    e = math.exp(self.data)
    out = Value(e, (self,), "exp")
    def _backward():
        self.grad += e * out.grad
    out._backward = _backward
    return out

# ---- Backward pass ----

def backward(self):
    """Compute gradients for the entire computation graph.

    Uses topological sort to ensure each node is processed
    *after* all nodes that depend on it.
    """
    topo = []

```

```

visited = set()
def build_topo(v):
    if v not in visited:
        visited.add(v)
        for child in v._prev:
            build_topo(child)
        topo.append(v)
build_topo(self)

# Start: dL/dL = 1
self.grad = 1.0
# Walk in reverse topological order (output -> inputs)
for node in reversed(topo):
    node._backward()

def __repr__(self):
    return f"Value(data={self.data:.4f}, grad={self.grad:.4f})"

```

Quick sanity check

Let's create a couple of `Value` objects and verify arithmetic works:

```

In [3]: a = Value(3.0, label="a")
        b = Value(-2.0, label="b")
        c = a + b
        d = a * b + b ** 3
        print(f"a = {a}")
        print(f"b = {b}")
        print(f"a + b = {c}")
        print(f"a * b + b^3 = {d}")
        print(f" Expected: 3*(-2) + (-2)^3 = -6 + (-8) = -14 -> got {d.data}")

```

```

a = Value(data=3.0000, grad=0.0000)
b = Value(data=-2.0000, grad=0.0000)
a + b = Value(data=1.0000, grad=0.0000)
a * b + b^3 = Value(data=-14.0000, grad=0.0000)
Expected: 3*(-2) + (-2)^3 = -6 + (-8) = -14 -> got -14.0

```

Visualizing the computation graph (text-based)

Below is a simple helper that prints the computation graph as an indented tree. Each node shows value, its gradient (after `backward()`), and the operation that created it.

```

In [4]: def draw_graph(root, indent=0, visited=None):
        """Print a text-based tree of the computation graph."""
        if visited is None:
            visited = set()
        prefix = " " * indent
        op_str = f" [{root._op}]" if root._op else ""
        lbl = f" ({root.label})" if root.label else ""
        grad_str = f" grad={root.grad:.4f}" if root.grad != 0 else ""
        print(f"{prefix}Value={root.data:.4f}{lbl}{op_str}{grad_str}")
        if id(root) not in visited:
            visited.add(id(root))
            for child in root._prev:
                draw_graph(child, indent + 1, visited)

```

```
# Build a small expression and visualize
x = Value(2.0, label="x")
y = Value(3.0, label="y")
z = x * y + y ** 2
z.backward()

print("Computation graph for z = x*y + y^2:")
print("=" * 50)
draw_graph(z)
print()
print(f"dz/dx = {x.grad:.4f} (expected: y = 3)")
print(f"dz/dy = {y.grad:.4f} (expected: x + 2y = 2 + 6 = 8)")
```

Computation graph for z = x*y + y^2:

=====

```
Value=15.0000 [+] grad=1.0000
  Value=9.0000 [*2] grad=1.0000
    Value=3.0000 (y) grad=8.0000
  Value=6.0000 [*] grad=1.0000
    Value=2.0000 (x) grad=3.0000
    Value=3.0000 (y) grad=8.0000
```

```
dz/dx = 3.0000 (expected: y = 3)
dz/dy = 8.0000 (expected: x + 2y = 2 + 6 = 8)
```

Part 2 -- Gradient Verification

⑤ Curriculum Point: Backpropagation is just the chain rule.

Let's prove it by hand-computing gradients for a simple expression, then checking that our engine gets the same answer.

Example: $f = (a + b) \cdot c$ where $a=2$, $b=3$, $c=-4$.

By hand:

- Let $d = a + b = 5$
- $f = d \cdot c = -20$
- $\frac{\partial f}{\partial c} = d = 5$
- $\frac{\partial f}{\partial d} = c = -4$
- $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial d} \cdot \frac{\partial d}{\partial a} = -4 \cdot 1 = -4$
- $\frac{\partial f}{\partial b} = \frac{\partial f}{\partial d} \cdot \frac{\partial d}{\partial b} = -4 \cdot 1 = -4$

In [5]:

```
# Verify with our engine
a = Value(2.0, label="a")
b = Value(3.0, label="b")
c = Value(-4.0, label="c")
```

```
d = a + b # d = 5
f = d * c # f = -20
```

```
f.backward()

print("f = (a + b) * c")
print(f"  f = {f.data:.1f}")
print(f"  df/da = {a.grad:.1f}  (expected: -4)")
print(f"  df/db = {b.grad:.1f}  (expected: -4)")
print(f"  df/dc = {c.grad:.1f}  (expected: 5)")
```

```
f = (a + b) * c
f = -20.0
df/da = -4.0  (expected: -4)
df/db = -4.0  (expected: -4)
df/dc = 5.0   (expected: 5)
```

A more complex example: verify gradients through an activation

Let's check: $g = \tanh(2x + 3y)$ at $x=1, y=2$.

By hand:

- $u = 2x + 3y = 8$
- $g = \tanh(8) \approx 0.99999977$
- $\frac{\partial g}{\partial u} = 1 - \tanh^2(8) \approx 4.5 \times 10^{-7}$
- $\frac{\partial g}{\partial x} = \frac{\partial g}{\partial u} \cdot 2 \approx 9.1 \times 10^{-7}$
- $\frac{\partial g}{\partial y} = \frac{\partial g}{\partial u} \cdot 3 \approx 1.4 \times 10^{-6}$

These tiny gradients are the **vanishing gradient problem** in action!

```
In [6]: x = Value(1.0, label="x")
y = Value(2.0, label="y")
g = (Value(2.0) * x + Value(3.0) * y).tanh()
g.backward()

print("g = tanh(2x + 3y) at x=1, y=2")
print(f"  g = {g.data:.8f}")
print(f"  dg/dx = {x.grad:.2e}  (expected: ~9.1e-7)")
print(f"  dg/dy = {y.grad:.2e}  (expected: ~1.4e-6)")
print()
print("The gradients are tiny because tanh saturates near +/-1.")
print("This is the vanishing gradient problem we explore next.")
```

```
g = tanh(2x + 3y) at x=1, y=2
g = 0.99999977
dg/dx = 9.00e-07  (expected: ~9.1e-7)
dg/dy = 1.35e-06  (expected: ~1.4e-6)
```

The gradients are tiny because tanh saturates near +/-1.

This is the vanishing gradient problem we explore next.

Numerical gradient check

We can also verify our analytical gradients by comparing with **finite differences**: $\frac{\partial f}{\partial x} \approx \frac{f(x+h) - f(x-h)}{2h}$

This is slow but serves as a ground truth.

```
In [7]: def numerical_gradient(func, inputs, idx, h=1e-5):
        """Compute df/d(inputs[idx]) using central finite differences."""
        # Forward: f(x + h)
        inputs[idx].data += h
        f_plus = func().data
        # Backward: f(x - h)
        inputs[idx].data -= 2 * h
        f_minus = func().data
        # Restore
        inputs[idx].data += h
        return (f_plus - f_minus) / (2 * h)

        # Test: f = (a * b + c) ** 2
        a = Value(1.5)
        b = Value(2.0)
        c = Value(-1.0)

        def test_fn():
            return (a * b + c) ** 2

        result = test_fn()
        result.backward()

        # Compare analytical vs numerical
        for name, val in [("a", a), ("b", b), ("c", c)]:
            num_grad = numerical_gradient(test_fn, [a, b, c], [a, b, c].index(val)
            print(f"d/d{name}: analytical={val.grad:.6f} numerical={num_grad:.6f}")

d/da: analytical=8.000000 numerical=8.000000 match=True
d/db: analytical=6.000000 numerical=6.000000 match=True
d/dc: analytical=4.000000 numerical=4.000000 match=True
```

Part 3 -- Sigmoid vs ReLU: The Vanishing Gradient Problem

② Curriculum Point: ReLU changed everything.

Before ~2011, most neural networks used sigmoid or tanh activations. These functions **squash** their inputs into a bounded range, which means their derivatives are always less than 1. When you multiply many such derivatives together (the chain rule through many layers), the gradient **vanishes exponentially**.

ReLU ($\max(0, x)$) has gradient exactly **0 or 1**. No squashing, no vanishing. This simple change unlocked deep networks.

```
In [8]: # ---- Plot sigmoid, tanh, ReLU and their derivatives ----
        xs = np.linspace(-5, 5, 500)

        # Compute activations
        sigmoid_y = 1 / (1 + np.exp(-xs))
        tanh_y = np.tanh(xs)
        relu_y = np.maximum(0, xs)

        # Compute derivatives
```

```

sigmoid_dy = sigmoid_y * (1 - sigmoid_y)
tanh_dy = 1 - tanh_y ** 2
relu_dy = (xs > 0).astype(float)

fig, axes = plt.subplots(2, 3, figsize=(14, 7))

# Top row: activations
for ax, y, name, color in zip(axes[0],
                              [sigmoid_y, tanh_y, relu_y],
                              ["Sigmoid", "Tanh", "ReLU"],
                              ["#e74c3c", "#2ecc71", "#3498db"]):
    ax.plot(xs, y, color=color, linewidth=2.5)
    ax.set_title(f"{name}(x)", fontsize=14, fontweight="bold")
    ax.axhline(y=0, color="gray", linewidth=0.5)
    ax.axvline(x=0, color="gray", linewidth=0.5)
    ax.set_xlabel("x")
    ax.grid(True, alpha=0.3)

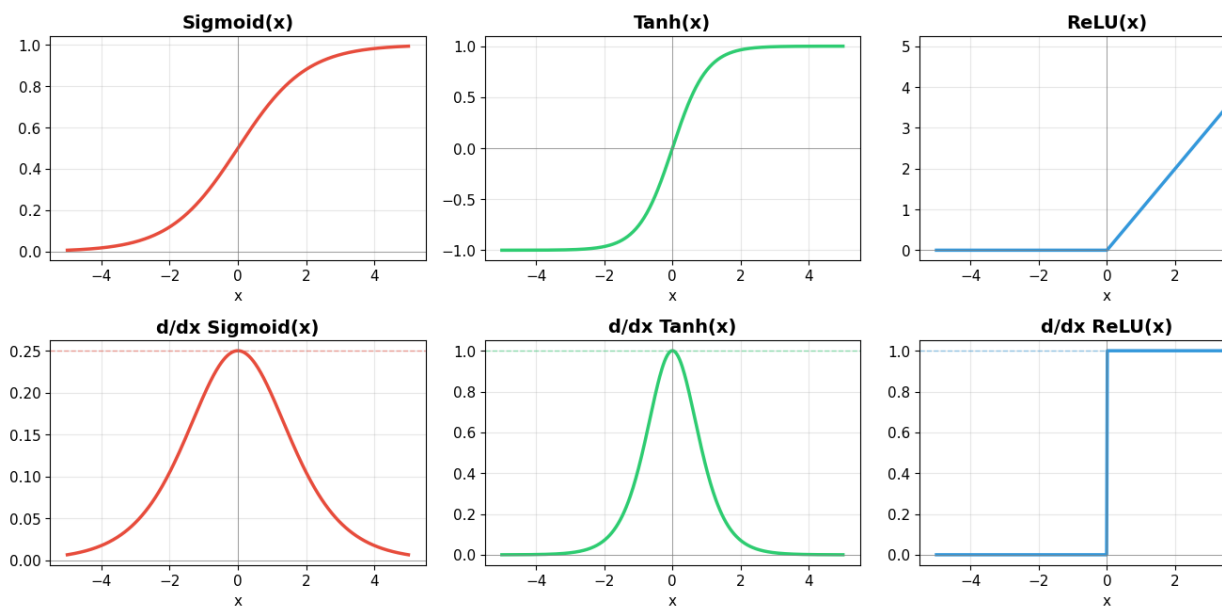
# Bottom row: derivatives
for ax, dy, name, color in zip(axes[1],
                              [sigmoid_dy, tanh_dy, relu_dy],
                              ["Sigmoid", "Tanh", "ReLU"],
                              ["#e74c3c", "#2ecc71", "#3498db"]):
    ax.plot(xs, dy, color=color, linewidth=2.5)
    ax.set_title(f"d/dx {name}(x)", fontsize=14, fontweight="bold")
    ax.axhline(y=0, color="gray", linewidth=0.5)
    ax.axvline(x=0, color="gray", linewidth=0.5)
    ax.set_xlabel("x")
    ax.grid(True, alpha=0.3)
    # Highlight max derivative
    ax.axhline(y=max(dy), color=color, linewidth=1, linestyle="--", alpha=0.5)

fig.suptitle("Activations and Their Derivatives", fontsize=16, fontweight="bold")
plt.tight_layout()
plt.show()

print("KEY OBSERVATION:")
print(f"  Sigmoid max derivative: {max(sigmoid_dy):.4f}  (always < 0.25)")
print(f"  Tanh max derivative:    {max(tanh_dy):.4f}  (always < 1.0)")
print(f"  ReLU derivative:        0 or 1 -- no squashing!")

```


Activations and Their Derivatives



KEY OBSERVATION:

Sigmoid max derivative: 0.2500 (always < 0.25)
 Tanh max derivative: 0.9999 (always < 1.0)
 ReLU derivative: 0 or 1 -- no squashing!

The vanishing gradient problem -- numerically

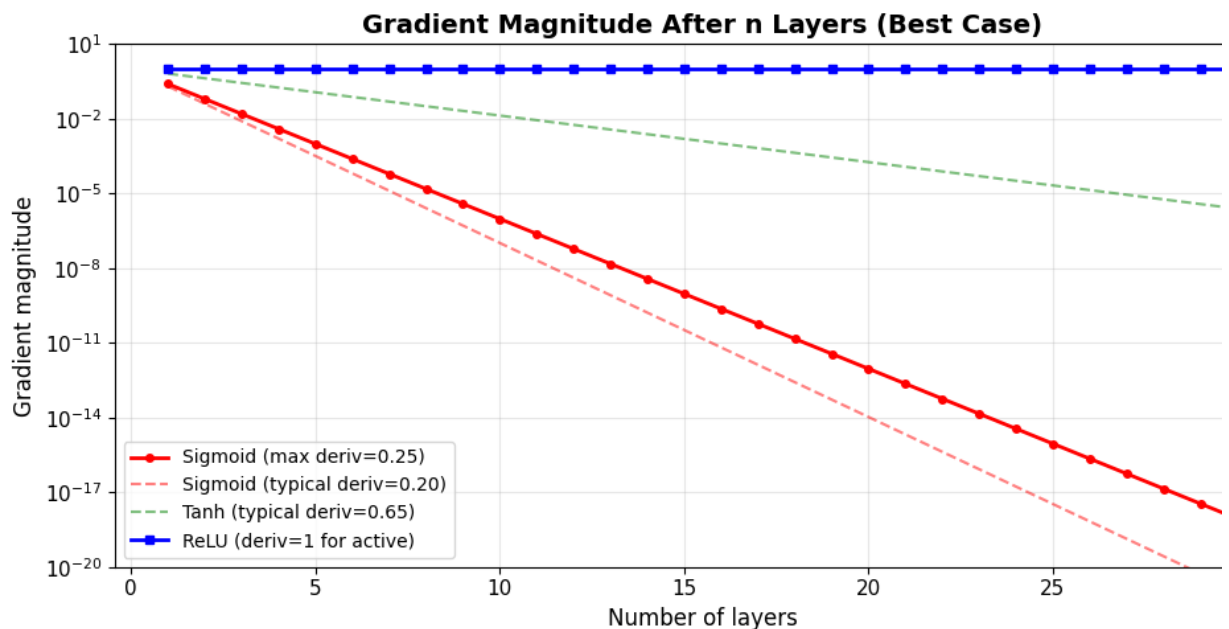
Let's simulate what happens when a gradient passes through n layers. In the best case, each multiplies the gradient by the **maximum** derivative of the activation function.

```
In [9]: # Simulate gradient magnitude after passing through n layers
# We assume "best case" -- gradient at the maximum of each activation's d
layers = np.arange(1, 31)
sigmoid_grad = 0.25 ** layers      # max sigmoid derivative = 0.25
tanh_grad = 1.0 ** layers          # max tanh derivative = 1.0 (but rarely
relu_grad = 1.0 ** layers          # ReLU derivative = 1 for active neurons

# More realistic: average gradient magnitudes
sigmoid_avg_grad = 0.20 ** layers  # typical sigmoid derivative
tanh_avg_grad = 0.65 ** layers     # typical tanh derivative

fig, ax = plt.subplots(figsize=(10, 5))
ax.semilogy(layers, sigmoid_grad, "r-o", markersize=4, label="Sigmoid (max)")
ax.semilogy(layers, sigmoid_avg_grad, "r--", alpha=0.5, label="Sigmoid (typical)")
ax.semilogy(layers, tanh_avg_grad, "g--", alpha=0.5, label="Tanh (typical)")
ax.semilogy(layers, relu_grad, "b-s", markersize=4, label="ReLU (deriv=1)")
ax.set_xlabel("Number of layers", fontsize=12)
ax.set_ylabel("Gradient magnitude", fontsize=12)
ax.set_title("Gradient Magnitude After n Layers (Best Case)", fontsize=14)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
ax.set_ylim(1e-20, 10)
plt.tight_layout()
plt.show()

print(f"After 10 sigmoid layers, gradient shrinks to: {0.25**10:.2e}")
print(f"After 20 sigmoid layers, gradient shrinks to: {0.25**20:.2e}")
print(f"After 10 ReLU layers, gradient remains: {1.0**10:.1f}")
```



After 10 sigmoid layers, gradient shrinks to: 9.54e-07
 After 20 sigmoid layers, gradient shrinks to: 9.09e-13
 After 10 ReLU layers, gradient remains: 1.0

Verify with our autograd engine

Let's stack activations and watch the gradient flow through our actual `Value` engine:

```
In [10]: # Stack 10 sigmoid activations: y = sigmoid(sigmoid(...sigmoid(x)...))
x_sig = Value(0.5, label="x")
y_sig = x_sig
for i in range(10):
    y_sig = y_sig.sigmoid()
y_sig.backward()

# Stack 10 ReLU activations: y = relu(relu(...relu(x)...))
x_relu = Value(0.5, label="x")
y_relu = x_relu
for i in range(10):
    y_relu = y_relu.relu()
y_relu.backward()

print("Gradient at input after 10 stacked activations:")
print(f" 10x Sigmoid: dy/dx = {x_sig.grad:.2e}")
print(f" 10x ReLU:    dy/dx = {x_relu.grad:.2e}")
print()
print("ReLU preserves the gradient perfectly (when active).")
print("Sigmoid crushes it to near-zero -- this is why deep sigmoid networks")
print("were nearly impossible to train before ReLU became standard (~2011)
```

Gradient at input after 10 stacked activations:

10x Sigmoid: dy/dx = 3.48e-07
 10x ReLU: dy/dx = 1.00e+00

ReLU preserves the gradient perfectly (when active).
 Sigmoid crushes it to near-zero -- this is why deep sigmoid networks
 were nearly impossible to train before ReLU became standard (~2011).

Part 4 -- Building a Neural Network from Value Object

Now we turn our autograd engine into a full neural network library. We build three classes, each the one below:

```
MLP  (Multi-Layer Perceptron)
  -> Layer  (a list of neurons)
      -> Neuron  (weighted sum + bias + activation)
          -> Value  (our autograd scalar)
```

```
In [11]: class Neuron:
    """A single neuron: weighted sum + bias + activation."""

    def __init__(self, n_in, activation="relu"):
        # Initialize weights from uniform[-1, 1] -- small random values
        self.w = [Value(random.uniform(-1, 1), label=f"w{i}") for i in range(n_in)]
        self.b = Value(0.0, label="b")
        self.activation = activation

    def __call__(self, x):
        # Weighted sum: w0*x0 + w1*x1 + ... + b
        act = sum((wi * xi for wi, xi in zip(self.w, x)), self.b)
        # Apply activation
        if self.activation == "relu":
            return act.relu()
        elif self.activation == "sigmoid":
            return act.sigmoid()
        elif self.activation == "tanh":
            return act.tanh()
        elif self.activation == "linear":
            return act
        else:
            raise ValueError(f"Unknown activation: {self.activation}")

    def parameters(self):
        return self.w + [self.b]

class Layer:
    """A layer of neurons."""

    def __init__(self, n_in, n_out, activation="relu"):
        self.neurons = [Neuron(n_in, activation) for _ in range(n_out)]

    def __call__(self, x):
        outs = [n(x) for n in self.neurons]
        return outs[0] if len(outs) == 1 else outs

    def parameters(self):
        return [p for n in self.neurons for p in n.parameters()]

class MLP:
    """Multi-layer perceptron."""

    def __init__(self, n_in, layer_sizes, activations=None):
```

```

        sizes = [n_in] + layer_sizes
        if activations is None:
            # ReLU for hidden layers, linear for output
            activations = ["relu"] * (len(layer_sizes) - 1) + ["linear"]
        self.layers = [Layer(sizes[i], sizes[i + 1], activations[i])
                        for i in range(len(layer_sizes))]

    def __call__(self, x):
        for layer in self.layers:
            x = layer(x)
        return x

    def parameters(self):
        return [p for layer in self.layers for p in layer.parameters()]

    def zero_grad(self):
        for p in self.parameters():
            p.grad = 0.0

# Quick test
random.seed(42)
net = MLP(2, [4, 4, 1])
x_test = [Value(1.0), Value(2.0)]
y_test = net(x_test)
print(f"Test forward pass: input=[1.0, 2.0] -> output={y_test.data:.4f}")
print(f"Total parameters: {len(net.parameters())}")

```

Test forward pass: input=[1.0, 2.0] -> output=-0.7102
 Total parameters: 37

Part 5 -- Training on the Moons Dataset

We'll use scikit-learn's `make_moons` to generate a non-linearly-separable 2D dataset, then train MLP to classify it. This demonstrates the full training loop:

1. Forward pass (compute predictions)
2. Loss computation (hinge loss + L2 regularization)
3. Backward pass (compute all gradients)
4. Parameter update (gradient descent)

Note: Our scalar autograd is intentionally slow -- this is pedagogical code. We use a small dataset (points) and small network (8 -> 8 -> 1) to keep runtime under 30s.

```

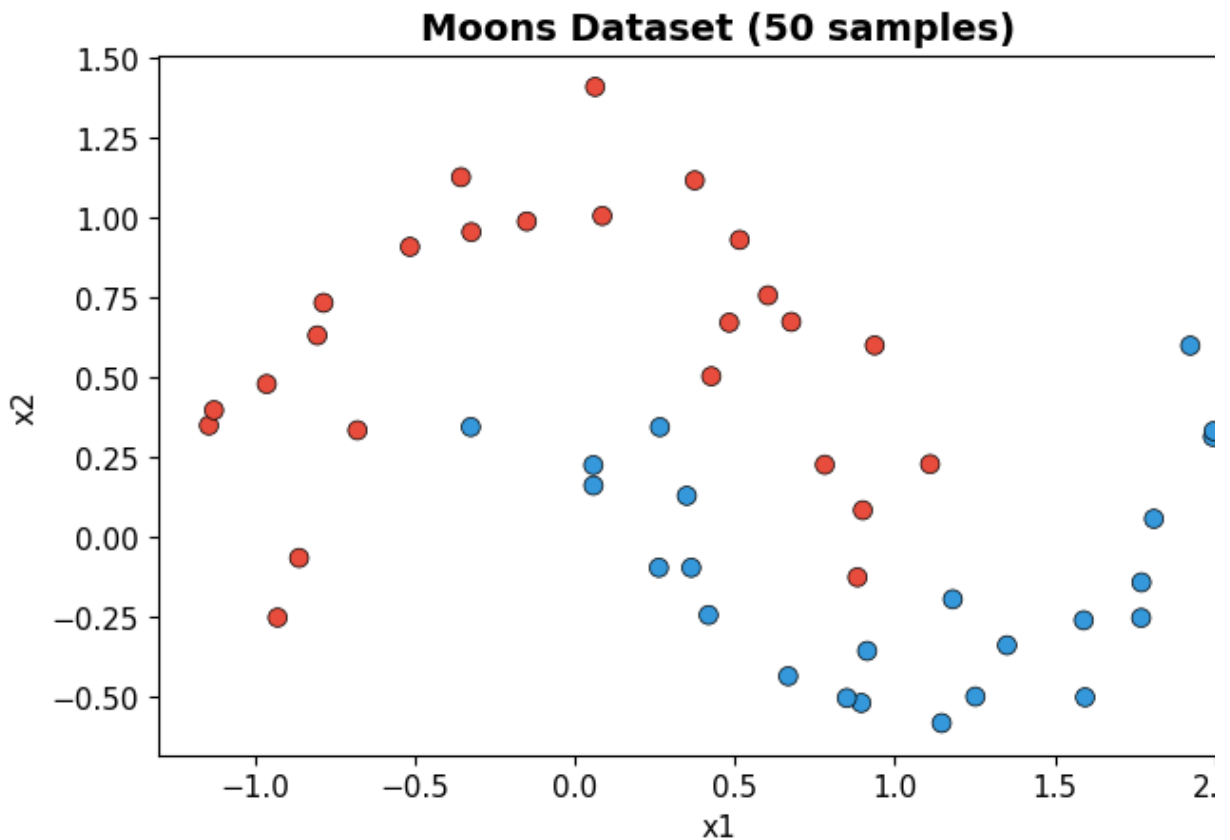
In [12]: from sklearn.datasets import make_moons

# Generate data -- deliberately small for our scalar engine
X, y = make_moons(n_samples=50, noise=0.15, random_state=42)
# Convert labels from {0, 1} to {-1, +1} for hinge loss
y = 2 * y - 1

fig, ax = plt.subplots(figsize=(7, 5))
colors = ["#e74c3c" if yi == -1 else "#3498db" for yi in y]
ax.scatter(X[:, 0], X[:, 1], c=colors, s=50, edgecolors="k", linewidth=0.5)
ax.set_xlabel("x1")

```

```
ax.set_ylabel("x2")
ax.set_title("Moons Dataset (50 samples)", fontsize=14, fontweight="bold")
ax.set_aspect("equal")
plt.tight_layout()
plt.show()
```



Training loop

We'll use a simple **hinge loss** (like an SVM) plus L2 regularization:

$$\mathcal{L} = \frac{1}{N} \sum_i \max(0, 1 - y_i \hat{y}_i) + \lambda \sum_j w_j^2$$

Each epoch:

1. Forward pass for all samples
2. Compute loss
3. Zero gradients, then backward
4. Update parameters with gradient descent

```
In [13]: # Train the network on moons data
random.seed(42)
np.random.seed(42)

# Build network: 2 inputs -> 8 -> 8 -> 1 output
model = MLP(2, [8, 8, 1])
print(f"Parameters: {len(model.parameters())}")

# Hyperparameters
learning_rate = 0.1
reg_lambda = 1e-4
epochs = 30 # Keep low -- scalar autograd is slow by design
```

```

# Storage for decision boundary snapshots
snapshots = {}
loss_history = []

for epoch in range(epochs):
    # Forward pass -- compute predictions for all samples
    predictions = [model([Value(x[0]), Value(x[1])]) for x in X]

    # Hinge loss: max(0, 1 - y * yhat)
    losses = [(1 + Value(-yi) * pred).relu() for yi, pred in zip(y, predictions)]
    data_loss = sum(losses, Value(0.0)) * (1.0 / len(losses))

    # L2 regularization
    reg_loss = sum((p * p for p in model.parameters()), Value(0.0)) * reg
    total_loss = data_loss + reg_loss

    # Backward pass
    model.zero_grad()
    total_loss.backward()

    # Gradient descent update
    for p in model.parameters():
        p.data -= learning_rate * p.grad

    # Accuracy
    accuracy = sum(1 for pred, yi in zip(predictions, y)
                   if (pred.data > 0) == (yi > 0)) / len(y)

    loss_history.append(total_loss.data)

    # Save snapshots for visualization
    if epoch in [0, 4, 9, 19, 29]:
        # Save parameter values at this epoch
        snapshots[epoch] = [p.data for p in model.parameters()]

    if epoch % 5 == 0 or epoch == epochs - 1:
        print(f"Epoch {epoch:3d} | Loss: {total_loss.data:.4f} | Accu

print("\nTraining complete!")

```

Parameters: 105

Epoch	0		Loss: 0.9290		Accuracy: 66%
Epoch	5		Loss: 0.8802		Accuracy: 68%
Epoch	10		Loss: 0.8281		Accuracy: 68%
Epoch	15		Loss: 0.7725		Accuracy: 72%
Epoch	20		Loss: 0.6764		Accuracy: 72%
Epoch	25		Loss: 0.5216		Accuracy: 74%
Epoch	29		Loss: 0.4662		Accuracy: 78%

Training complete!

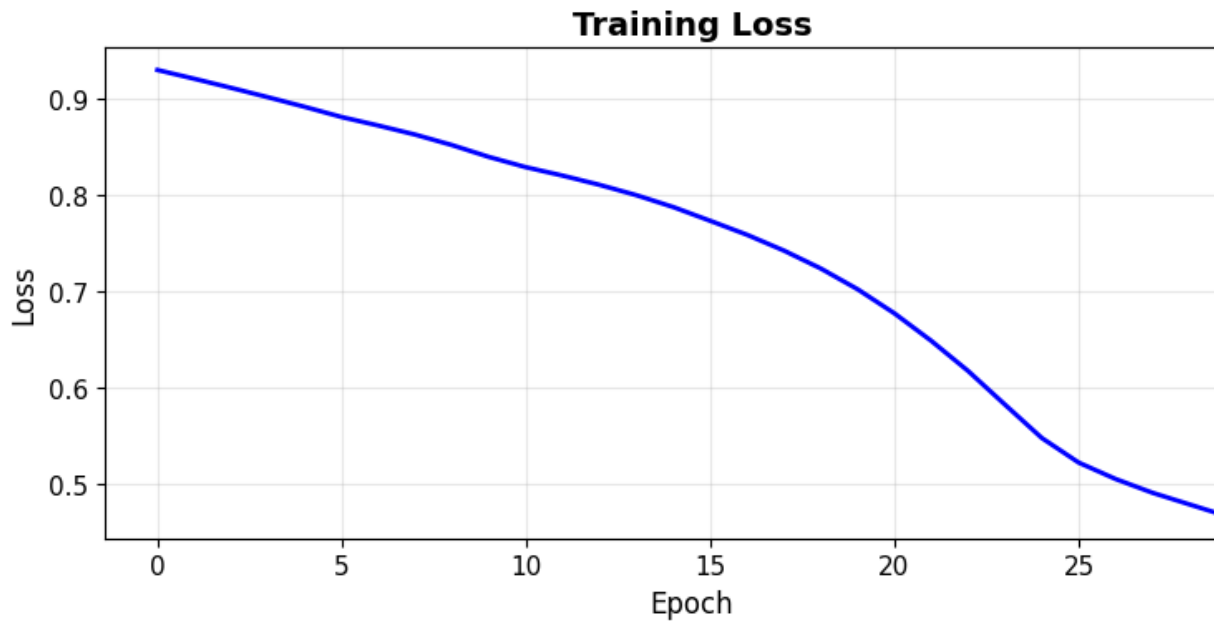
Loss curve

```

In [14]: fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(loss_history, "b-", linewidth=2)
ax.set_xlabel("Epoch", fontsize=12)
ax.set_ylabel("Loss", fontsize=12)
ax.set_title("Training Loss", fontsize=14, fontweight="bold")

```

```
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Decision boundary evolution

Let's see how the model's decision boundary changes during training. We need to rebuild the model from each saved snapshot to visualize it.

```
In [15]: def plot_decision_boundary(model, X, y, ax, title=""):
    """Plot the decision boundary of a binary classifier."""
    h = 0.1 # Grid resolution
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))
    grid_points = np.c_[xx.ravel(), yy.ravel()]

    # Predict on grid
    Z = []
    for pt in grid_points:
        pred = model([Value(pt[0]), Value(pt[1])])
        Z.append(pred.data)
    Z = np.array(Z).reshape(xx.shape)

    # Plot
    ax.contourf(xx, yy, Z, levels=[-1e9, 0, 1e9], colors=["#ffcccc", "#ccffcc"])
    ax.contour(xx, yy, Z, levels=[0], colors=["k"], linewidths=2)
    colors = ["#e74c3c" if yi == -1 else "#3498db" for yi in y]
    ax.scatter(X[:, 0], X[:, 1], c=colors, s=30, edgecolors="k", linewidth=1)
    ax.set_title(title, fontsize=11, fontweight="bold")
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)
    ax.set_aspect("equal")

    # Rebuild model at each snapshot and plot
    snapshot_epochs = sorted(snapshots.keys())
    fig, axes = plt.subplots(1, len(snapshot_epochs), figsize=(4 * len(snapshot_epochs), 4 * len(snapshot_epochs)))
```

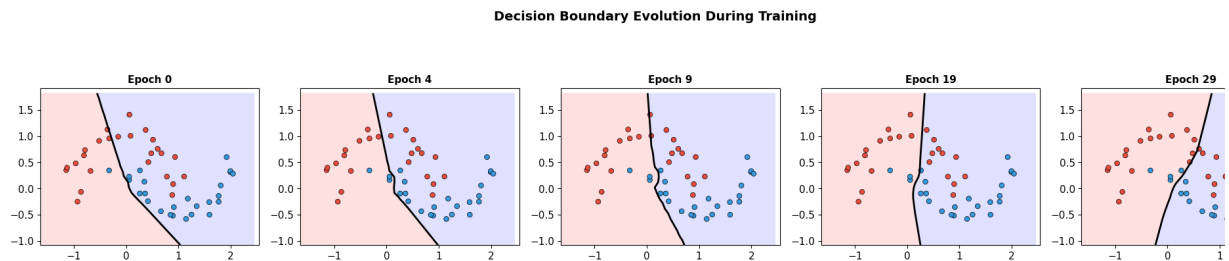
```

if len(snapshot_epochs) == 1:
    axes = [axes]

for ax, ep in zip(axes, snapshot_epochs):
    # Rebuild model with saved parameters
    random.seed(42)
    snap_model = MLP(2, [8, 8, 1])
    for p, saved_val in zip(snap_model.parameters(), snapshots[ep]):
        p.data = saved_val
    plot_decision_boundary(snap_model, X, y, ax, title=f"Epoch {ep}")

fig.suptitle("Decision Boundary Evolution During Training", fontsize=14,
plt.tight_layout()
plt.show()

```



Part 6 -- Universal Approximation Theorem

① Curriculum Point: Universal Approximation Theorem.

Theorem (informal): A neural network with a single hidden layer and enough neurons c approximate *any* continuous function on a bounded domain to arbitrary accuracy.

But: "enough neurons" might mean an astronomical number. In practice, **deeper networks** achieve the same quality with far fewer total parameters. Depth buys you **compositionality** -- each layer learns features on top of the previous layer's features.

Target function

We'll try to learn: $f(x) = \sin(2x) + 0.5 \cos(5x)$ on $[-\pi, \pi]$. This is a complex, non-trivial function that requires significant capacity.

```

In [16]: # Target function
def target_fn(x):
    return math.sin(2 * x) + 0.5 * math.cos(5 * x)

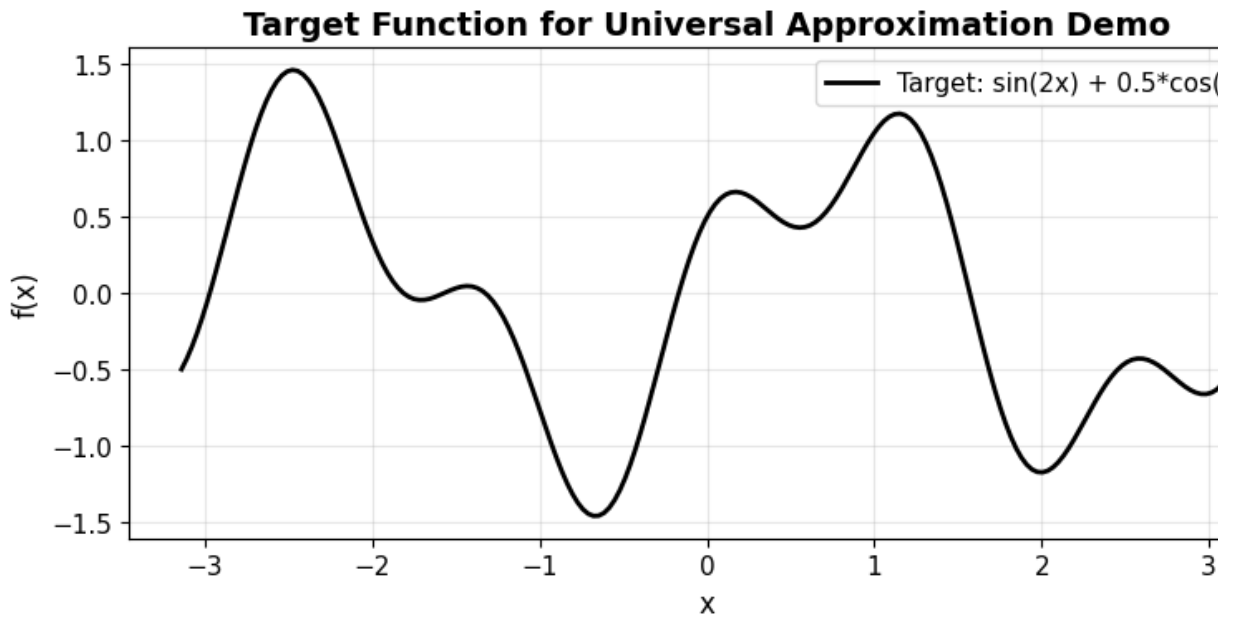
# Plot the target
xs_plot = np.linspace(-math.pi, math.pi, 200)
ys_plot = [target_fn(x) for x in xs_plot]

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(xs_plot, ys_plot, "k-", linewidth=2, label="Target: sin(2x) + 0.5 cos(5x)")
ax.set_xlabel("x", fontsize=12)
ax.set_ylabel("f(x)", fontsize=12)
ax.set_title("Target Function for Universal Approximation Demo", fontsize=12)
ax.legend(fontsize=11)

```



```
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Training helper

We'll write a reusable training function so we can compare architectures fairly.

Note: We use only 20 training points and keep epochs low for speed. Even with these constraint difference between wide/shallow and deep/narrow architectures is clearly visible.

```
In [17]: def train_1d_model(model, x_train, y_train, epochs=200, lr=0.01):
    """Train a 1D regression model using MSE loss."""
    loss_history = []
    for epoch in range(epochs):
        # Forward pass
        predictions = [model([Value(x)]) for x in x_train]
        targets = [Value(yt) for yt in y_train]

        # MSE loss
        losses = [(pred - tgt) ** 2 for pred, tgt in zip(predictions, targets)]
        loss = sum(losses, Value(0.0)) * (1.0 / len(losses))

        # Backward
        model.zero_grad()
        loss.backward()

        # Update
        for p in model.parameters():
            p.data -= lr * p.grad

        loss_history.append(loss.data)

    if epoch % 50 == 0:
        print(f" Epoch {epoch:3d} Loss: {loss.data:.6f}")

    return loss_history
```

```
def predict_1d(model, xs):
    """Predict for an array of x values."""
    return [model([Value(x)]).data for x in xs]
```

Experiment 1: Wide shallow network (1 hidden layer, 32 neurons)

```
In [18]: # Generate training data (20 points)
random.seed(42)
np.random.seed(42)
x_train = np.linspace(-math.pi, math.pi, 20)
y_train = [target_fn(x) for x in x_train]

# Wide shallow: 1 -> 32 -> 1
print("Training WIDE SHALLOW network (1 -> 32 -> 1)...")
random.seed(42)
wide_model = MLP(1, [32, 1], activations=["relu", "linear"])
print(f" Parameters: {len(wide_model.parameters())}")
wide_loss = train_1d_model(wide_model, x_train, y_train, epochs=200, lr=0
```

Training WIDE SHALLOW network (1 -> 32 -> 1)...

```
Parameters: 97
Epoch 0 Loss: 10.755298
Epoch 50 Loss: 0.556638
Epoch 100 Loss: 0.477699
Epoch 150 Loss: 0.429315
```

Experiment 2: Deep narrow network (3 hidden layers, 8 neurons each)

```
In [19]: # Deep narrow: 1 -> 8 -> 8 -> 8 -> 1
print("Training DEEP NARROW network (1 -> 8 -> 8 -> 8 -> 1)...")
random.seed(42)
deep_model = MLP(1, [8, 8, 8, 1], activations=["relu", "relu", "relu", "linear"])
print(f" Parameters: {len(deep_model.parameters())}")
deep_loss = train_1d_model(deep_model, x_train, y_train, epochs=200, lr=0
```

Training DEEP NARROW network (1 -> 8 -> 8 -> 8 -> 1)...

```
Parameters: 169
Epoch 0 Loss: 1.434629
Epoch 50 Loss: 0.519712
Epoch 100 Loss: 0.502747
Epoch 150 Loss: 0.486791
```

Comparison

```
In [20]: wide_preds = predict_1d(wide_model, xs_plot)
deep_preds = predict_1d(deep_model, xs_plot)

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

# Left: wide model fit
axes[0].plot(xs_plot, ys_plot, "k-", linewidth=2, label="Target")
axes[0].plot(xs_plot, wide_preds, "r-", linewidth=2, alpha=0.8, label="Wide Shallow")
axes[0].scatter(x_train, y_train, c="black", s=30, zorder=5, label="Train")
axes[0].set_title(f"Wide Shallow (32 neurons)\n{len(wide_model.parameters())} parameters")
axes[0].legend(fontsize=9)
axes[0].grid(True, alpha=0.3)

# Middle: deep model fit
```

```

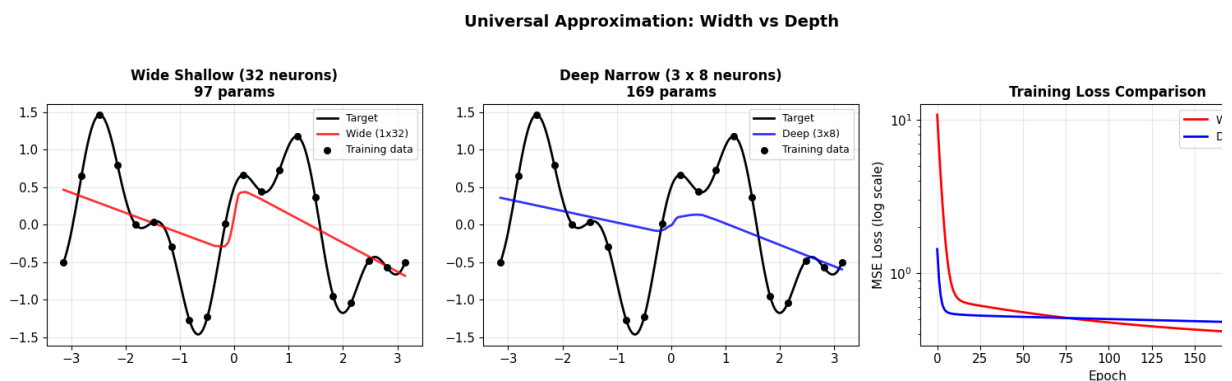
axes[1].plot(xs_plot, ys_plot, "k-", linewidth=2, label="Target")
axes[1].plot(xs_plot, deep_preds, "b-", linewidth=2, alpha=0.8, label="Deep")
axes[1].scatter(x_train, y_train, c="black", s=30, zorder=5, label="Train")
axes[1].set_title(f"Deep Narrow (3 x 8 neurons)\n{len(deep_model.parameters())}")
axes[1].legend(fontsize=9)
axes[1].grid(True, alpha=0.3)

# Right: loss comparison
axes[2].semilogy(wide_loss, "r-", linewidth=2, label="Wide (1x32)")
axes[2].semilogy(deep_loss, "b-", linewidth=2, label="Deep (3x8)")
axes[2].set_xlabel("Epoch")
axes[2].set_ylabel("MSE Loss (log scale)")
axes[2].set_title("Training Loss Comparison", fontsize=12, fontweight="bold")
axes[2].legend(fontsize=10)
axes[2].grid(True, alpha=0.3)

fig.suptitle("Universal Approximation: Width vs Depth", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()

print("KEY INSIGHT:")
print(f" Wide model (32 neurons, 1 layer): {len(wide_model.parameters())}")
print(f" Deep model (8 neurons, 3 layers): {len(deep_model.parameters())}")
print(" Both can approximate the target, but depth enables composition of features.")
print(" Deeper networks learn hierarchical representations -- each layer builds on the last.")

```



KEY INSIGHT:

Wide model (32 neurons, 1 layer): 97 parameters

Deep model (8 neurons, 3 layers): 169 parameters

Both can approximate the target, but depth enables composition of features.

Deeper networks learn hierarchical representations -- each layer builds on the last.

① Universal Approximation in Practice

The theorem guarantees that a wide-enough single layer *can* fit any function, but it says nothing about how *efficiently*. In practice:

- **Wide + shallow** needs many neurons and may not generalize well
- **Deep + narrow** learns compositional features and often generalizes better
- This is why modern networks are deep, not just wide

In Notebook 2 we'll see this play out with real image data (MNIST).

Part 7 -- Exercises

Work through these exercises to solidify your understanding. Each exercise has a solution below it yourself first!

Exercise 1: Hand-Compute and Verify Gradients

Task: For the expression $f = (a \cdot b + c)^2$ where $a=3$, $b=-2$, $c=5$:

1. Compute f by hand
2. Compute $\frac{\partial f}{\partial a}$, $\frac{\partial f}{\partial b}$, $\frac{\partial f}{\partial c}$ by hand
3. Verify with our `Value` engine

Hints:

- $f = (3 \cdot (-2) + 5)^2 = (-6 + 5)^2 = 1$
- Use the chain rule: $\frac{\partial f}{\partial a} = 2(ab+c) \cdot b$

```
In [21]: # ===== EXERCISE 1 -- Try it yourself first! =====
#
# Hand computation:
#   f = (a*b + c)^2 = (3*(-2) + 5)^2 = (-1)^2 = 1
#   Let u = a*b + c = -1
#   df/du = 2*u = -2
#   du/da = b = -2  -> df/da = df/du * du/da = (-2)*(-2) = 4
#   du/db = a = 3    -> df/db = df/du * du/db = (-2)*(3) = -6
#   du/dc = 1        -> df/dc = df/du * du/dc = (-2)*(1) = -2
#
# --- Solution ---

a = Value(3.0, label="a")
b = Value(-2.0, label="b")
c = Value(5.0, label="c")

f = (a * b + c) ** 2
f.backward()

print("f = (a * b + c)^2 where a=3, b=-2, c=5")
print(f"  f      = {f.data:.1f} (expected: 1)")
print(f"  df/da = {a.grad:.1f} (expected: 4)")
print(f"  df/db = {b.grad:.1f} (expected: -6)")
print(f"  df/dc = {c.grad:.1f} (expected: -2)")
```

```
f = (a * b + c)^2 where a=3, b=-2, c=5
f      = 1.0 (expected: 1)
df/da = 4.0 (expected: 4)
df/db = -6.0 (expected: -6)
df/dc = -2.0 (expected: -2)
```

Exercise 2: Implement and Test Leaky ReLU

Task: Add a `leaky_relu(alpha=0.01)` method to the `Value` class and verify it works.

Leaky ReLU: $f(x) = x$ if $x > 0$, else $\alpha \cdot x$

Its derivative: $f'(x) = 1$ if $x > 0$, else α

This fixes the "dead neuron" problem of standard ReLU (where neurons with negative inputs have no gradient and can never recover).

```
In [22]: # ===== EXERCISE 2 -- Try it yourself first! =====
# --- Solution ---

def leaky_relu(self, alpha=0.01):
    """Leaky ReLU: max(alpha*x, x)"""
    out_val = self.data if self.data > 0 else alpha * self.data
    out = Value(out_val, (self,), "leaky_relu")
    def _backward():
        self.grad += (1.0 if self.data > 0 else alpha) * out.grad
    out._backward = _backward
    return out

# Monkey-patch it onto Value
Value.leaky_relu = leaky_relu

# Test with positive input
x = Value(3.0)
y = x.leaky_relu()
y.backward()
print(f"leaky_relu(3.0) = {y.data:.4f} (expected: 3.0)")
print(f" gradient = {x.grad:.4f} (expected: 1.0)")

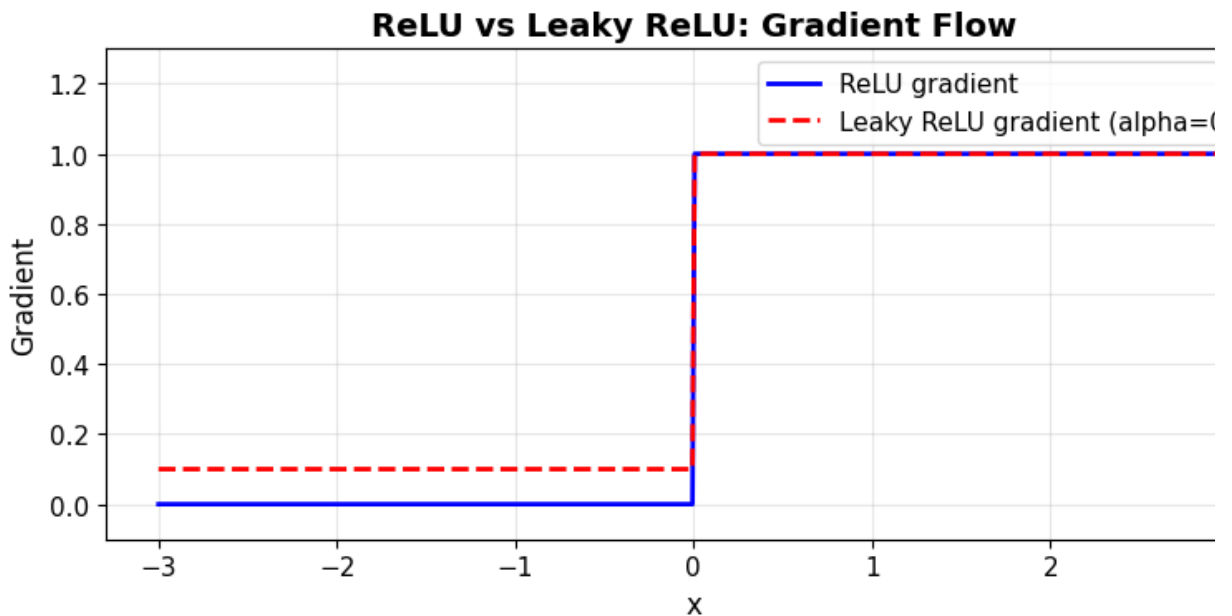
# Test with negative input
x = Value(-2.0)
y = x.leaky_relu(alpha=0.1)
y.backward()
print(f"leaky_relu(-2.0, alpha=0.1) = {y.data:.4f} (expected: -0.2)")
print(f" gradient = {x.grad:.4f} (expected: 0.1)")

# Compare gradients: plot relu vs leaky_relu derivatives
xs = np.linspace(-3, 3, 500)
relu_grads = [1.0 if x > 0 else 0.0 for x in xs]
leaky_grads = [1.0 if x > 0 else 0.1 for x in xs]

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(xs, relu_grads, "b-", linewidth=2, label="ReLU gradient")
ax.plot(xs, leaky_grads, "r--", linewidth=2, label="Leaky ReLU gradient")
ax.set_xlabel("x", fontsize=12)
ax.set_ylabel("Gradient", fontsize=12)
ax.set_title("ReLU vs Leaky ReLU: Gradient Flow", fontsize=14, fontweight="bold")
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
ax.set_ylim(-0.1, 1.3)
plt.tight_layout()
plt.show()
```

```
print("\nLeaky ReLU lets gradients flow even for negative inputs.")
print("This prevents dead neurons that can never recover.")
```

```
leaky_relu(3.0) = 3.0000 (expected: 3.0)
gradient = 1.0000 (expected: 1.0)
leaky_relu(-2.0, alpha=0.1) = -0.2000 (expected: -0.2)
gradient = 0.1000 (expected: 0.1)
```



Leaky ReLU lets gradients flow even for negative inputs.
This prevents dead neurons that can never recover.

Exercise 3: Compare Training with Sigmoid vs ReLU Activations

Task: Train two identical networks on the moons dataset -- one using sigmoid activations and on ReLU activations. Compare their loss curves and final accuracy.

```
In [23]: # ===== EXERCISE 3 -- Try it yourself first! =====
# --- Solution ---

from sklearn.datasets import make_moons

X_ex, y_ex = make_moons(n_samples=50, noise=0.15, random_state=42)
y_ex = 2 * y_ex - 1 # Convert to {-1, +1}

def train_classifier(model, X, y, epochs=20, lr=0.1):
    """Train a binary classifier and return loss history."""
    losses_hist = []
    for epoch in range(epochs):
        preds = [model([Value(x[0]), Value(x[1])]) for x in X]
        data_loss = sum((1 + Value(-yi) * pred).relu()
                        for yi, pred in zip(y, preds))
        data_loss = data_loss * (1.0 / len(y))
        reg = sum((p * p for p in model.parameters()), Value(0.0)) * 1e-4
        total = data_loss + reg

        model.zero_grad()
        total.backward()
        for p in model.parameters():
            p.data -= lr * p.grad
```

```

        losses_hist.append(total.data)
        acc = sum(1 for pred, yi in zip(preds, y)
                  if (pred.data > 0) == (yi > 0)) / len(y)
        if epoch % 5 == 0 or epoch == epochs - 1:
            print(f" Epoch {epoch:2d}: loss={total.data:.4f} acc={acc:.4f}")
    return losses_hist

# Train with ReLU
print("=== ReLU Network ===")
random.seed(42)
relu_net = MLP(2, [8, 8, 1], activations=["relu", "relu", "linear"])
relu_losses = train_classifier(relu_net, X_ex, y_ex, epochs=20, lr=0.1)

# Train with Sigmoid
print("\n=== Sigmoid Network ===")
random.seed(42)
sig_net = MLP(2, [8, 8, 1], activations=["sigmoid", "sigmoid", "linear"])
sig_losses = train_classifier(sig_net, X_ex, y_ex, epochs=20, lr=0.5)

# Plot comparison
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(relu_losses, "b-", linewidth=2, label="ReLU")
ax.plot(sig_losses, "r-", linewidth=2, label="Sigmoid")
ax.set_xlabel("Epoch", fontsize=12)
ax.set_ylabel("Loss", fontsize=12)
ax.set_title("ReLU vs Sigmoid: Training Loss on Moons", fontsize=14, fontweight="bold")
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\nReLU typically converges faster and more reliably than sigmoid,"
      "especially in deeper networks where sigmoid suffers from vanishing")

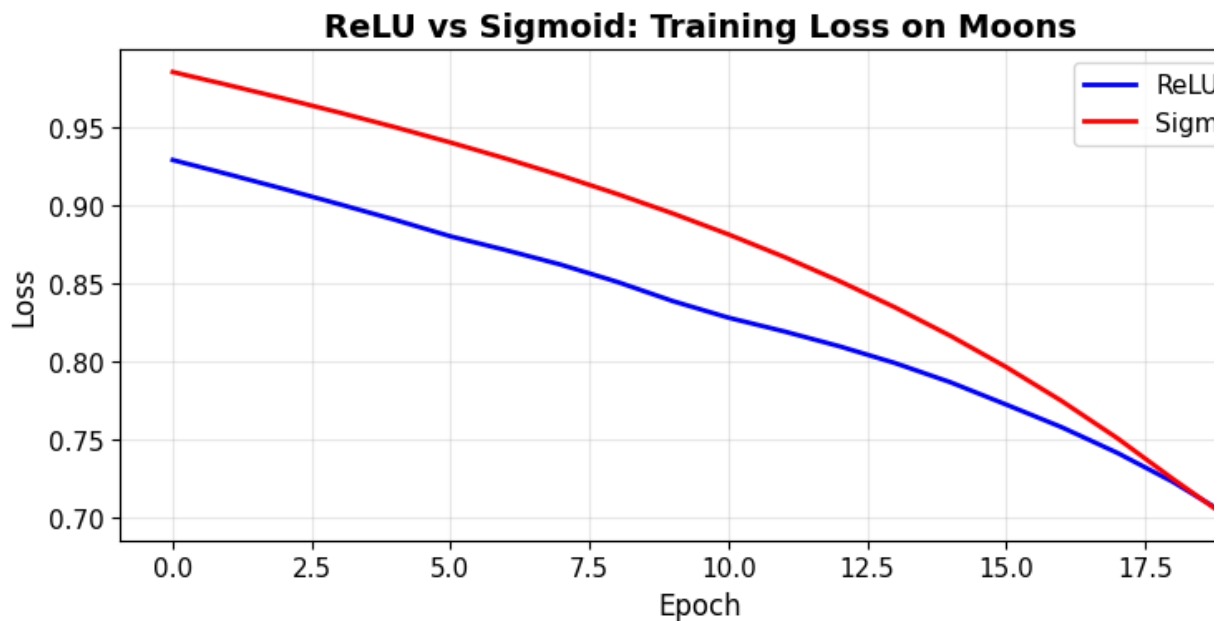
```

=== ReLU Network ===

Epoch 0:	loss=0.9290	acc=66%
Epoch 5:	loss=0.8802	acc=68%
Epoch 10:	loss=0.8281	acc=68%
Epoch 15:	loss=0.7725	acc=72%
Epoch 19:	loss=0.7015	acc=72%

=== Sigmoid Network ===

Epoch 0:	loss=0.9853	acc=52%
Epoch 5:	loss=0.9402	acc=60%
Epoch 10:	loss=0.8814	acc=72%
Epoch 15:	loss=0.7965	acc=78%
Epoch 19:	loss=0.6999	acc=76%



ReLU typically converges faster and more reliably than sigmoid, especially in deeper networks where sigmoid suffers from vanishing gradients.

Summary

In this notebook we built a complete **automatic differentiation engine** from scratch and used it to construct and train neural networks. Here's what we covered:

Section	What We Built	Key Insight
Value class	Scalar autograd with +, *, /, **, relu, sigmoid, tanh	Each op stores its local derivative
Gradient verification	Hand-computed vs engine gradients	⑤ Backprop = chain rule
Sigmoid vs ReLU	Gradient flow comparison	② ReLU changed everything
Neural network	Neuron, Layer, MLP classes	Networks are just nested Value computations
Moons training	Full training loop with decision boundary	SGD + backprop learns nonlinear boundaries
Universal approximation	Wide vs deep comparison	① UAT: width works, depth is

What Comes Next

In **Notebook 2** we rebuild everything with matrix operations (no more scalar autograd) and explore training dynamics: initialization (He/Xavier), learning rate schedules, loss landscapes, and overparameterization.

Notebook 2: Training Dynamics from Scratch

From Scalar Autograd to Real Matrix-Based Networks

In Notebook 1, we built a scalar autograd engine — the `Value` class — and used it to train a tiny neural network on a 2D toy dataset. That was great for understanding the *mechanics* of backpropagation, but real neural networks don't operate on individual scalars. They operate on **matrices** of activations and weights, leveraging NumPy's (or PyTorch's/JAX's) optimized linear algebra routines for speed.

In this notebook, we move from "scalar autograd as a learning tool" to **real matrix-based neural networks** implemented entirely in NumPy. We will:

1. **Build a matrix-based MLP** with manual forward pass, cross-entropy loss, and full backpropagation — all in NumPy matrix form.
2. **Train on MNIST digits** — a real image classification task.
3. **Run experiments** that reveal deep truths about how neural networks actually learn:
 - Why **initialization** (He vs Xavier vs naive) matters enormously
 - Why **vanishing gradients** plagued early deep learning
 - Why **overparameterized** models generalize despite having far more parameters than data
 - What the **loss landscape** actually looks like

Curriculum Points Covered

#	Concept	Where
③	Overparameterization helps	Section 6
⑥	Geometry matters (loss landscape)	Section 7
⑦	Capacity ≠ performance	Sections 4 & 5

Prerequisites: Notebook 1 (scalar autograd, backprop basics, chain rule). **Tools:** Python + NumPy + matplotlib + sklearn (data loading only). No PyTorch.

2. Matrix-Based MLP in NumPy

Why matrices?

In Notebook 1, each weight was a `Value` object and gradients flowed through individual scalar operations. That's pedagogically perfect but computationally slow. Real networks compute $\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ as a single matrix multiply, which NumPy (and GPU) can execute orders of magnitude faster.

The plan

We'll implement:

- **Forward pass:** matrix multiplications + activation functions
- **Cross-entropy loss:** the standard loss for classification
- **Backward pass:** manual gradient computation using matrix calculus
- **SGD with momentum:** a simple but effective optimizer

All layer operations follow this pattern for a layer l :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$

where $\mathbf{a}^{(0)} = \mathbf{x}$ is the input.

2.1 Activation Functions and Their Derivatives

Before building the network, let's implement our activation functions. We need both the function & derivative for backpropagation.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Reproducibility
np.random.seed(42)

# --- Activation functions ---
# Each returns the activation value. Derivatives are separate functions f

def relu(z):
    """ReLU: max(0, z). Derivative is 0 or 1."""
    return np.maximum(0, z)

def relu_deriv(z):
    """Derivative of ReLU w.r.t. its input z."""
    return (z > 0).astype(float)

def sigmoid(z):
    """Sigmoid: 1/(1+exp(-z)). Numerically stable version."""
    z_safe = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z_safe))

def sigmoid_deriv(z):
    """Derivative of sigmoid: sig(z) * (1 - sig(z))."""
    s = sigmoid(z)
    return s * (1.0 - s)

def tanh_act(z):
    """Tanh activation."""
    return np.tanh(z)
```

```

def tanh_deriv(z):
    """Derivative of tanh: 1 - tanh(z)^2."""
    t = np.tanh(z)
    return 1.0 - t ** 2

# Quick visual check
z = np.linspace(-4, 4, 200)
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

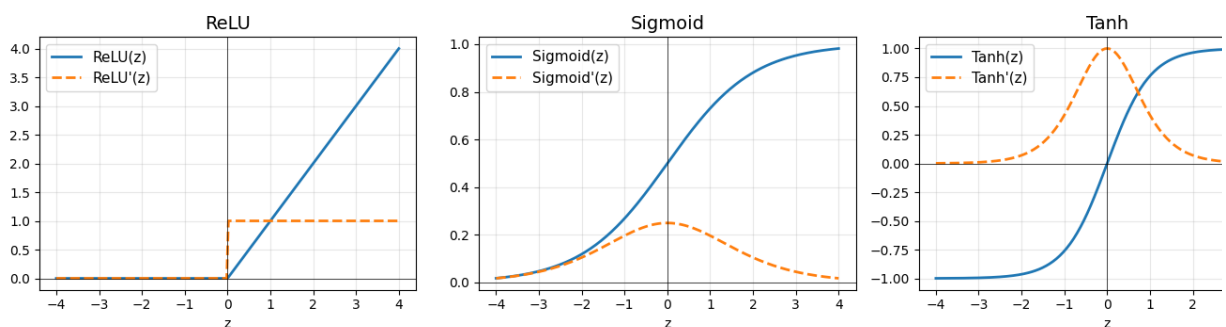
for ax, fn, dfn, name in [
    (axes[0], relu, relu_deriv, "ReLU"),
    (axes[1], sigmoid, sigmoid_deriv, "Sigmoid"),
    (axes[2], tanh_act, tanh_deriv, "Tanh"),
]:
    ax.plot(z, fn(z), label=f"{name}(z)", linewidth=2)
    ax.plot(z, dfn(z), '--', label=f"{name}'(z)", linewidth=2)
    ax.set_title(name, fontsize=14)
    ax.legend(fontsize=11)
    ax.grid(True, alpha=0.3)
    ax.axhline(0, color='k', linewidth=0.5)
    ax.axvline(0, color='k', linewidth=0.5)
    ax.set_xlabel("z")

plt.suptitle("Activation Functions and Their Derivatives", fontsize=15, y
plt.tight_layout()
plt.show()

print("Notice: Sigmoid derivative peaks at 0.25 -- this is why gradients
print("ReLU derivative is exactly 0 or 1 -- gradients flow unchanged thro

```

Activation Functions and Their Derivatives



Notice: Sigmoid derivative peaks at 0.25 -- this is why gradients vanish in d
sigmoid nets!

ReLU derivative is exactly 0 or 1 -- gradients flow unchanged through active
neurons.

2.2 Softmax and Cross-Entropy Loss

For multi-class classification, we use **softmax** to convert raw scores (logits) into probabilities, the **cross-entropy** to measure how far our predictions are from the truth.

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \log p_{y_n}$$

A beautiful fact: the gradient of cross-entropy loss w.r.t. the logits $\mathbf{z}$ is simply:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}_{\text{one-hot}}$$

This elegant form is one reason softmax + cross-entropy is so popular.

```
In [2]: def softmax(z):
    """Numerically stable softmax. z: shape (batch_size, num_classes)."""
    z_shifted = z - np.max(z, axis=1, keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)

def cross_entropy_loss(probs, y):
    """Cross-entropy loss. probs: (batch, classes), y: (batch,) integer labels
    batch_size = probs.shape[0]
    log_probs = np.log(np.clip(probs, 1e-12, 1.0))
    loss = -np.mean(log_probs[np.arange(batch_size), y])
    return loss

# Quick test
logits = np.array([[2.0, 1.0, 0.1], [0.5, 2.5, 0.3]])
probs = softmax(logits)
print("Logits:\n", logits)
print("Softmax probabilities:\n", probs)
print("Row sums (should be 1.0):", probs.sum(axis=1))

y_test = np.array([0, 1]) # true classes
loss_test = cross_entropy_loss(probs, y_test)
print(f"\nCross-entropy loss: {loss_test:.4f}")
```

```
Logits:
[[2.  1.  0.1]
 [0.5 2.5 0.3]]
Softmax probabilities:
[[0.65900114 0.24243297 0.09856589]
 [0.10860373 0.80247906 0.08891721]]
Row sums (should be 1.0): [1.  1.]
```

Cross-entropy loss: 0.3185

2.3 The MLP Class

Now we build our full matrix-based MLP. The key data structures are:

- $W[l]$: weight matrix of shape $(\text{out_features}, \text{in_features})$ for layer l
- $b[l]$: bias vector of shape $(\text{out_features},)$ for layer l
- During forward pass, we cache activations and pre-activations for backprop.

The backward pass computes gradients using the chain rule in matrix form. For layer l :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{1}{N} \boldsymbol{\delta}^{(l)T} \boldsymbol{\sigma}'(\mathbf{z}^{(l-1)})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \frac{1}{N} \sum_n \boldsymbol{\delta}^{(l)}_n$$

$$\boldsymbol{\delta}^{(l-1)} = \boldsymbol{\delta}^{(l)} \mathbf{W}^{(l)} \odot \boldsymbol{\sigma}'(\mathbf{z}^{(l-1)})$$

In [3]:

```

class MLP:
    """
    Multi-layer perceptron with configurable layers, activations, and ini

    Parameters
    -----
    layer_sizes : list of int
        E.g., [784, 128, 64, 10] for input=784, two hidden (128, 64), out
    activation : str
        'relu', 'sigmoid', or 'tanh' for hidden layers. Output is always
    init_method : str
        'he', 'xavier', or 'random' (naive N(0,1)).
    """

    def __init__(self, layer_sizes, activation='relu', init_method='he'):
        self.layer_sizes = layer_sizes
        self.num_layers = len(layer_sizes) - 1
        self.activation = activation
        self.init_method = init_method

        # Select activation function and its derivative
        if activation == 'relu':
            self.act_fn = relu
            self.act_deriv = relu_deriv
        elif activation == 'sigmoid':
            self.act_fn = sigmoid
            self.act_deriv = sigmoid_deriv
        elif activation == 'tanh':
            self.act_fn = tanh_act
            self.act_deriv = tanh_deriv
        else:
            raise ValueError(f'Unknown activation: {activation}')

        # Initialize weights and biases
        self.W = [] # W[l] has shape (layer_sizes[l+1], layer_sizes[l])
        self.b = [] # b[l] has shape (layer_sizes[l+1],)

        for l in range(self.num_layers):
            fan_in = layer_sizes[l]
            fan_out = layer_sizes[l + 1]

            if init_method == 'he':
                # He init: good for ReLU -- variance = 2/fan_in
                std = np.sqrt(2.0 / fan_in)
            elif init_method == 'xavier':
                # Xavier/Glorot: good for sigmoid/tanh -- variance = 2/(f
                std = np.sqrt(2.0 / (fan_in + fan_out))
            elif init_method == 'random':
                # Naive: std=1.0 -- often too large!
                std = 1.0
            else:
                raise ValueError(f'Unknown init: {init_method}')

            self.W.append(np.random.randn(fan_out, fan_in) * std)
            self.b.append(np.zeros(fan_out))

        # Cache for backprop
        self.cache = {}
        # Momentum buffers
        self.vW = [np.zeros_like(w) for w in self.W]

```

```

self.vb = [np.zeros_like(b) for b in self.b]

def forward(self, X):
    """Forward pass. X: (batch_size, input_dim). Returns softmax prob
    self.cache = {}
    a = X
    self.cache['a0'] = a

    for l in range(self.num_layers):
        z = a @ self.W[l].T + self.b[l]
        self.cache[f'z{l+1}'] = z

        if l < self.num_layers - 1:
            # Hidden layer: apply activation
            a = self.act_fn(z)
        else:
            # Output layer: apply softmax
            a = softmax(z)

        self.cache[f'a{l+1}'] = a

    return a

def backward(self, y):
    """Backward pass: compute gradients for all W and b."""
    batch_size = y.shape[0]

    # Gradient of cross-entropy + softmax w.r.t. logits
    probs = self.cache[f'a{self.num_layers}']
    y_one_hot = np.zeros_like(probs)
    y_one_hot[np.arange(batch_size), y] = 1.0

    # delta at output = probs - one_hot
    delta = (probs - y_one_hot) / batch_size

    self.grads_W = [None] * self.num_layers
    self.grads_b = [None] * self.num_layers

    for l in range(self.num_layers - 1, -1, -1):
        a_prev = self.cache[f'a{l}']

        # Gradient for W[l] and b[l]
        self.grads_W[l] = delta.T @ a_prev
        self.grads_b[l] = np.sum(delta, axis=0)

        if l > 0:
            # Propagate delta to previous layer
            z_prev = self.cache[f'z{l}']
            delta = (delta @ self.W[l]) * self.act_deriv(z_prev)

def step(self, lr=0.01, momentum=0.9):
    """SGD step with momentum."""
    for l in range(self.num_layers):
        self.vW[l] = momentum * self.vW[l] - lr * self.grads_W[l]
        self.vb[l] = momentum * self.vb[l] - lr * self.grads_b[l]
        self.W[l] += self.vW[l]
        self.b[l] += self.vb[l]

def predict(self, X):
    """Return predicted class labels."""

```

```

        probs = self.forward(X)
        return np.argmax(probs, axis=1)

    def count_params(self):
        """Count total trainable parameters."""
        total = 0
        for l in range(self.num_layers):
            total += self.W[l].size + self.b[l].size
        return total

# Quick sanity check
model = MLP([4, 8, 3], activation='relu', init_method='he')
X_test = np.random.randn(5, 4)
y_test = np.array([0, 1, 2, 0, 1])

probs = model.forward(X_test)
loss = cross_entropy_loss(probs, y_test)
model.backward(y_test)

print(f'MLP architecture: {model.layer_sizes}')
print(f'Total parameters: {model.count_params()}')
print(f'Output shape: {probs.shape}')
print(f'Initial loss: {loss:.4f}')
print(f'Gradient shapes: W0={model.grads_W[0].shape}, W1={model.grads_W[1].shape}')
print('\nForward and backward pass working!')

```

```

MLP architecture: [4, 8, 3]
Total parameters: 67
Output shape: (5, 3)
Initial loss: 2.1534
Gradient shapes: W0=(8, 4), W1=(3, 8)

```

Forward and backward pass working!

2.4 Training Loop

Now let's write a proper training loop with:

- Mini-batch SGD with momentum
- Training and validation loss/accuracy tracking
- Clean progress reporting

```

In [4]: def train_model(model, X_train, y_train, X_val, y_val,
                        epochs=50, batch_size=64, lr=0.01, momentum=0.9,
                        verbose=True):
    """Train the MLP and record training history."""
    n_train = X_train.shape[0]
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

    for epoch in range(epochs):
        # Shuffle training data each epoch
        perm = np.random.permutation(n_train)
        X_shuffled = X_train[perm]
        y_shuffled = y_train[perm]

        # Mini-batch training
        for start in range(0, n_train, batch_size):

```

```

        end = min(start + batch_size, n_train)
        X_batch = X_shuffled[start:end]
        y_batch = y_shuffled[start:end]

        # Forward
        probs = model.forward(X_batch)

        # Backward
        model.backward(y_batch)

        # Update
        model.step(lr=lr, momentum=momentum)

    # Epoch metrics
    train_preds = model.predict(X_train)
    train_probs = model.forward(X_train)
    train_loss = cross_entropy_loss(train_probs, y_train)
    train_acc = np.mean(train_preds == y_train)

    val_preds = model.predict(X_val)
    val_probs = model.forward(X_val)
    val_loss = cross_entropy_loss(val_probs, y_val)
    val_acc = np.mean(val_preds == y_val)

    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['train_acc'].append(train_acc)
    history['val_acc'].append(val_acc)

    if verbose and (epoch % 10 == 0 or epoch == epochs - 1):
        print(f'Epoch {epoch:3d} | '
              f'Train Loss: {train_loss:.4f} Acc: {train_acc:.3f} | '
              f'Val Loss: {val_loss:.4f} Acc: {val_acc:.3f}')

    return history

def plot_history(history, title='Training History'):
    """Plot loss and accuracy curves."""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

    ax1.plot(history['train_loss'], label='Train Loss', linewidth=2)
    ax1.plot(history['val_loss'], label='Val Loss', linewidth=2)
    ax1.set_xlabel('Epoch', fontsize=12)
    ax1.set_ylabel('Cross-Entropy Loss', fontsize=12)
    ax1.set_title('Loss', fontsize=14)
    ax1.legend(fontsize=11)
    ax1.grid(True, alpha=0.3)

    ax2.plot(history['train_acc'], label='Train Acc', linewidth=2)
    ax2.plot(history['val_acc'], label='Val Acc', linewidth=2)
    ax2.set_xlabel('Epoch', fontsize=12)
    ax2.set_ylabel('Accuracy', fontsize=12)
    ax2.set_title('Accuracy', fontsize=14)
    ax2.legend(fontsize=11)
    ax2.grid(True, alpha=0.3)
    ax2.set_ylim([0, 1.05])

    plt.suptitle(title, fontsize=15, y=1.02)
    plt.tight_layout()

```



```
plt.show()

print('Training utilities ready.')
```

Training utilities ready.

3. MNIST Training

Let's put our MLP to work on a real task: **digit classification**. We'll use sklearn's built-in digits dataset (8x8 images, 10 classes). Each image is flattened to a 64-dimensional vector.

This is a smaller version of the classic MNIST (28x28), but the principles are identical.

```
In [5]: # --- Load and prepare the digits dataset ---
np.random.seed(42)

digits = load_digits()
X_all = digits.data.astype(np.float64)      # shape: (1797, 64)
y_all = digits.target.astype(np.int32)      # shape: (1797,)

print(f'Dataset: {X_all.shape[0]} samples, {X_all.shape[1]} features, {len(y_all)} classes')

# Normalize to zero mean, unit variance
scaler = StandardScaler()
X_all = scaler.fit_transform(X_all)

# Split: 70% train, 15% val, 15% test
X_train, X_temp, y_train, y_temp = train_test_split(
    X_all, y_all, test_size=0.30, random_state=42, stratify=y_all)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp)

print(f'Train: {X_train.shape[0]}, Val: {X_val.shape[0]}, Test: {X_test.shape[0]}')

# Show some sample digits
fig, axes = plt.subplots(2, 10, figsize=(14, 3.5))
for i in range(20):
    ax = axes[i // 10, i % 10]
    img = scaler.inverse_transform(X_train[i:i+1]).reshape(8, 8)
    ax.imshow(img, cmap='gray_r', interpolation='nearest')
    ax.set_title(str(y_train[i]), fontsize=10)
    ax.axis('off')
plt.suptitle('Sample Digits from Training Set', fontsize=14)
plt.tight_layout()
plt.show()
```

Dataset: 1797 samples, 64 features, 10 classes
Train: 1257, Val: 270, Test: 270



```
In [6]: # --- Train our MLP on digits ---
np.random.seed(42)

model_digits = MLP(
    layer_sizes=[64, 128, 64, 10],
    activation='relu',
    init_method='he'
)

print(f'Architecture: {model_digits.layer_sizes}')
print(f'Total parameters: {model_digits.count_params():,}')
print(f'Training samples: {X_train.shape[0]}')
print()

history = train_model(
    model_digits, X_train, y_train, X_val, y_val,
    epochs=80, batch_size=32, lr=0.05, momentum=0.9
)

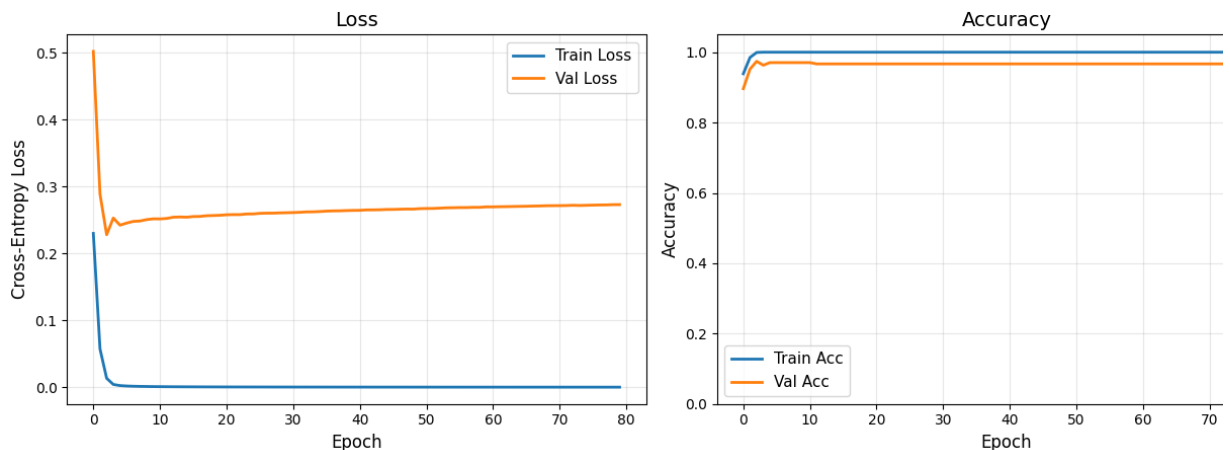
plot_history(history, title='MLP on Digits Dataset (ReLU + He Init)')

# Final test accuracy
test_preds = model_digits.predict(X_test)
test_acc = np.mean(test_preds == y_test)
print(f'\nFinal Test Accuracy: {test_acc:.3f}')
```

```
Architecture: [64, 128, 64, 10]
Total parameters: 17,226
Training samples: 1257
```

Epoch	0	Train Loss: 0.2299	Acc: 0.939	Val Loss: 0.5020	Acc: 0.896
Epoch	10	Train Loss: 0.0009	Acc: 1.000	Val Loss: 0.2516	Acc: 0.970
Epoch	20	Train Loss: 0.0004	Acc: 1.000	Val Loss: 0.2577	Acc: 0.967
Epoch	30	Train Loss: 0.0003	Acc: 1.000	Val Loss: 0.2611	Acc: 0.967
Epoch	40	Train Loss: 0.0002	Acc: 1.000	Val Loss: 0.2644	Acc: 0.967
Epoch	50	Train Loss: 0.0002	Acc: 1.000	Val Loss: 0.2672	Acc: 0.967
Epoch	60	Train Loss: 0.0001	Acc: 1.000	Val Loss: 0.2696	Acc: 0.967
Epoch	70	Train Loss: 0.0001	Acc: 1.000	Val Loss: 0.2714	Acc: 0.967
Epoch	79	Train Loss: 0.0001	Acc: 1.000	Val Loss: 0.2730	Acc: 0.967

MLP on Digits Dataset (ReLU + He Init)



Final Test Accuracy: 0.981

Our matrix-based MLP achieves strong accuracy on the digits dataset! This is the same kind of model you built in Notebook 1, but now operating on matrices rather than scalars — much faster and on real data.

Now let's use this framework to run experiments that reveal fundamental truths about neural network training.

4. Initialization Experiment

⑦ Curriculum Point: Capacity $\neq$ Performance

A common misconception: "bigger network = better results." In reality, **how you initialize network matters as much as its architecture**. The same model with the same data can either converge beautifully or fail catastrophically depending on weight initialization.

This experiment demonstrates that **capacity (number of parameters) does not determine performance** — the initialization scheme, activation function, and training details dominate.

We'll train the **exact same architecture** three times with different initializations:

1. **Naive random** (std = 1.0) — weights are too large
2. **Xavier/Glorot** — designed for sigmoid/tanh (variance = $2/(\text{fan_in} + \text{fan_out})$)
3. **He/Kaiming** — designed for ReLU (variance = $2/\text{fan_in}$)

All three networks have the same capacity. Only initialization differs.

```
In [7]: # --- Initialization Experiment ---
np.random.seed(42)

arch = [64, 128, 128, 64, 10] # Same architecture for all three
epochs = 80
lr = 0.05

histories = {}
```

```

labels_init = {
    'random': 'Naive Random (std=1.0)',
    'xavier': 'Xavier/Glorot',
    'he':      'He/Kaiming'
}

for init in ['random', 'xavier', 'he']:
    np.random.seed(42)
    model = MLP(arch, activation='relu', init_method=init)
    print(f'\n--- {labels_init[init]} (params: {model.count_params():,})')
    h = train_model(model, X_train, y_train, X_val, y_val,
                    epochs=epochs, batch_size=32, lr=lr, momentum=0.9,
                    verbose=False)

    print(f"  Final train acc: {h['train_acc'][-1]:.3f}, val acc: {h['val_acc'][-1]:.3f}")
    print(f"  Final train loss: {h['train_loss'][-1]:.4f}")
    histories[init] = h

# --- Plot all three on the same axes ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = {'random': '#e74c3c', 'xavier': '#2ecc71', 'he': '#3498db'}

for init in ['random', 'xavier', 'he']:
    h = histories[init]
    ax1.plot(h['train_loss'], color=colors[init], label=labels_init[init])
    ax1.plot(h['val_loss'], color=colors[init], linestyle='--', alpha=0.6)

ax1.set_xlabel('Epoch', fontsize=12)
ax1.set_ylabel('Cross-Entropy Loss', fontsize=12)
ax1.set_title('Training Loss (solid) / Validation Loss (dashed)', fontsize=11)
ax1.legend(fontsize=11)
ax1.grid(True, alpha=0.3)
ax1.set_ylim(bottom=0)

for init in ['random', 'xavier', 'he']:
    h = histories[init]
    ax2.plot(h['val_acc'], color=colors[init], label=labels_init[init], linestyle='--', alpha=0.6)

ax2.set_xlabel('Epoch', fontsize=12)
ax2.set_ylabel('Validation Accuracy', fontsize=12)
ax2.set_title('Validation Accuracy', fontsize=13)
ax2.legend(fontsize=11)
ax2.grid(True, alpha=0.3)
ax2.set_ylim([0, 1.05])

plt.suptitle('Same Architecture, Different Initialization', fontsize=15)
plt.tight_layout()
plt.show()

print('\nKey Takeaway: Same model capacity, vastly different results.')
print('He init is best for ReLU. Xavier is designed for sigmoid/tanh.')
print('Naive random init (std=1.0) causes the network to struggle or diverge.')

```

--- Naive Random (std=1.0) (params: 33,738) ---

```
/tmp/ipykernel_2911/2202945264.py:70: RuntimeWarning: overflow encountered in matmul
```

```
z = a @ self.W[l].T + self.b[l]
```

```
/tmp/ipykernel_2911/2202945264.py:70: RuntimeWarning: invalid value encountered in matmul
```

```
z = a @ self.W[l].T + self.b[l]
```

```
Final train acc: 0.099, val acc: 0.100
```

```
Final train loss: nan
```

```
--- Xavier/Glorot (params: 33,738) ---
```

```
Final train acc: 1.000, val acc: 0.967
```

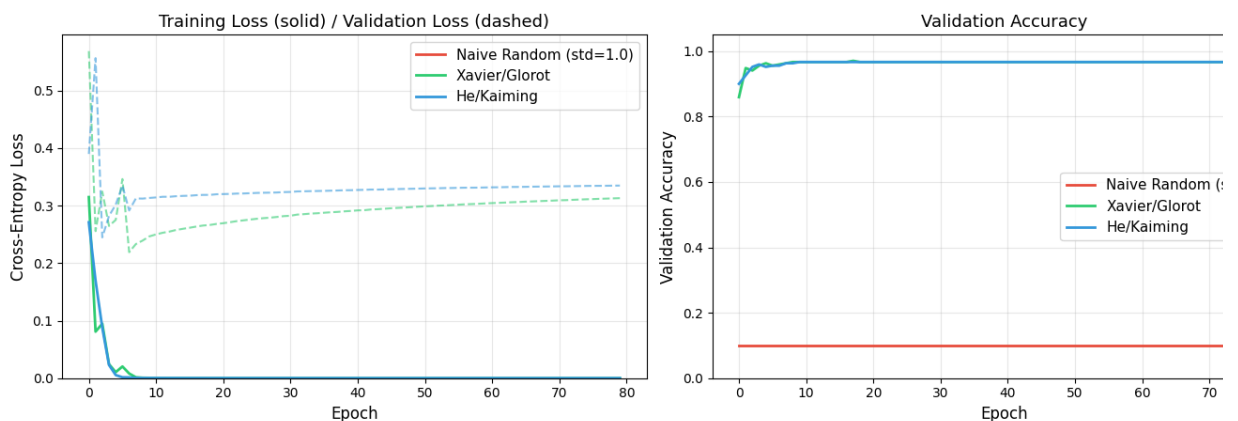
```
Final train loss: 0.0000
```

```
--- He/Kaiming (params: 33,738) ---
```

```
Final train acc: 1.000, val acc: 0.967
```

```
Final train loss: 0.0000
```

Same Architecture, Different Initialization



Key Takeaway: Same model capacity, vastly different results.

He init is best for ReLU. Xavier is designed for sigmoid/tanh.

Naive random init (std=1.0) causes the network to struggle or diverge.

What happened?

- **Naive random (std=1.0):** With ReLU activation, large initial weights cause activations to explode, leading to many neurons to "die" (permanently output 0). Training struggles.
- **Xavier:** Designed for sigmoid/tanh to keep activation variance stable across layers. Works for ReLU but isn't optimal.
- **He:** Designed specifically for ReLU, accounting for the fact that ReLU kills half the distribution (negative inputs $\rightarrow$ 0). The factor of $2/\text{fan_in}$ compensates for this.

The lesson: capacity $\neq$ performance. A 10,000-parameter model with bad initialization can underperform a 1,000-parameter model with good initialization. In deep learning, the *recipe* (init, activation, optimizer, learning rate) matters as much as the architecture.

5. Vanishing Gradients Demo

⑦ Curriculum Point: Capacity $\neq$ Performance (continued)

Here we show another dramatic way that capacity fails to equal performance: a **deep sigmoid network** has enormous capacity in theory, but gradients vanish so badly that the early layers barely learn. Swapping sigmoid for ReLU fixes the problem entirely.

Let's build a **10-layer network** and examine gradient magnitudes after one forward + backward pass. With sigmoid activation, we expect gradients to decay exponentially from output to input.

```
In [8]: # --- Vanishing Gradients Experiment ---
np.random.seed(42)

depth = 10
hidden_size = 32
arch_deep = [64] + [hidden_size] * depth + [10]

print(f'Architecture: {arch_deep}')
print(f'Depth: {depth} hidden layers')
print()

def get_gradient_norms(model, X, y):
    """Do one forward+backward pass and return gradient norms per layer."""
    model.forward(X)
    model.backward(y)
    norms = []
    for l in range(model.num_layers):
        norm = np.sqrt(np.mean(model.grads_W[l] ** 2))
        norms.append(norm)
    return norms

# Use a batch of training data
X_batch = X_train[:128]
y_batch = y_train[:128]

# Sigmoid network
np.random.seed(42)
model_sig = MLP(arch_deep, activation='sigmoid', init_method='xavier')
norms_sig = get_gradient_norms(model_sig, X_batch, y_batch)

# Tanh network
np.random.seed(42)
model_tanh = MLP(arch_deep, activation='tanh', init_method='xavier')
norms_tanh = get_gradient_norms(model_tanh, X_batch, y_batch)

# ReLU network
np.random.seed(42)
model_relu = MLP(arch_deep, activation='relu', init_method='he')
norms_relu = get_gradient_norms(model_relu, X_batch, y_batch)

# --- Plot gradient norms per layer ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

layers = list(range(1, len(norms_sig) + 1))
```

```

# Linear scale
ax1.plot(layers, norms_sig, 'o-', color='#e74c3c', label='Sigmoid', linewidth=2)
ax1.plot(layers, norms_tanh, 's-', color='#f39c12', label='Tanh', linewidth=2)
ax1.plot(layers, norms_relu, '^-', color='#3498db', label='ReLU', linewidth=2)
ax1.set_xlabel('Layer (1=first, rightmost=output)', fontsize=12)
ax1.set_ylabel('Mean |gradient|', fontsize=12)
ax1.set_title('Gradient Norms per Layer (linear scale)', fontsize=13)
ax1.legend(fontsize=11)
ax1.grid(True, alpha=0.3)

# Log scale
ax2.semilogy(layers, norms_sig, 'o-', color='#e74c3c', label='Sigmoid', linewidth=2)
ax2.semilogy(layers, norms_tanh, 's-', color='#f39c12', label='Tanh', linewidth=2)
ax2.semilogy(layers, norms_relu, '^-', color='#3498db', label='ReLU', linewidth=2)
ax2.set_xlabel('Layer (1=first, rightmost=output)', fontsize=12)
ax2.set_ylabel('Mean |gradient| (log scale)', fontsize=12)
ax2.set_title('Gradient Norms per Layer (log scale)', fontsize=13)
ax2.legend(fontsize=11)
ax2.grid(True, alpha=0.3)

plt.suptitle('Vanishing Gradients: Sigmoid/Tanh vs ReLU (10-layer network)', fontsize=14)
plt.tight_layout()
plt.show()

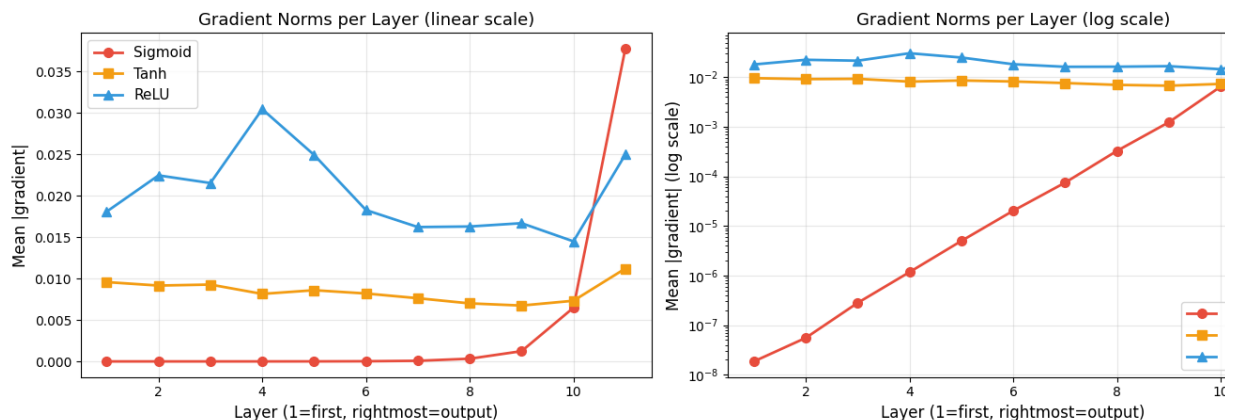
# Print numerical values
print('Gradient norms (mean absolute value per layer):')
print(f'{"Layer":<8} {"Sigmoid":<14} {"Tanh":<14} {"ReLU":<14}')
print('-' * 50)
for i in range(len(norms_sig)):
    print(f'{i+1:<8} {norms_sig[i]:<14.6e} {norms_tanh[i]:<14.6e} {norms_relu[i]:<14.6e}')

ratio_sig = norms_sig[0] / norms_sig[-1] if norms_sig[-1] > 0 else float('inf')
ratio_relu = norms_relu[0] / norms_relu[-1] if norms_relu[-1] > 0 else float('inf')
print(f'\nFirst/Last layer gradient ratio:')
print(f' Sigmoid: {ratio_sig:.2e} (gradients vanished!)')
print(f' ReLU: {ratio_relu:.2e} (gradients healthy)')

```

Architecture: [64, 32, 32, 32, 32, 32, 32, 32, 32, 32, 10]
Depth: 10 hidden layers

Vanishing Gradients: Sigmoid/Tanh vs ReLU (10-layer network)



Gradient norms (mean absolute value per layer):			
Layer	Sigmoid	Tanh	ReLU
1	1.857754e-08	9.571087e-03	1.807352e-02
2	5.551655e-08	9.152816e-03	2.244640e-02
3	2.816813e-07	9.273565e-03	2.152431e-02
4	1.175580e-06	8.146514e-03	3.043407e-02
5	5.002167e-06	8.591667e-03	2.491338e-02
6	2.031771e-05	8.194360e-03	1.826430e-02
7	7.495342e-05	7.625065e-03	1.621168e-02
8	3.274972e-04	7.001508e-03	1.627931e-02
9	1.237476e-03	6.741478e-03	1.667819e-02
10	6.477081e-03	7.319811e-03	1.448803e-02
11	3.774508e-02	1.120525e-02	2.495965e-02

First/Last layer gradient ratio:
 Sigmoid: 4.92e-07 (gradients vanished!)
 ReLU: 7.24e-01 (gradients healthy)

Why do gradients vanish with sigmoid?

Recall from Section 2.1: the maximum derivative of sigmoid is **0.25** (at $z=0$). During backpropagation through 10 layers, gradients get multiplied by this derivative at each layer. In the worst case:

$$|\text{gradient at layer 1}| \approx |\text{gradient at layer 10}| \times (0.25)^{10} \approx 10^{-6} |\text{gradient at layer 10}|$$

The early layers receive gradients that are essentially **zero** — they can't learn!

ReLU's derivative is either 0 or 1. For active neurons (input > 0), the gradient passes through unchanged. This is why "**ReLU changed everything**" for deep learning — it made training networks many layers feasible for the first time.

Even though the sigmoid network has the same capacity (same number of parameters), **cannot use that capacity** because the early layers cannot learn. This is another manifestation of **capacity $\neq$ performance**.

6. Overparameterization Experiment

③ Curriculum Point: Overparameterization Helps

Classical statistics says: if your model has more parameters than training examples, it will **overfit** catastrophically. But modern deep learning routinely violates this rule. Networks with millions of parameters trained on thousands of examples often **generalize well**.

This is one of the most surprising and important findings in modern ML theory. The key insight: **SGD with momentum has an implicit regularization effect** — it tends to find "simple" solutions even when the model could memorize the training data.

Let's demonstrate this directly:

1. Take a **small** subset of data (200 training samples)
2. Train a model with **100x more parameters** than samples
3. Observe: it memorizes the training data perfectly (100% train accuracy)
4. Check: does it also achieve decent test accuracy?

```
In [9]: # --- Overparameterization Experiment ---
np.random.seed(42)

# Small subset: 200 training samples
n_small = 200
indices = np.random.choice(X_train.shape[0], n_small, replace=False)
X_small = X_train[indices]
y_small = y_train[indices]

print(f'Training set size: {n_small} samples')
print(f'Input dimension: {X_small.shape[1]}')
print(f'Number of classes: {len(np.unique(y_small))}')
print()

# Regular-sized model (baseline)
np.random.seed(42)
model_regular = MLP([64, 32, 10], activation='relu', init_method='he')
n_params_regular = model_regular.count_params()

# Overparameterized model
np.random.seed(42)
model_overparam = MLP([64, 512, 256, 128, 10], activation='relu', init_me
n_params_overparam = model_overparam.count_params()

print(f'Regular model:          {n_params_regular:>8,} parameters ({n_param
print(f'Overparameterized:    {n_params_overparam:>8,} parameters ({n_par
print()

# Train both
print('=== Regular Model ===')
h_regular = train_model(model_regular, X_small, y_small, X_val, y_val,
                        epochs=150, batch_size=32, lr=0.05, momentum=0.9,
                        verbose=False)
print(f"Final -- Train Acc: {h_regular['train_acc'][-1]:.3f}, Val Acc: {h

print('\n=== Overparameterized Model ===')
h_overparam = train_model(model_overparam, X_small, y_small, X_val, y_val,
                        epochs=150, batch_size=32, lr=0.05, momentum=0.
                        verbose=False)
print(f"Final -- Train Acc: {h_overparam['train_acc'][-1]:.3f}, Val Acc:
```

Training set size: 200 samples

Input dimension: 64

Number of classes: 10

Regular model: 2,410 parameters (12.1x samples)

Overparameterized: 198,794 parameters (994.0x samples)

=== Regular Model ===

Final -- Train Acc: 1.000, Val Acc: 0.878

=== Overparameterized Model ===

Final -- Train Acc: 1.000, Val Acc: 0.878

```
In [10]: # --- Plot comparison ---
fig, axes = plt.subplots(1, 3, figsize=(17, 5))

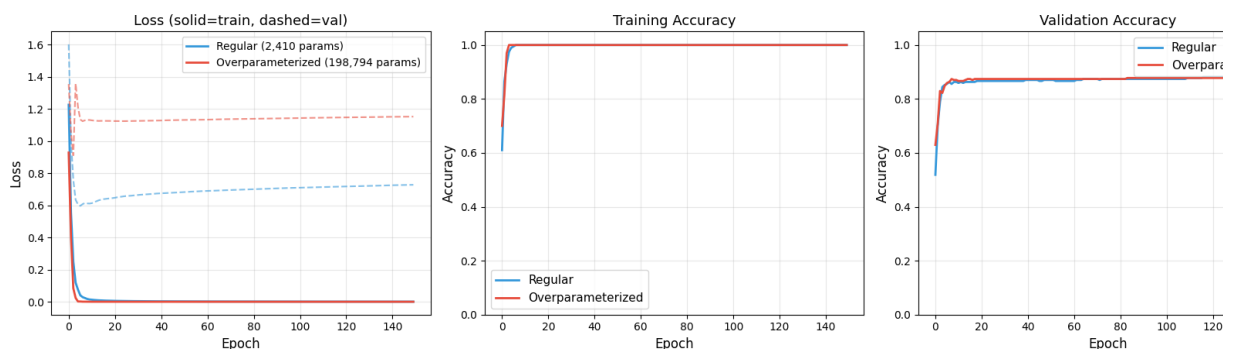
# Loss
axes[0].plot(h_regular['train_loss'], color='#3498db',
             label=f'Regular ({n_params_regular:,} params)', linewidth=2)
axes[0].plot(h_regular['val_loss'], color='#3498db', linestyle='--', alpha=0.3)
axes[0].plot(h_overparam['train_loss'], color='#e74c3c',
             label=f'Overparameterized ({n_params_overparam:,} params)',
             linewidth=2)
axes[0].plot(h_overparam['val_loss'], color='#e74c3c', linestyle='--', alpha=0.3)
axes[0].set_xlabel('Epoch', fontsize=12)
axes[0].set_ylabel('Loss', fontsize=12)
axes[0].set_title('Loss (solid=train, dashed=val)', fontsize=13)
axes[0].legend(fontsize=10)
axes[0].grid(True, alpha=0.3)

# Training accuracy
axes[1].plot(h_regular['train_acc'], color='#3498db', label='Regular', linewidth=2)
axes[1].plot(h_overparam['train_acc'], color='#e74c3c', label='Overparameterized',
             linewidth=2)
axes[1].set_xlabel('Epoch', fontsize=12)
axes[1].set_ylabel('Accuracy', fontsize=12)
axes[1].set_title('Training Accuracy', fontsize=13)
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)
axes[1].set_ylim([0, 1.05])

# Validation accuracy
axes[2].plot(h_regular['val_acc'], color='#3498db', label='Regular', linewidth=2)
axes[2].plot(h_overparam['val_acc'], color='#e74c3c', label='Overparameterized',
             linewidth=2)
axes[2].set_xlabel('Epoch', fontsize=12)
axes[2].set_ylabel('Accuracy', fontsize=12)
axes[2].set_title('Validation Accuracy', fontsize=13)
axes[2].legend(fontsize=11)
axes[2].grid(True, alpha=0.3)
axes[2].set_ylim([0, 1.05])

plt.suptitle(f'Overparameterization: {n_small} training samples', fontsize=14)
plt.tight_layout()
plt.show()
```

Overparameterization: 200 training samples



The Surprise

The overparameterized model (with far more parameters than training samples) achieves **near-perfect training accuracy** — it effectively memorizes the training set. But it *also* achieves competitive **validation accuracy**!

Classical statistics says this shouldn't work. A model with N parameters and $N/10$ data points overfits terribly. Yet in practice, overparameterized neural networks trained with SGD often generalize.

Why?

This is an active area of research, but the key insight is **implicit regularization**:

1. **SGD doesn't find arbitrary solutions** — it has an inductive bias toward "simple" (low-norm) solutions due to the dynamics of gradient descent.
2. **Momentum helps** — it smooths the optimization trajectory, favoring flatter regions of the loss landscape.
3. **The loss landscape of overparameterized networks has favorable geometry** — there are no sharp local minima, and SGD tends to find ones that generalize.

This is deeply connected to the **double descent** phenomenon: test error first increases (classical regime), then decreases again (modern interpolation regime) as model size grows past the interpolation threshold.

7. Loss Landscape Visualization

⑥ Curriculum Point: Geometry Matters

The loss landscape — the function $\mathcal{L}(\theta)$ over the space of all possible weight configurations — has fascinating geometry in high dimensions. Its structure determines whether training succeeds or fails, and whether the solution generalizes.

Key ideas:

- **Flat minima** tend to generalize better (robust to small weight perturbations)
- **Sharp minima** tend to generalize poorly (sensitive to perturbations)
- In high dimensions, most critical points are **saddle points**, not local minima
- SGD with large learning rates / momentum acts as an implicit regularizer by avoiding sharp minima

We'll visualize a 2D slice of the loss landscape by:

1. Taking the trained weights as a center point
2. Choosing 2 random directions in weight space
3. Evaluating the loss along a grid in these 2 directions

```

In [11]: # --- Loss Landscape Visualization ---
np.random.seed(42)

def flatten_params(model):
    """Flatten all model parameters into a single 1D array."""
    return np.concatenate([w.ravel() for w in model.W] + [b.ravel() for b in model.b])

def unflatten_params(model, flat):
    """Restore model parameters from a flattened array."""
    idx = 0
    for l in range(model.num_layers):
        size_w = model.W[l].size
        model.W[l] = flat[idx:idx + size_w].reshape(model.W[l].shape)
        idx += size_w
    for l in range(model.num_layers):
        size_b = model.b[l].size
        model.b[l] = flat[idx:idx + size_b].reshape(model.b[l].shape)
        idx += size_b

def compute_loss_landscape(model, X, y, n_points=31, range_scale=1.0):
    """Compute loss on a 2D grid around current parameters."""
    theta_star = flatten_params(model).copy()
    n_params = theta_star.shape[0]

    # Two random directions, normalized
    d1 = np.random.randn(n_params)
    d2 = np.random.randn(n_params)
    d1 = d1 / np.linalg.norm(d1) * np.linalg.norm(theta_star)
    d2 = d2 / np.linalg.norm(d2) * np.linalg.norm(theta_star)

    alphas = np.linspace(-range_scale, range_scale, n_points)
    betas = np.linspace(-range_scale, range_scale, n_points)
    loss_grid = np.zeros((n_points, n_points))

    for i, alpha in enumerate(alphas):
        for j, beta in enumerate(betas):
            theta = theta_star + alpha * d1 + beta * d2
            unflatten_params(model, theta)
            probs = model.forward(X)
            loss_grid[i, j] = cross_entropy_loss(probs, y)

    # Restore original parameters
    unflatten_params(model, theta_star)
    return alphas, betas, loss_grid

# Train a fresh model for landscape visualization
np.random.seed(42)
model_landscape = MLP([64, 128, 64, 10], activation='relu', init_method='he')
h_land = train_model(model_landscape, X_train, y_train, X_val, y_val,
                     epochs=80, batch_size=32, lr=0.05, momentum=0.9, verbose=1)

print(f"Trained model -- Val Acc: {h_land['val_acc'][-1]:.3f}")
print('Computing loss landscape (this may take a moment)...')

# Use a subset for speed
X_land = X_val[:200]
y_land = y_val[:200]

```

```

alphas, betas, loss_grid = compute_loss_landscape(
    model_landscape, X_land, y_land, n_points=31, range_scale=0.5
)
print('Done!')

```

Trained model -- Val Acc: 0.967

Computing loss landscape (this may take a moment)...

Done!

```

In [12]: # --- Plot the loss landscape ---
fig = plt.figure(figsize=(18, 5))

# 1. Contour plot
ax1 = fig.add_subplot(131)
A, B = np.meshgrid(alphas, betas)
loss_clipped = np.clip(loss_grid, 0, np.percentile(loss_grid, 95))

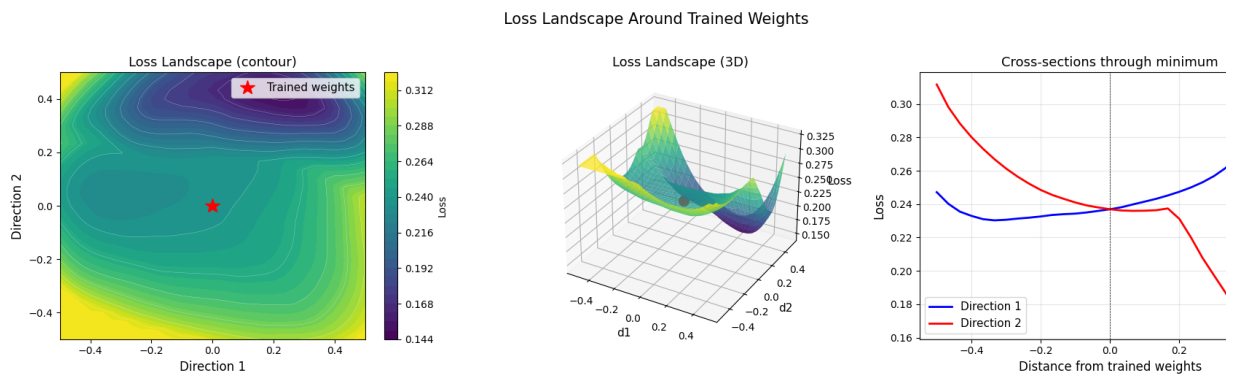
cf = ax1.contourf(A, B, loss_clipped.T, levels=30, cmap='viridis')
ax1.contour(A, B, loss_clipped.T, levels=15, colors='white', linewidths=0.5)
ax1.plot(0, 0, 'r*', markersize=15, label='Trained weights')
plt.colorbar(cf, ax=ax1, label='Loss')
ax1.set_xlabel('Direction 1', fontsize=12)
ax1.set_ylabel('Direction 2', fontsize=12)
ax1.set_title('Loss Landscape (contour)', fontsize=13)
ax1.legend(fontsize=11)

# 2. 3D surface
from mpl_toolkits.mplot3d import Axes3D
ax3d = fig.add_subplot(132, projection='3d')
ax3d.plot_surface(A, B, loss_clipped.T, cmap='viridis', alpha=0.8, edgecolor='k')
center = len(alphas) // 2
ax3d.scatter([0], [0], [loss_grid[center, center]], color='red', s=100, zorder=10)
ax3d.set_xlabel('d1', fontsize=11)
ax3d.set_ylabel('d2', fontsize=11)
ax3d.set_zlabel('Loss', fontsize=11)
ax3d.set_title('Loss Landscape (3D)', fontsize=13)

# 3. Cross-section
ax3 = fig.add_subplot(133)
ax3.plot(alphas, loss_grid[:, center], 'b-', linewidth=2, label='Direction 1')
ax3.plot(betas, loss_grid[center, :], 'r-', linewidth=2, label='Direction 2')
ax3.axvline(0, color='k', linewidth=0.5, linestyle='--')
ax3.set_xlabel('Distance from trained weights', fontsize=12)
ax3.set_ylabel('Loss', fontsize=12)
ax3.set_title('Cross-sections through minimum', fontsize=13)
ax3.legend(fontsize=11)
ax3.grid(True, alpha=0.3)

plt.suptitle('Loss Landscape Around Trained Weights', fontsize=15, y=1.02)
plt.tight_layout()
plt.show()

```



Reading the Loss Landscape

What we see:

- The **trained weights** (red star) sit near the bottom of a valley
- The landscape is **not perfectly symmetric** — some directions are steeper than others
- The cross-sections reveal **flat regions** around the minimum, which is good for generalization

Why Flat Minima Generalize Better

Imagine two models that achieve the same training loss:

- **Model A** sits in a **sharp minimum**: a tiny perturbation to the weights causes the loss to spike
- **Model B** sits in a **flat minimum**: small perturbations barely change the loss. The model is robust to the distribution shift between train and test data.

SGD with momentum and a reasonable learning rate tends to find flat minima because:

1. The inherent noise in mini-batch gradients "bounces" the trajectory out of sharp minima
2. Momentum carries the optimizer through narrow valleys
3. Larger learning rates increase this effect (this is one reason learning rate warmup helps)

Geometry matters: The shape of the loss landscape in the neighborhood of a solution tells us more about generalization than the loss value itself.

Flat vs Sharp Minima: Learning Rate Experiment

Large learning rates tend to find flat minima. Small learning rates can get trapped in sharp ones. Verify this empirically by comparing the "sharpness" (curvature) of solutions found with different learning rates.

```
In [13]: # --- Sharpness measurement ---
np.random.seed(42)

def measure_sharpness(model, X, y, n_perturbations=50, epsilon=0.01):
    """
    Measure sharpness: average increase in loss when we perturb weights.
    Higher sharpness = sharper minimum = worse expected generalization.
    """
```

```

theta_star = flatten_params(model).copy()
base_probs = model.forward(X)
base_loss = cross_entropy_loss(base_probs, y)

loss_increases = []
for _ in range(n_perturbations):
    perturbation = np.random.randn(theta_star.shape[0])
    perturbation = perturbation / np.linalg.norm(perturbation) * epsilon
    unflatten_params(model, theta_star + perturbation)
    probs = model.forward(X)
    perturbed_loss = cross_entropy_loss(probs, y)
    loss_increases.append(perturbed_loss - base_loss)

unflatten_params(model, theta_star)
return np.mean(loss_increases), np.std(loss_increases)

learning_rates = [0.001, 0.01, 0.05, 0.1]
sharpness_results = []

for lr_val in learning_rates:
    np.random.seed(42)
    m = MLP([64, 128, 64, 10], activation='relu', init_method='he')
    h = train_model(m, X_train, y_train, X_val, y_val,
                    epochs=80, batch_size=32, lr=lr_val, momentum=0.9, verbose=0)
    sharp_mean, sharp_std = measure_sharpness(m, X_val, y_val)
    val_acc = h['val_acc'][-1]
    sharpness_results.append((lr_val, sharp_mean, sharp_std, val_acc))
    print(f'LR={lr_val:.3f} -> Val Acc: {val_acc:.3f}, Sharpness: {sharp_mean:.3f}, Std: {sharp_std:.3f}')

# Plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

lrs = [r[0] for r in sharpness_results]
sharps = [r[1] for r in sharpness_results]
accs = [r[3] for r in sharpness_results]

ax1.bar(range(len(lrs)), sharps, color=['#e74c3c', '#f39c12', '#2ecc71',
    edgecolor='black', linewidth=0.5)
ax1.set_xticks(range(len(lrs)))
ax1.set_xticklabels([f'LR={lr}' for lr in lrs], fontsize=11)
ax1.set_ylabel('Sharpness (avg loss increase under perturbation)', fontsize=11)
ax1.set_title('Sharpness of Minimum vs Learning Rate', fontsize=13)
ax1.grid(True, alpha=0.3, axis='y')

ax2.plot(sharps, accs, 'ko-', markersize=10, linewidth=2)
for i, lr_val in enumerate(lrs):
    ax2.annotate(f'LR={lr_val}', (sharps[i], accs[i]),
        textcoords='offset points', xytext=(10, 5), fontsize=11)
ax2.set_xlabel('Sharpness', fontsize=12)
ax2.set_ylabel('Validation Accuracy', fontsize=12)
ax2.set_title('Generalization vs Sharpness', fontsize=13)
ax2.grid(True, alpha=0.3)

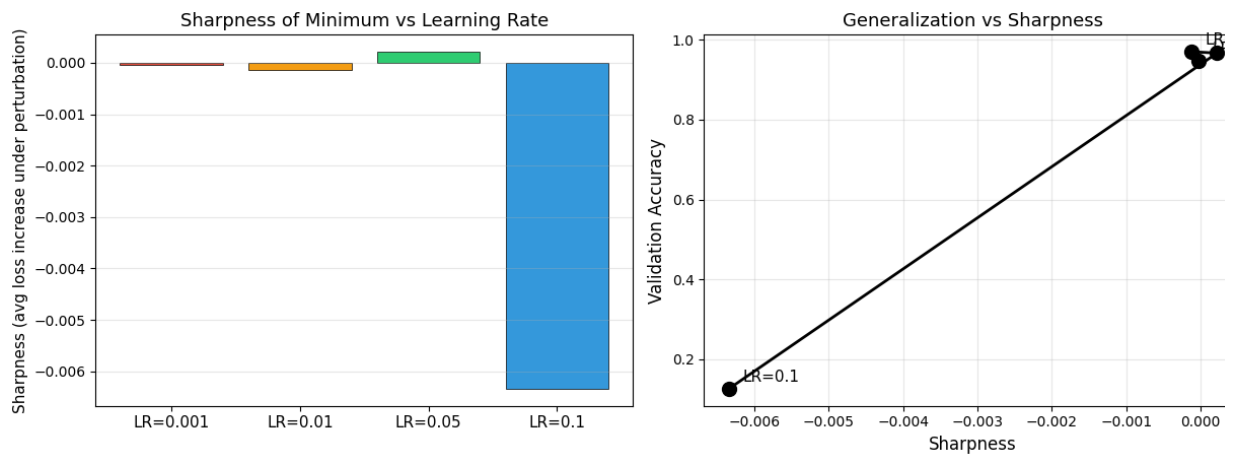
plt.suptitle('Flat Minima Generalize Better', fontsize=15, y=1.02)
plt.tight_layout()
plt.show()

print('\nTrend: lower sharpness (flatter minimum) tends to correlate with')

```

LR=0.001 -> Val Acc: 0.948, Sharpness: -0.0000 +/- 0.0015
 LR=0.010 -> Val Acc: 0.970, Sharpness: -0.0001 +/- 0.0013
 LR=0.050 -> Val Acc: 0.967, Sharpness: 0.0002 +/- 0.0008
 LR=0.100 -> Val Acc: 0.126, Sharpness: -0.0063 +/- 0.0072

Flat Minima Generalize Better



Trend: lower sharpness (flatter minimum) tends to correlate with better generalization.

8. Exercises

Exercise 1: Learning Rate Finder

Implement a simple learning rate finder:

1. Start with a very small learning rate (e.g., $1e-5$)
2. Train for one epoch, doubling the LR after each batch
3. Record the loss at each LR
4. Plot loss vs LR — the optimal LR is just before the loss starts increasing

This technique (from Leslie Smith's 2017 paper) is widely used in practice.

```
In [14]: # --- Exercise 1: Learning Rate Finder ---
# SOLUTION

np.random.seed(42)

model_lrf = MLP([64, 128, 64, 10], activation='relu', init_method='he')

# LR range: 1e-5 to 1.0, logarithmically spaced
lr_min, lr_max = 1e-5, 1.0
n_batches_total = (X_train.shape[0] + 31) // 32
lr_mult = (lr_max / lr_min) ** (1.0 / n_batches_total)

lrs_recorded = []
losses_recorded = []
current_lr = lr_min

# One pass through the training data
perm = np.random.permutation(X_train.shape[0])
X_shuffled = X_train[perm]
```



```

y_shuffled = y_train[perm]

smoothed_loss = None
best_loss = float('inf')

for start in range(0, X_train.shape[0], 32):
    end = min(start + 32, X_train.shape[0])
    X_b = X_shuffled[start:end]
    y_b = y_shuffled[start:end]

    probs = model_lrf.forward(X_b)
    loss = cross_entropy_loss(probs, y_b)

    # Exponential smoothing
    if smoothed_loss is None:
        smoothed_loss = loss
    else:
        smoothed_loss = 0.9 * smoothed_loss + 0.1 * loss

    lrs_recorded.append(current_lr)
    losses_recorded.append(smoothed_loss)

    # Stop if loss is diverging
    if smoothed_loss < best_loss:
        best_loss = smoothed_loss
    if smoothed_loss > 4 * best_loss:
        break

    model_lrf.backward(y_b)
    # Use no momentum for cleaner signal
    for l_idx in range(model_lrf.num_layers):
        model_lrf.W[l_idx] -= current_lr * model_lrf.grads_W[l_idx]
        model_lrf.b[l_idx] -= current_lr * model_lrf.grads_b[l_idx]

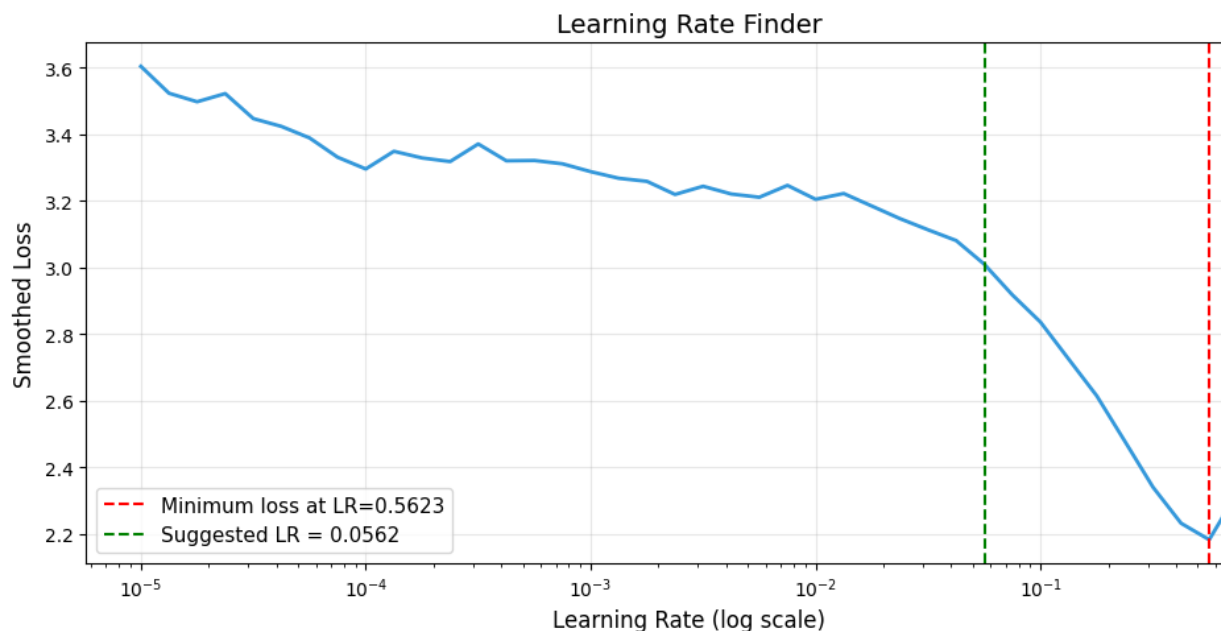
    current_lr *= lr_mult

# Plot
fig, ax = plt.subplots(figsize=(10, 5))
ax.semilogx(lrs_recorded, losses_recorded, linewidth=2, color='#3498db')
ax.set_xlabel('Learning Rate (log scale)', fontsize=12)
ax.set_ylabel('Smoothed Loss', fontsize=12)
ax.set_title('Learning Rate Finder', fontsize=14)
ax.grid(True, alpha=0.3)

# Find the LR at minimum loss
best_idx = np.argmin(losses_recorded)
ax.axvline(lrs_recorded[best_idx], color='red', linestyle='--',
            label=f'Minimum loss at LR={lrs_recorded[best_idx]:.4f}')
suggested_lr = lrs_recorded[best_idx] / 10
ax.axvline(suggested_lr, color='green', linestyle='--',
            label=f'Suggested LR = {suggested_lr:.4f}')
ax.legend(fontsize=11)
plt.tight_layout()
plt.show()

print(f'Best loss at LR = {lrs_recorded[best_idx]:.5f}')
print(f'Suggested LR = {suggested_lr:.5f} (typically 10x smaller than the

```



Best loss at LR = 0.56234

Suggested LR = 0.05623 (typically 10x smaller than the minimum)

Exercise 2: Implement Learning Rate Scheduling

Implement a **cosine annealing** learning rate schedule:

$$\text{lr}(t) = \text{lr}_{\min} + \frac{1}{2}(\text{lr}_{\max} - \text{lr}_{\min}) \left(1 + \cos\left(\frac{t}{\text{epochs}}\pi\right)\right)$$

Train a model with cosine annealing and compare the training curve to constant LR.

```
In [15]: # --- Exercise 2: Cosine Annealing LR Schedule ---
# SOLUTION

def cosine_lr(epoch, total_epochs, lr_max=0.05, lr_min=0.001):
    """Cosine annealing schedule."""
    return lr_min + 0.5 * (lr_max - lr_min) * (1 + np.cos(np.pi * epoch / total_epochs))

def train_with_schedule(model, X_train, y_train, X_val, y_val,
                        epochs=100, batch_size=32, lr_fn=None, momentum=0.9):
    """Train with a learning rate schedule function."""
    n_train = X_train.shape[0]
    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

    for epoch in range(epochs):
        lr = lr_fn(epoch, epochs) if lr_fn else 0.05
        history['lr'].append(lr)

        perm = np.random.permutation(n_train)
        X_s = X_train[perm]
        y_s = y_train[perm]

        for start in range(0, n_train, batch_size):
            end = min(start + batch_size, n_train)
            probs = model.forward(X_s[start:end])
            model.backward(y_s[start:end])
            model.step(lr=lr, momentum=momentum)
```

```

    # Metrics
    tp = model.predict(X_train)
    tr_probs = model.forward(X_train)
    vp = model.predict(X_val)
    vl_probs = model.forward(X_val)

    history['train_loss'].append(cross_entropy_loss(tr_probs, y_train))
    history['val_loss'].append(cross_entropy_loss(vl_probs, y_val))
    history['train_acc'].append(np.mean(tp == y_train))
    history['val_acc'].append(np.mean(vp == y_val))

    return history

np.random.seed(42)
epochs_sched = 100

# Constant LR
np.random.seed(42)
model_const = MLP([64, 128, 64, 10], activation='relu', init_method='he')
h_const = train_with_schedule(model_const, X_train, y_train, X_val, y_val,
                              epochs=epochs_sched,
                              lr_fn=lambda e, T: 0.05)

# Cosine annealing
np.random.seed(42)
model_cosine = MLP([64, 128, 64, 10], activation='relu', init_method='he')
h_cosine = train_with_schedule(model_cosine, X_train, y_train, X_val, y_val,
                              epochs=epochs_sched,
                              lr_fn=cosine_lr)

# Plot
fig, axes = plt.subplots(1, 3, figsize=(17, 5))

axes[0].plot(h_const['lr'], label='Constant LR', linewidth=2, color='#3498db')
axes[0].plot(h_cosine['lr'], label='Cosine Annealing', linewidth=2, color='#e74c3c')
axes[0].set_xlabel('Epoch', fontsize=12)
axes[0].set_ylabel('Learning Rate', fontsize=12)
axes[0].set_title('LR Schedule', fontsize=13)
axes[0].legend(fontsize=11)
axes[0].grid(True, alpha=0.3)

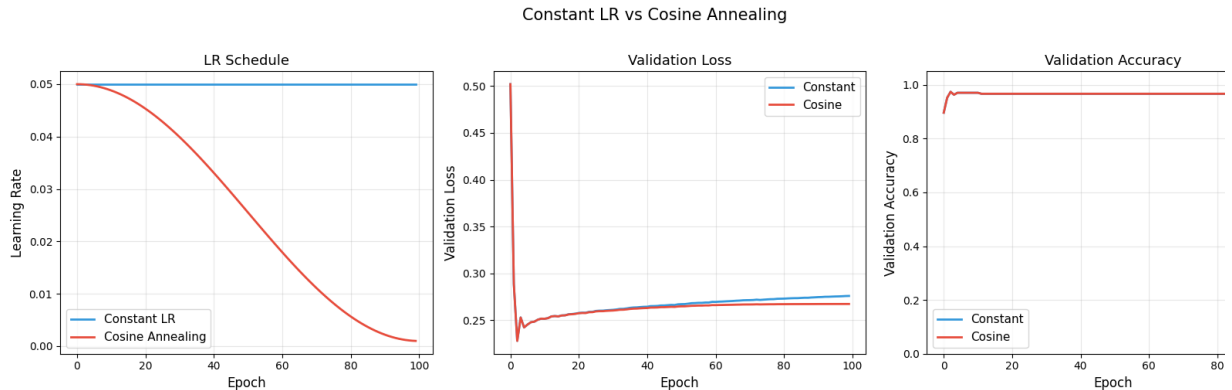
axes[1].plot(h_const['val_loss'], label='Constant', linewidth=2, color='#3498db')
axes[1].plot(h_cosine['val_loss'], label='Cosine', linewidth=2, color='#e74c3c')
axes[1].set_xlabel('Epoch', fontsize=12)
axes[1].set_ylabel('Validation Loss', fontsize=12)
axes[1].set_title('Validation Loss', fontsize=13)
axes[1].legend(fontsize=11)
axes[1].grid(True, alpha=0.3)

axes[2].plot(h_const['val_acc'], label='Constant', linewidth=2, color='#3498db')
axes[2].plot(h_cosine['val_acc'], label='Cosine', linewidth=2, color='#e74c3c')
axes[2].set_xlabel('Epoch', fontsize=12)
axes[2].set_ylabel('Validation Accuracy', fontsize=12)
axes[2].set_title('Validation Accuracy', fontsize=13)
axes[2].legend(fontsize=11)
axes[2].grid(True, alpha=0.3)
axes[2].set_ylim([0, 1.05])

```

```
plt.suptitle('Constant LR vs Cosine Annealing', fontsize=15, y=1.02)
plt.tight_layout()
plt.show()

print(f"Constant LR -- Final Val Acc: {h_const['val_acc'][-1]:.3f}")
print(f"Cosine Annealing -- Final Val Acc: {h_cosine['val_acc'][-1]:.3f}")
print('\nCosine annealing gradually reduces the learning rate, allowing t
print('to fine-tune its solution in the later stages of training.')
```



Constant LR -- Final Val Acc: 0.967

Cosine Annealing -- Final Val Acc: 0.967

Cosine annealing gradually reduces the learning rate, allowing the optimizer to fine-tune its solution in the later stages of training.

Exercise 3: Batch Size Effects

Train the same model with batch sizes of 8, 32, 128, and the full dataset. Compare:

- Training speed (loss per epoch)
- Final generalization (validation accuracy)
- Gradient noise (measure variance of loss across batches)

Hypothesis: Small batches = more noise = better generalization but slower convergence. Large = less noise = faster convergence but potentially worse generalization.

```
In [16]: # --- Exercise 3: Batch Size Effects ---
# SOLUTION

np.random.seed(42)

batch_sizes = [8, 32, 128, X_train.shape[0]]
batch_labels = ['8', '32', '128', f'{X_train.shape[0]} (full)']
batch_colors = ['#e74c3c', '#2ecc71', '#3498db', '#9b59b6']
batch_histories = []

for bs in batch_sizes:
    np.random.seed(42)
    m = MLP([64, 128, 64, 10], activation='relu', init_method='he')
    h = train_model(m, X_train, y_train, X_val, y_val,
                    epochs=80, batch_size=bs, lr=0.05, momentum=0.9, verb
    batch_histories.append(h)

# Plot
fig, axes = plt.subplots(1, 3, figsize=(17, 5))
```

```

for i, (bs_label, h, color) in enumerate(zip(batch_labels, batch_histories)):
    axes[0].plot(h['train_loss'], label=f'BS={bs_label}', linewidth=2, color=color)
    axes[0].set_xlabel('Epoch', fontsize=12)
    axes[0].set_ylabel('Training Loss', fontsize=12)
    axes[0].set_title('Training Loss', fontsize=13)
    axes[0].legend(fontsize=10)
    axes[0].grid(True, alpha=0.3)

for i, (bs_label, h, color) in enumerate(zip(batch_labels, batch_histories)):
    axes[1].plot(h['val_acc'], label=f'BS={bs_label}', linewidth=2, color=color)
    axes[1].set_xlabel('Epoch', fontsize=12)
    axes[1].set_ylabel('Validation Accuracy', fontsize=12)
    axes[1].set_title('Validation Accuracy', fontsize=13)
    axes[1].legend(fontsize=10)
    axes[1].grid(True, alpha=0.3)
    axes[1].set_ylim([0, 1.05])

# Final accuracy comparison
final_accs = [h['val_acc'][-1] for h in batch_histories]
bars = axes[2].bar(range(len(batch_sizes)), final_accs, color=batch_color,
                  edgecolor='black', linewidth=0.5)
axes[2].set_xticks(range(len(batch_sizes)))
axes[2].set_xticklabels([f'BS={l}' for l in batch_labels], fontsize=10)
axes[2].set_ylabel('Final Validation Accuracy', fontsize=12)
axes[2].set_title('Final Accuracy by Batch Size', fontsize=13)
axes[2].grid(True, alpha=0.3, axis='y')
axes[2].set_ylim([0.8, 1.0])
for bar, acc in zip(bars, final_accs):
    axes[2].text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.02,
                 f'{acc:.3f}', ha='center', va='bottom', fontsize=11, fontcolor='black')

plt.suptitle('Effect of Batch Size on Training', fontsize=15, y=1.02)
plt.tight_layout()
plt.show()

print('Observations:')
print('- Small batches: noisier gradients, but the noise can help escape')
print('- Large batches: smoother convergence, but may get stuck in sharp')
print('- In practice, batch size is a key hyperparameter that interacts w')
print(' (often: if you double batch size, double the learning rate)')

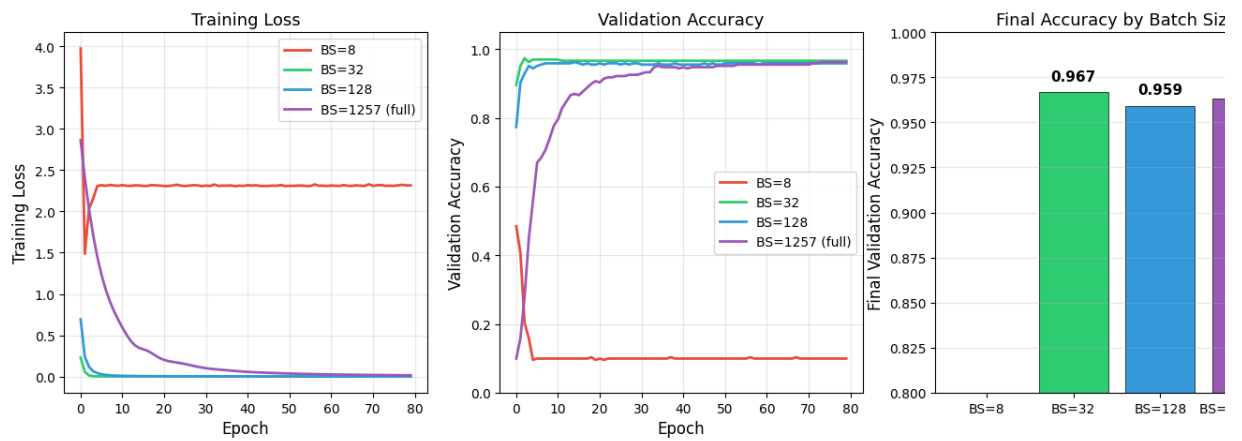
```

```

/tmp/ipykernel_2911/4007798829.py:53: UserWarning: Tight layout not applied.
bottom and top margins cannot be made large enough to accommodate all Axes
decorations.
    plt.tight_layout()

```

Effect of Batch Size on Training



0.100

Observations:

- Small batches: noisier gradients, but the noise can help escape sharp minima
- Large batches: smoother convergence, but may get stuck in sharp minima
- In practice, batch size is a key hyperparameter that interacts with learning rate (often: if you double batch size, double the learning rate)

Summary

In this notebook, we built a complete matrix-based neural network from scratch in NumPy and we explore three fundamental aspects of training dynamics:

Key Curriculum Points

③ **Overparameterization Helps:** Models with far more parameters than training samples can still generalize, thanks to the implicit regularization of SGD. This challenges classical statistical learning theory.

⑥ **Geometry Matters:** The loss landscape determines training success. Flat minima generalize better than sharp ones. SGD with momentum naturally favors flat minima due to gradient noise. The learning rate, batch size, and momentum all influence which minimum the optimizer finds.

⑦ **Capacity $\neq$ Performance:** Having more parameters doesn't guarantee better results. Initialization (He vs Xavier vs naive), activation function choice, learning rate, and optimizer details dominate raw model capacity.

What's Next?

In **Notebook 3** (Generative Models from Scratch), we'll build on these foundations to create models that can *generate* new data — character-level language models and autoencoders — still in pure Python and NumPy.

Notebook 3: Generative Models from Scratch

From Classification to Creation

In Notebooks 1 and 2, we trained neural networks to **classify** -- to map inputs to labels. But the most exciting recent breakthroughs in AI are about **generation** -- creating new data that *looks like* the data. ChatGPT generates text, DALL-E generates images, and music models generate audio. How does this work?

The core idea is deceptively simple: instead of learning $P(y|\mathbf{x})$ (the probability of a label given an input), generative models learn $P(\mathbf{x})$ -- the probability distribution over the data itself. If you can model $P(\mathbf{x})$, you can *sample* from it to create new data points.

In this notebook, we build two generative models entirely in NumPy:

1. **A character-level language model** -- learns to generate text one character at a time by modeling $P(c_{t+1} | c_1, \dots, c_t)$.
2. **A tiny autoencoder** -- learns a compressed representation of MNIST digits and generates new digits by decoding random points in that latent space.

Curriculum Point Covered

#	Concept	Where
④	Neural networks don't "understand"	Sections 3, 5, 6

We also **tie together** all concepts from the series: backpropagation (point ⑤), activation function (point ②), loss landscapes (point ⑥), and capacity (point ⑦).

Prerequisites: Notebooks 1 & 2 (backprop, matrix MLP, cross-entropy, SGD). **Tools:** Python + NumPy + matplotlib. No PyTorch.

2. Setup

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from collections import Counter, defaultdict
import string

np.random.seed(42)
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

print('Setup complete.')
```

3. Character-Level Language Modeling

The Idea

A **language model** assigns probabilities to sequences of tokens (characters, words, or subwords). The simplest useful language model is the **bigram model**: it predicts the next character using *only* the previous character.

$$P(c_{t+1} | c_1, \dots, c_t) \approx P(c_{t+1} | c_t)$$

This is a drastic simplification -- real language has long-range dependencies -- but it's a perfect starting point because:

- It's easy to understand and implement
- It already captures some statistical structure of language
- Its limitations make **curriculum point** ④ crystal clear

Let's build one from scratch.

3.1 Preparing the Text Corpus

We'll use a small Shakespeare-like text corpus. If the file isn't available, we'll generate a synthetic corpus from a famous soliloquy.

```
In [ ]: # Create a small text corpus (Shakespeare excerpt)
corpus = (
    "To be, or not to be, that is the question: "
    "Whether 'tis nobler in the mind to suffer "
    "The slings and arrows of outrageous fortune, "
    "Or to take arms against a sea of troubles, "
    "And by opposing end them. To die, to sleep; "
    "No more; and by a sleep to say we end "
    "The heart-ache and the thousand natural shocks "
    "That flesh is heir to: 'tis a consummation "
    "Devoutly to be wish'd. To die, to sleep; "
    "To sleep, perchance to dream: ay, there's the rub; "
    "For in that sleep of death what dreams may come, "
    "When we have shuffled off this mortal coil, "
    "Must give us pause. There's the respect "
    "That makes calamity of so long life; "
    "For who would bear the whips and scorns of time, "
    "The oppressor's wrong, the proud man's contumely, "
    "The pangs of despised love, the law's delay, "
    "The insolence of office and the spurns "
    "That patient merit of the unworthy takes, "
    "When he himself might his quietus make "
    "With a bare bodkin? Who would fardels bear, "
    "To grunt and sweat under a weary life, "
    "But that the dread of something after death, "
    "The undiscover'd country from whose bourn "
    "No traveller returns, puzzles the will "
    "And makes us rather bear those ills we have "
    "Than fly to others that we know not of? "
```

```

    "Thus conscience does make cowards of us all; "
    "And thus the native hue of resolution "
    "Is sicklied o'er with the pale cast of thought, "
    "And enterprises of great pith and moment "
    "With this regard their currents turn awry, "
    "And lose the name of action."
)

# Repeat the corpus to get more training data
corpus = corpus * 10

# Build vocabulary
chars = sorted(set(corpus))
vocab_size = len(chars)
char_to_idx = {c: i for i, c in enumerate(chars)}
idx_to_char = {i: c for i, c in enumerate(chars)}

print(f'Corpus length: {len(corpus)} characters')
print(f'Vocabulary size: {vocab_size} unique characters')
print(f'Characters: {chars}')

```

3.2 Bigram Count Model

The simplest language model: count how often each character follows each other character, then normalize to get probabilities.

```

In [ ]: # Build bigram counts
bigram_counts = np.zeros((vocab_size, vocab_size), dtype=np.float64)

for i in range(len(corpus) - 1):
    c1 = char_to_idx[corpus[i]]
    c2 = char_to_idx[corpus[i + 1]]
    bigram_counts[c1, c2] += 1

# Add smoothing to avoid zero probabilities
bigram_probs = bigram_counts + 1 # Laplace smoothing
bigram_probs = bigram_probs / bigram_probs.sum(axis=1, keepdims=True)

# Visualize the bigram probability matrix
fig, ax = plt.subplots(figsize=(12, 10))
im = ax.imshow(bigram_probs, cmap='Blues')
ax.set_xticks(range(vocab_size))
ax.set_yticks(range(vocab_size))
ax.set_xticklabels(chars, fontsize=8)
ax.set_yticklabels(chars, fontsize=8)
ax.set_xlabel('Next character')
ax.set_ylabel('Current character')
ax.set_title('Bigram Probability Matrix P(next | current)')
plt.colorbar(im)
plt.tight_layout()
plt.show()

# Show top transitions
print('Top 10 most likely bigrams:')
flat_idx = np.argsort(bigram_counts.ravel())[::-1][:10]
for idx in flat_idx:
    r, c = divmod(idx, vocab_size)
    print(f' "{chars[r]}{chars[c]}" : {int(bigram_counts[r, c])} occurrences')

```

3.3 Sampling from the Bigram Model

Now we can **generate** text by sampling from these learned probabilities. This is the core of generative modeling: learn a distribution, then sample from it.

```

In [ ]: def sample_bigram(bigram_probs, idx_to_char, char_to_idx, seed_char=' ',
    """Generate text by sampling from bigram probabilities."""
    result = [seed_char]
    current = char_to_idx[seed_char]
    for _ in range(length):
        probs = bigram_probs[current]
        next_idx = np.random.choice(len(probs), p=probs)
        result.append(idx_to_char[next_idx])
        current = next_idx
    return ''.join(result)

print('=== Bigram Model Output ===')
print(sample_bigram(bigram_probs, idx_to_char, char_to_idx, seed_char='T', length=300))
print()
print('=== Original text (first 300 chars) ===')
print(corpus[:300])

```

3.4 Observation: Statistical Patterns, Not Understanding

Look at the bigram model's output. You'll notice:

- **Common letter pairs** appear frequently (e.g., "th", "he", "in") -- the model learned these
- **Word-like fragments** sometimes emerge -- but not real words
- **No grammar, no meaning** -- it can't plan ahead even one character

Curriculum Point ④: The model has learned *statistical structure* -- which characters tend to follow which -- but it has zero understanding of English. It doesn't know what "to be" means; it only knows that 'b' often follows "to " in the training data.

Can we do better? Yes -- by giving the model more context.

4. Neural Network Character Model

The bigram model only looks at **one** previous character. Let's give our model a wider **context window** -- the last k characters -- and use a small MLP to map that context to a probability distribution over next character.

This is exactly what Bengio et al. (2003) proposed in their landmark paper "A Neural Probabilistic Language Model" -- one of the foundational ideas behind modern LLMs.

Architecture:

1. **Embedding layer:** Each character gets a learned vector representation
2. **Hidden layer:** Processes the concatenated embeddings
3. **Output layer:** Produces logits for each vocabulary character
4. **Softmax:** Converts logits to probabilities

4.1 Preparing Training Data with Context

We create (context, target) pairs where context is a window of previous characters.

```
In [ ]: # Hyperparameters
CONTEXT_SIZE = 8      # Look at the last 8 characters
EMBED_DIM = 16        # Each character gets a 16-dim embedding
HIDDEN_DIM = 128      # Hidden layer size

# Build training data: (context of CONTEXT_SIZE chars) -> next char
def build_dataset(text, context_size):
    X, Y = [], []
    for i in range(context_size, len(text)):
        context = [char_to_idx[text[j]] for j in range(i - context_size, i)]
        target = char_to_idx[text[i]]
        X.append(context)
        Y.append(target)
    return np.array(X), np.array(Y)
```

```

X_train, Y_train = build_dataset(corpus, CONTEXT_SIZE)
print(f'Training samples: {len(X_train)}')
print(f'Input shape: {X_train.shape} (each row = {CONTEXT_SIZE} characters)')
print(f'Target shape: {Y_train.shape}')

# Show a few examples
print('\nExample (context -> target):')
for i in range(3):
    ctx_chars = ''.join(idx_to_char[c] for c in X_train[i])
    tgt_char = idx_to_char[Y_train[i]]
    print(f' "{ctx_chars}" -> "{tgt_char}"')

```

4.2 The MLP Language Model

We implement an MLP with:

- A **character embedding matrix** $\mathbf{C} \in \mathbb{R}^{V \times d}$ that maps each character index to a dense vector
- A **hidden layer** with ReLU activation
- An **output layer** with softmax

This is fully manual -- forward pass, loss, backprop, and parameter updates -- just like Notebook applied to language modeling.

```

In [ ]: class CharMLP:
    """Character-level MLP language model."""

    def __init__(self, vocab_size, context_size, embed_dim, hidden_dim):
        self.vocab_size = vocab_size
        self.context_size = context_size
        self.embed_dim = embed_dim
        self.hidden_dim = hidden_dim

        # Embedding matrix: each character gets an embed_dim vector
        self.C = np.random.randn(vocab_size, embed_dim) * 0.1

        # Hidden layer
        fan_in = context_size * embed_dim
        self.W1 = np.random.randn(fan_in, hidden_dim) * np.sqrt(2.0 / fan_in)
        self.b1 = np.zeros(hidden_dim)

        # Output layer
        self.W2 = np.random.randn(hidden_dim, vocab_size) * np.sqrt(2.0 / hidden_dim)
        self.b2 = np.zeros(vocab_size)

        # Parameter count
        n_params = (vocab_size * embed_dim + fan_in * hidden_dim + hidden_dim * vocab_size + vocab_size)
        print(f'CharMLP: {n_params:,} parameters')
        print(f' Embedding: {vocab_size} x {embed_dim} = {vocab_size * embed_dim:,}')
        print(f' Hidden: {fan_in} x {hidden_dim} + {hidden_dim} = {fan_in * hidden_dim + hidden_dim:,}')
        print(f' Output: {hidden_dim} x {vocab_size} + {vocab_size} = {hidden_dim * vocab_size + vocab_size:,}')

    def forward(self, X):
        """Forward pass. X is (batch, context_size) of integer indices."""
        # Lookup embeddings: (batch, context_size) -> (batch, context_size * embed_dim)
        self.emb = self.C[X] # fancy indexing

```

```

# Flatten: (batch, context_size * embed_dim)
batch_size = X.shape[0]
self.emb_flat = self.emb.reshape(batch_size, -1)

# Hidden layer
self.z1 = self.emb_flat @ self.W1 + self.b1
self.h1 = np.maximum(0, self.z1) # ReLU

# Output logits
self.logits = self.h1 @ self.W2 + self.b2

# Softmax (numerically stable)
logits_max = self.logits.max(axis=1, keepdims=True)
exp_logits = np.exp(self.logits - logits_max)
self.probs = exp_logits / exp_logits.sum(axis=1, keepdims=True)

return self.probs

def loss(self, probs, targets):
    """Cross-entropy loss."""
    batch_size = len(targets)
    log_probs = np.log(probs[np.arange(batch_size), targets] + 1e-10)
    return -log_probs.mean()

def backward(self, X, targets):
    """Manual backpropagation."""
    batch_size = len(targets)

    # Gradient of loss w.r.t. logits (softmax + cross-entropy shortcut)
    dlogits = self.probs.copy()
    dlogits[np.arange(batch_size), targets] -= 1
    dlogits /= batch_size

    # Output layer gradients
    self.dW2 = self.h1.T @ dlogits
    self.db2 = dlogits.sum(axis=0)

    # Backprop through hidden layer
    dh1 = dlogits @ self.W2.T
    dz1 = dh1 * (self.z1 > 0).astype(float) # ReLU derivative

    # Hidden layer gradients
    self.dW1 = self.emb_flat.T @ dz1
    self.db1 = dz1.sum(axis=0)

    # Backprop through embedding
    demb_flat = dz1 @ self.W1.T
    demb = demb_flat.reshape(self.emb.shape) # (batch, context_size, embed_dim)

    # Accumulate embedding gradients
    self.dC = np.zeros_like(self.C)
    for i in range(self.context_size):
        np.add.at(self.dC, X[:, i], demb[:, i, :])

def step(self, lr):
    """SGD parameter update."""
    self.C -= lr * self.dC
    self.W1 -= lr * self.dW1
    self.b1 -= lr * self.db1

```

```

        self.W2 -= lr * self.dW2
        self.b2 -= lr * self.db2

model = CharMLP(vocab_size, CONTEXT_SIZE, EMBED_DIM, HIDDEN_DIM)

```

4.3 Training the Character Model

```

In [ ]: # Training loop
EPOCHS = 50
BATCH_SIZE = 256
LR = 0.01
n_samples = len(X_train)

loss_history = []

for epoch in range(EPOCHS):
    # Shuffle
    perm = np.random.permutation(n_samples)
    X_shuf = X_train[perm]
    Y_shuf = Y_train[perm]

    epoch_loss = 0.0
    n_batches = 0

    for start in range(0, n_samples, BATCH_SIZE):
        end = min(start + BATCH_SIZE, n_samples)
        xb = X_shuf[start:end]
        yb = Y_shuf[start:end]

        probs = model.forward(xb)
        batch_loss = model.loss(probs, yb)
        model.backward(xb, yb)
        model.step(LR)

        epoch_loss += batch_loss
        n_batches += 1

    avg_loss = epoch_loss / n_batches
    loss_history.append(avg_loss)

    if (epoch + 1) % 10 == 0:
        print(f'Epoch {epoch+1}/{EPOCHS} Loss: {avg_loss:.4f}')

# Plot training loss
plt.figure(figsize=(8, 4))
plt.plot(loss_history)
plt.xlabel('Epoch')
plt.ylabel('Cross-Entropy Loss')
plt.title('Character Model Training Loss')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

4.4 Generating Text with Temperature Sampling

Temperature is a crucial concept in generative models. It controls the "creativity" of the output:

- **Low temperature** ($T \rightarrow 0$): The model becomes more deterministic, always picking the most likely character. Output is repetitive but "safe."
- **High temperature** ($T \rightarrow \infty$): The distribution becomes uniform -- every character is equally likely. Output is random noise.
- **Temperature = 1.0**: The model samples from the learned distribution as-is.

Mathematically, temperature scaling modifies the softmax:

$$P(c_i) = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

where z_i are the logits and T is the temperature.

```
In [ ]: def generate_text(model, seed_text, length=200, temperature=1.0):
        """Generate text using the trained MLP character model."""
        # Pad seed if needed
        if len(seed_text) < CONTEXT_SIZE:
            seed_text = ' ' * (CONTEXT_SIZE - len(seed_text)) + seed_text

        context = [char_to_idx.get(c, 0) for c in seed_text[-CONTEXT_SIZE:]]
        result = list(seed_text)

        for _ in range(length):
            x = np.array([context])
            probs = model.forward(x)
            logits = model.logits[0]

            # Apply temperature
            scaled_logits = logits / temperature
            scaled_logits -= scaled_logits.max()
            exp_l = np.exp(scaled_logits)
            temp_probs = exp_l / exp_l.sum()

            next_idx = np.random.choice(vocab_size, p=temp_probs)
            result.append(idx_to_char[next_idx])
            context = context[1:] + [next_idx]

        return ''.join(result)

# Generate with different temperatures
print('=' * 60)
for temp in [0.3, 0.7, 1.0, 1.5]:
    print(f'\n--- Temperature = {temp} ---')
    print(generate_text(model, 'To be', length=200, temperature=temp))
    print()
```

4.5 Comparing Bigram vs MLP Generation

Let's put the two models side by side. The MLP model has more context and should produce better looking text -- but both are still just doing statistical pattern matching.


```
In [ ]: print('=== BIGRAM MODEL (1-char context) ===')
print(sample_bigram(bigram_probs, idx_to_char, char_to_idx, seed_char='T')
print()
print('=== MLP MODEL (8-char context) ===')
print(generate_text(model, 'To be or', length=200, temperature=0.8))
print()
print('=== ORIGINAL TEXT ===')
print(corpus[:200])
```

4.6 Key Insight: More Context = Better Statistics, Not Understanding

The MLP model produces more convincing text than the bigram model because it has access to context (8 characters vs 1). But it still doesn't "understand" anything about Shakespeare, English or the concept of words.

Curriculum Point ④: Both models learn statistical regularities:

- The bigram model learns: "after 't', 'h' is likely"
- The MLP model learns: "after 'To be, o', 'r' is likely"

Neither model knows what "to be" *means*. They are sophisticated probability machines that detect and reproduce patterns. This same principle scales all the way up to GPT-4 and beyond -- the patterns become richer, the context window grows to millions of tokens, but the fundamental mechanism is the same: **statistical pattern matching over sequences**

5. Visualizing What the Model Learned

Let's peek inside the model to see what the character embeddings look like. If the model learned meaningful representations, characters that behave similarly (e.g., vowels, consonants) should be close together in embedding space.

```
In [ ]: # Visualize character embeddings using PCA
# Simple 2D PCA by hand (no sklearn needed)
C = model.C # (vocab_size, embed_dim)
C_centered = C - C.mean(axis=0)
cov = C_centered.T @ C_centered / (len(C_centered) - 1)
eigenvalues, eigenvectors = np.linalg.eigh(cov)

# Take top 2 eigenvectors (largest eigenvalues are at the end for eigh)
top2 = eigenvectors[:, -2:]
C_2d = C_centered @ top2

# Plot
fig, ax = plt.subplots(figsize=(10, 8))
for i, char in enumerate(chars):
    ax.scatter(C_2d[i, 0], C_2d[i, 1], s=100, zorder=5)
    label = repr(char) if char == ' ' else char
    ax.annotate(label, (C_2d[i, 0], C_2d[i, 1]),
                fontsize=12, ha='center', va='bottom',
                xytext=(0, 5), textcoords='offset points')
```

```

ax.set_title('Character Embeddings (PCA to 2D)')
ax.set_xlabel('PC 1')
ax.set_ylabel('PC 2')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Highlight: the model groups characters by their statistical role, not m
vowels = 'aeiou'
consonants = 'bcdfghjklmnpqrstvwxyz'
print('Vowel positions:', {c: C_2d[char_to_idx[c]].round(2).tolist()
                           for c in vowels if c in char_to_idx})
print('Some consonant positions:', {c: C_2d[char_to_idx[c]].round(2).toli
                                   for c in 'thrns' if c in char_to_idx})

```

What do we see? The embeddings organize characters by their **statistical behavior**:

- Vowels tend to cluster together (they appear in similar contexts)
- Common consonants like 't', 'h', 'n', 's' form their own clusters
- Punctuation and spaces occupy distinct regions

This is a microcosm of how word embeddings work in large language models (e.g., Word2Vec, GPT). The representations are shaped entirely by co-occurrence statistics by any semantic understanding.

6. Tiny Autoencoder for MNIST Digit Generation

Now let's build a different kind of generative model: an **autoencoder**. Instead of modeling sequences, an autoencoder learns a compressed representation of data and can generate new samples by decoding random points in that compressed space.

The Architecture

An autoencoder has two parts:

1. **Encoder:** Maps input $\mathbf{x} \in \mathbb{R}^{784}$ to a low-dimensional **latent code** $\mathbf{z} \in \mathbb{R}^d$ (where $d \ll 784$)
2. **Decoder:** Maps $\mathbf{z}$ back to a reconstruction $\hat{\mathbf{x}} \in \mathbb{R}^{784}$

The loss is the **reconstruction error**: $\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$

If the autoencoder works, it means the latent space captures the essential structure of digits. We generate new digits by sampling random $\mathbf{z}$ vectors and decoding.

$\text{Encoder: } \mathbf{x} \rightarrow \mathbf{z} \quad \text{Decoder: } \mathbf{z} \rightarrow \hat{\mathbf{x}}$

6.1 Loading MNIST Data

```

In [ ]: # Load sklearn digits (8x8 images, 10 classes)
        from sklearn.datasets import load_digits

```

```

digits = load_digits()
X_digits = digits.data / 16.0 # Normalize to [0, 1]
y_digits = digits.target

print(f'Dataset: {X_digits.shape[0]} images, each {X_digits.shape[1]} fea
print(f'Pixel range: [{X_digits.min():.2f}, {X_digits.max():.2f}]')

# Show some digits
fig, axes = plt.subplots(2, 10, figsize=(12, 3))
for i in range(20):
    ax = axes[i // 10, i % 10]
    ax.imshow(X_digits[i].reshape(8, 8), cmap='gray')
    ax.set_title(str(y_digits[i]), fontsize=8)
    ax.axis('off')
plt.suptitle('Sample Digits from sklearn.datasets')
plt.tight_layout()
plt.show()

```

6.2 Building the Autoencoder

We build a simple autoencoder with one hidden layer in the encoder and one in the decoder. The bottleneck (latent dimension) is just **2** -- so we can visualize the latent space.

```

In [ ]: class Autoencoder:
    """Simple autoencoder with one hidden layer in encoder and decoder."""

    def __init__(self, input_dim, hidden_dim, latent_dim):
        self.input_dim = input_dim
        self.hidden_dim = hidden_dim
        self.latent_dim = latent_dim

        # Encoder: input -> hidden -> latent
        self.W_enc1 = np.random.randn(input_dim, hidden_dim) * np.sqrt(2.
        self.b_enc1 = np.zeros(hidden_dim)
        self.W_enc2 = np.random.randn(hidden_dim, latent_dim) * np.sqrt(2
        self.b_enc2 = np.zeros(latent_dim)

        # Decoder: latent -> hidden -> output
        self.W_dec1 = np.random.randn(latent_dim, hidden_dim) * np.sqrt(2
        self.b_dec1 = np.zeros(hidden_dim)
        self.W_dec2 = np.random.randn(hidden_dim, input_dim) * np.sqrt(2.
        self.b_dec2 = np.zeros(input_dim)

        n_params = (input_dim * hidden_dim + hidden_dim +
                     hidden_dim * latent_dim + latent_dim +
                     latent_dim * hidden_dim + hidden_dim +
                     hidden_dim * input_dim + input_dim)
        print(f'Autoencoder: {n_params:,} parameters')
        print(f' Architecture: {input_dim} -> {hidden_dim} -> {latent_di

    def encode(self, X):
        """Encode input to latent space."""
        self.enc_z1 = X @ self.W_enc1 + self.b_enc1
        self.enc_h1 = np.maximum(0, self.enc_z1) # ReLU
        self.z = self.enc_h1 @ self.W_enc2 + self.b_enc2 # Linear (no ac
        return self.z

```

```

def decode(self, z):
    """Decode latent code to reconstruction."""
    self.dec_z1 = z @ self.W_dec1 + self.b_dec1
    self.dec_h1 = np.maximum(0, self.dec_z1) # ReLU
    self.logits_out = self.dec_h1 @ self.W_dec2 + self.b_dec2
    # Sigmoid to map output to [0, 1]
    self.x_hat = 1.0 / (1.0 + np.exp(-np.clip(self.logits_out, -15, 15)))
    return self.x_hat

def forward(self, X):
    """Full forward pass."""
    self.X_input = X
    z = self.encode(X)
    x_hat = self.decode(z)
    return x_hat

def loss(self, X, x_hat):
    """MSE reconstruction loss."""
    return np.mean((X - x_hat) ** 2)

def backward(self, X):
    """Manual backpropagation through the autoencoder."""
    batch_size = X.shape[0]
    x_hat = self.x_hat

    # d(MSE)/d(x_hat) = 2/N * (x_hat - X)
    dx_hat = 2.0 * (x_hat - X) / batch_size

    # Through sigmoid: d/d(logits) = dx_hat * sigmoid * (1 - sigmoid)
    dsigmoid = dx_hat * x_hat * (1 - x_hat)

    # Decoder layer 2
    self.dW_dec2 = self.dec_h1.T @ dsigmoid
    self.db_dec2 = dsigmoid.sum(axis=0)

    # Through decoder ReLU
    ddec_h1 = dsigmoid @ self.W_dec2.T
    ddec_z1 = ddec_h1 * (self.dec_z1 > 0).astype(float)

    # Decoder layer 1
    self.dW_dec1 = self.z.T @ ddec_z1
    self.db_dec1 = ddec_z1.sum(axis=0)

    # Through latent (linear)
    dz = ddec_z1 @ self.W_dec1.T

    # Encoder layer 2 (linear output)
    self.dW_enc2 = self.enc_h1.T @ dz
    self.db_enc2 = dz.sum(axis=0)

    # Through encoder ReLU
    denc_h1 = dz @ self.W_enc2.T
    denc_z1 = denc_h1 * (self.enc_z1 > 0).astype(float)

    # Encoder layer 1
    self.dW_enc1 = X.T @ denc_z1
    self.db_enc1 = denc_z1.sum(axis=0)

def step(self, lr):
    """SGD parameter update."""

```

```

self.W_enc1 -= lr * self.dW_enc1
self.b_enc1 -= lr * self.db_enc1
self.W_enc2 -= lr * self.dW_enc2
self.b_enc2 -= lr * self.db_enc2
self.W_dec1 -= lr * self.dW_dec1
self.b_dec1 -= lr * self.db_dec1
self.W_dec2 -= lr * self.dW_dec2
self.b_dec2 -= lr * self.db_dec2

# Create autoencoder: 64 -> 32 -> 2 -> 32 -> 64
ae = Autoencoder(input_dim=64, hidden_dim=32, latent_dim=2)

```

6.3 Training the Autoencoder

```

In [ ]: # Training
AE_EPOCHS = 200
AE_BATCH = 128
AE_LR = 0.005

ae_losses = []
n_ae = len(X_digits)

for epoch in range(AE_EPOCHS):
    perm = np.random.permutation(n_ae)
    X_shuf = X_digits[perm]

    epoch_loss = 0.0
    n_batches = 0

    for start in range(0, n_ae, AE_BATCH):
        end = min(start + AE_BATCH, n_ae)
        xb = X_shuf[start:end]

        x_hat = ae.forward(xb)
        batch_loss = ae.loss(xb, x_hat)
        ae.backward(xb)
        ae.step(AE_LR)

        epoch_loss += batch_loss
        n_batches += 1

    ae_losses.append(epoch_loss / n_batches)

    if (epoch + 1) % 50 == 0:
        print(f'Epoch {epoch+1}/{AE_EPOCHS} MSE: {ae_losses[-1]:.6f}')

plt.figure(figsize=(8, 4))
plt.plot(ae_losses)
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Autoencoder Training Loss')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

6.4 Visualizing Reconstructions

Let's see how well the autoencoder reconstructs digits through its 2D bottleneck.

```
In [ ]: # Show original vs reconstruction
n_show = 10
sample_idx = np.random.choice(len(X_digits), n_show, replace=False)
x_sample = X_digits[sample_idx]
x_recon = ae.forward(x_sample)

fig, axes = plt.subplots(2, n_show, figsize=(12, 3))
for i in range(n_show):
    axes[0, i].imshow(x_sample[i].reshape(8, 8), cmap='gray')
    axes[0, i].axis('off')
    if i == 0:
        axes[0, i].set_title('Original', fontsize=9)

    axes[1, i].imshow(x_recon[i].reshape(8, 8), cmap='gray')
    axes[1, i].axis('off')
    if i == 0:
        axes[1, i].set_title('Reconstructed', fontsize=9)

plt.suptitle('Autoencoder: Original vs Reconstructed (2D latent space)')
plt.tight_layout()
plt.show()
```

6.5 The Latent Space: A Map of Digits

Since our latent space is 2D, we can directly visualize it. Each digit gets mapped to a point in 2D we can see how the autoencoder organizes them.

```
In [ ]: # Encode all digits
z_all = ae.encode(X_digits)

# Plot latent space colored by digit class
fig, ax = plt.subplots(figsize=(10, 8))
scatter = ax.scatter(z_all[:, 0], z_all[:, 1], c=y_digits, cmap='tab10',
                    alpha=0.6, s=15)
plt.colorbar(scatter, label='Digit class')
ax.set_xlabel('Latent dimension 1')
ax.set_ylabel('Latent dimension 2')
ax.set_title('Autoencoder Latent Space (colored by digit)')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('Notice: similar digits cluster together in latent space.')
print('The autoencoder discovered digit similarity from pixel data alone!')
```

6.6 Generating New Digits

Now for the fun part: **generation**. We can sample random points from the latent space and decode to create new digit images that never existed in the training data.

We'll also create a **latent space grid** -- systematically decoding points across the 2D space to see how digit appearance varies smoothly.

```
In [ ]: # Generate digits on a grid through latent space
n_grid = 15

# Find range of latent space
z_min = z_all.min(axis=0)
z_max = z_all.max(axis=0)
margin = (z_max - z_min) * 0.1

z1_range = np.linspace(z_min[0] + margin[0], z_max[0] - margin[0], n_grid)
z2_range = np.linspace(z_min[1] + margin[1], z_max[1] - margin[1], n_grid)

# Create grid of images
fig, axes = plt.subplots(n_grid, n_grid, figsize=(12, 12))

for i, z2 in enumerate(reversed(z2_range)):
    for j, z1 in enumerate(z1_range):
        z_point = np.array([z1, z2])
        generated = ae.decode(z_point)
        axes[i, j].imshow(generated.reshape(8, 8), cmap='gray')
        axes[i, j].axis('off')

plt.suptitle('Latent Space Grid: Decoded Digit Images', fontsize=14)
plt.tight_layout()
plt.show()

print('Each image is generated by decoding a point in the 2D latent space')
print('Notice how digits smoothly morph into each other across the grid.')
```

6.7 Interpolation: Morphing Between Digits

We can also **interpolate** between two digits in latent space. If the autoencoder learned a smooth representation, intermediate points should look like smooth transitions.

```
In [ ]: # Find examples of two different digits
def find_digit(y, digit):
    return np.where(y == digit)[0][0]

pairs = [(0, 1), (3, 8), (4, 9), (2, 7)]
n_steps = 8

fig, axes = plt.subplots(len(pairs), n_steps, figsize=(12, 6))

for row, (d1, d2) in enumerate(pairs):
    idx1, idx2 = find_digit(y_digits, d1), find_digit(y_digits, d2)
    z1 = ae.encode(X_digits[idx1:idx1+1])
    z2 = ae.encode(X_digits[idx2:idx2+1])

    for col, alpha in enumerate(np.linspace(0, 1, n_steps)):
        z_interp = (1 - alpha) * z1 + alpha * z2
        img = ae.decode(z_interp)
        axes[row, col].imshow(img.reshape(8, 8), cmap='gray')
        axes[row, col].axis('off')
        if row == 0:
            axes[row, col].set_title(f'{alpha:.1f}', fontsize=8)
```

```
axes[row, 0].set_ylabel(f'{d1} -> {d2}', fontsize=10)

plt.suptitle('Latent Space Interpolation Between Digits', fontsize=14)
plt.tight_layout()
plt.show()
```

7. What Does "Generative" Mean Mathematically?

Let's step back and think about what our two models are actually doing from a mathematical perspective:

The Language Model

Our character-level model learns a **conditional probability distribution**:

$$P(c_{t+1} \mid c_{t-k+1}, \dots, c_t)$$

The joint probability of a sequence is the product of these conditionals (chain rule of probability):

$$P(c_1, c_2, \dots, c_T) = \prod_{t=1}^T P(c_t \mid c_1, \dots, c_{t-1})$$

This is called **autoregressive modeling** -- it's exactly how GPT works, just with more parameter and much larger context window.

The Autoencoder

The autoencoder learns an **implicit density model**. It doesn't directly give us $P(\mathbf{x})$, but it learns a compressed representation that captures the structure of the data distribution.

- **Encoder:** $\mathbf{z} = f_{\theta}(\mathbf{x})$ compresses the data
- **Decoder:** $\hat{\mathbf{x}} = g_{\phi}(\mathbf{z})$ reconstructs from the code

A **variational autoencoder (VAE)** goes further by explicitly modeling the latent distribution as a Gaussian, giving a principled way to generate new samples. Our basic autoencoder is a stepping stone toward that idea.

The Common Thread

Both models learn **statistical structure** in the data. Neither "understands" the data:

- The language model doesn't know English -- it knows character co-occurrence patterns
- The autoencoder doesn't know what digits "look like" -- it knows which pixel patterns co-occur in training data

This is curriculum point ④ at its core: Neural networks are powerful function approximators that detect and reproduce statistical patterns. The quality of their output depends on the richness of patterns in the training data and the model's capacity to capture them -- not on any form of understanding.

8. Experiment: Does the Model "Understand" or Just Match Patterns?

Let's design a simple experiment to make this concrete. We'll test our language model on tasks that require understanding vs tasks that require pattern matching.

```
In [ ]: # Test 1: Character prediction from common patterns (should work well)
test_common = [
    ('To be, o', 'r'), # Very common in our corpus
    ('the ques', 't'), # Common pattern
    ('of death', ' '), # Common pattern
]

# Test 2: Requires understanding (model should fail)
# We'll reverse common patterns -- if the model understood meaning,
# it could still predict reasonably. Since it's just pattern matching,
# reversed text should confuse it completely.
test_reversed = [
    ('o ,eb oT', 'r'), # Reversed -- same 'meaning' but different statistics
    ('seuq eht', 't'), # Reversed
]

print('=== Test: Common patterns (high confidence expected) ===')
for context, expected in test_common:
    if len(context) >= CONTEXT_SIZE:
        ctx = context[-CONTEXT_SIZE:]
    else:
        ctx = ' ' * (CONTEXT_SIZE - len(context)) + context
    x = np.array([[char_to_idx.get(c, 0) for c in ctx]])
    probs = model.forward(x)[0]
    pred_idx = np.argmax(probs)
    pred_char = idx_to_char[pred_idx]
    expected_prob = probs[char_to_idx.get(expected, 0)] if expected in char_to_idx else 0
    print(f' "{context}" -> predicted: "{pred_char}" (p={probs[pred_idx]:.3f})'
          f'expected: "{expected}" (p={expected_prob:.3f})')

print()
print('=== Test: Reversed patterns (should fail -- no understanding) ===')
for context, expected in test_reversed:
    if len(context) >= CONTEXT_SIZE:
        ctx = context[-CONTEXT_SIZE:]
    else:
        ctx = ' ' * (CONTEXT_SIZE - len(context)) + context
    x = np.array([[char_to_idx.get(c, 0) for c in ctx]])
    probs = model.forward(x)[0]
    pred_idx = np.argmax(probs)
    pred_char = idx_to_char[pred_idx]
    expected_prob = probs[char_to_idx.get(expected, 0)] if expected in char_to_idx else 0
    print(f' "{context}" -> predicted: "{pred_char}" (p={probs[pred_idx]:.3f})'
          f'expected: "{expected}" (p={expected_prob:.3f})')

print()
print('Key insight: The model succeeds on seen patterns but fails on')
print('reversed text, even though the "meaning" is the same.')
print('This proves it relies on statistical patterns, not understanding.')
```

9. Tying It All Together: The Big Picture

Over these three notebooks, we've built neural networks from scratch and explored deep learning's most important ideas. Let's recap:

What We Built

Notebook	What	Key Insight
1: Neuron & Backprop	Scalar autograd, MLP	Backpropagation = chain rule (⑤). ReLU enables deep learning. One hidden layer can approximate anything (①).
2: Training Dynamics	Matrix MLP, MNIST	Initialization matters enormously (⑦). Loss landscape geometry affects generalization (⑥). Overparameterized models generalize via implicit regularization (③).
3: Generative Models	Language model, autoencoder	Neural networks are statistical pattern matchers, not reasoning machines (④).

The Seven Curriculum Points

1. **Universal Approximation:** A single hidden layer *can* approximate any continuous function. Increasing network depth makes it exponentially more efficient. (Notebook 1)
2. **ReLU Changed Everything:** Sigmoid/tanh cause vanishing gradients in deep networks. ReLU's simple $\max(0, x)$ preserves gradient flow and enables sparse activations. (Notebooks 1 & 2)
3. **Overparameterization Helps:** Networks with more parameters than training samples can still generalize. SGD's implicit bias finds flat, generalizable minima. (Notebook 2)
4. **Neural Networks Don't "Understand":** They minimize a loss function by detecting statistical structure. Our language model generates Shakespeare-like text without knowing English. Our autoencoder generates digits without knowing what numbers are. (This notebook)
5. **Backpropagation = Chain Rule:** Automatic differentiation is just systematic application of the chain rule, computed efficiently via dynamic programming (topological sort). (Notebook 1)
6. **Geometry Matters:** The loss landscape is high-dimensional with many local minima. Flat minima generalize better than sharp ones. Initialization and learning rate affect which minimum you find. (Notebook 2)
7. **Capacity $\neq$ Performance:** More layers don't automatically help. Architecture choices, initialization (He/Xavier), normalization, and data quality matter more than raw size. (Notebook 2)

Looking Ahead

In the PyTorch series (Notebooks 4-6), we'll rebuild these ideas using a production deep learning framework. You'll see how PyTorch automates everything we did by hand -- and how that automation enables us to build much larger, more powerful models.

10. Exercises

Exercise 1: Temperature Exploration

Modify the `generate_text` function to use temperature = 0.01 (nearly deterministic) and temperature = 5.0 (nearly random). Explain what you observe and why.

Exercise 2: Deeper Autoencoder

Add a second hidden layer to both the encoder and decoder. Does this improve reconstruction quality? Plot the training loss for both architectures.

Exercise 3: Latent Space Dimension

Change the autoencoder's latent dimension from 2 to 5. Does reconstruction improve? Can you visualize the latent space? (Hint: use PCA to project 5D -> 2D.)

Exercise 4: N-gram Models

Build a trigram model (3-character context) using count-based probabilities (like the bigram model with a 3D tensor). Compare its generation quality to the bigram and MLP models.

Exercise 5: Adversarial Test

Create a short text that is valid English but uses character patterns very different from Shakespeare (e.g., technical jargon). Feed it to the language model -- does it predict well? What does this tell you about the generalization of our model vs a "true understanding" model?

Notebook 4: PyTorch Fundamentals & Autograd

"You built autograd from scratch. Now meet PyTorch's version."

In Series 1 Notebook 1, you built a `Value` class that tracked operations on scalars, built a computation graph, and computed gradients via reverse-mode automatic differentiation. You then assembled `Neuron`, `Layer`, and `MLP` classes and trained a network on the moons dataset.

Now we do the same things -- but with PyTorch. At every step we'll compare: *"Remember when you built this by hand? Here's how PyTorch handles it."*

What this notebook covers

1. **Tensors** -- PyTorch's fundamental data structure (think: NumPy arrays + GPU + autograd)
2. **Autograd** -- the same chain-rule engine you built, but industrial-strength
3. **nn.Module** -- the building block for neural networks
4. **Training loops** -- the canonical forward -> loss -> backward -> step pattern
5. **Universal Approximation in PyTorch** -- shallow-wide vs deep-narrow
6. **Custom Autograd Functions** -- when you need to define your own forward/backward

Curriculum points covered

#	Point	Where
①	Universal Approximation Theorem	§6
②	ReLU changed everything	§3, §4
⑤	Backpropagation = chain rule	§3

```
In [1]: import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

# Reproducibility
torch.manual_seed(42)
np.random.seed(42)

# Plotting defaults
plt.rcParams['figure.figsize'] = (8, 5)
plt.rcParams['figure.dpi'] = 100
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3

print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
```

2. Tensors -- PyTorch's Fundamental Data Structure

Remember working with NumPy arrays in the pure-Python notebooks? PyTorch **tensors** are almost identical -- same shapes, same broadcasting rules, same slicing -- but they add two super-powers:

1. **Autograd**: tensors can track every operation so gradients flow automatically.
2. **GPU acceleration**: tensors can live on a GPU for massive parallelism.

Let's explore.

2.1 Creating Tensors

```
In [2]: # Creating tensors
# From Python lists
a = torch.tensor([1.0, 2.0, 3.0])
print("From list:      ", a, " dtype:", a.dtype)

# From NumPy
np_arr = np.array([[1, 2], [3, 4]], dtype=np.float32)
b = torch.from_numpy(np_arr)
print("From NumPy:\n", b)

# Common constructors
print("\nzeros:  ", torch.zeros(2, 3))
print("ones:    ", torch.ones(2, 3))
print("randn:   ", torch.randn(2, 3))      # standard normal
print("arange:  ", torch.arange(0, 1, 0.2))
print("eye:     ", torch.eye(3))

# Specifying dtype
c = torch.tensor([1, 2, 3], dtype=torch.float64)
print("\nfloat64 tensor:", c, " dtype:", c.dtype)
```

```
From list:      tensor([1., 2., 3.])  dtype: torch.float32
```

```
From NumPy:
tensor([[1., 2.],
        [3., 4.]])
```

```
zeros:  tensor([[0., 0., 0.],
                [0., 0., 0.]])
```

```
ones:   tensor([[1., 1., 1.],
                [1., 1., 1.]])
```

```
randn:  tensor([[ 0.3367,  0.1288,  0.2345],
                [ 0.2303, -1.1229, -0.1863]])
```

```
arange: tensor([0.0000, 0.2000, 0.4000, 0.6000, 0.8000])
```

```
eye:    tensor([[1., 0., 0.],
                [0., 1., 0.],
                [0., 0., 1.]])
```

```
float64 tensor: tensor([1., 2., 3.], dtype=torch.float64)  dtype: torch.float64
```

2.2 Tensor Operations

These should feel familiar -- they mirror NumPy almost exactly.

```
In [3]: # Basic arithmetic
x = torch.tensor([1.0, 2.0, 3.0])
y = torch.tensor([4.0, 5.0, 6.0])

print("x + y      =", x + y)          # element-wise add
print("x * y      =", x * y)          # element-wise mul (Hadamard)
print("x @ y      =", x @ y)          # dot product
print("x ** 2     =", x ** 2)         # element-wise power

# Matrix multiplication
A = torch.randn(2, 3)
B = torch.randn(3, 4)
C = A @ B                               # same as torch.matmul(A, B)
print(f"\nA shape: {A.shape}, B shape: {B.shape}, A@B shape: {C.shape}")

x + y      = tensor([5., 7., 9.])
x * y      = tensor([ 4., 10., 18.])
x @ y      = tensor(32.)
x ** 2     = tensor([1., 4., 9.])

A shape: torch.Size([2, 3]), B shape: torch.Size([3, 4]), A@B shape: torch.Size([2, 4])
```

```
In [4]: # Reshaping
t = torch.arange(12, dtype=torch.float32)
print("Original:      ", t.shape)

t2 = t.reshape(3, 4)                    # reshape (like np.reshape)
print("reshape(3,4): ", t2.shape)

t3 = t.view(2, 6)                       # view -- same data, new shape
print("view(2,6):    ", t3.shape)

t4 = t2.T                               # transpose
print("Transpose:    ", t4.shape)

# Unsqueeze / squeeze -- add or remove dimensions of size 1
t5 = t.unsqueeze(0)                     # (12,) -> (1, 12)
print("unsqueeze(0): ", t5.shape)
t6 = t5.squeeze(0)                      # (1, 12) -> (12,)
print("squeeze(0):   ", t6.shape)
```

```
Original:      torch.Size([12])
reshape(3,4):  torch.Size([3, 4])
view(2,6):     torch.Size([2, 6])
Transpose:     torch.Size([4, 3])
unsqueeze(0):  torch.Size([1, 12])
squeeze(0):    torch.Size([12])
```

```
In [5]: # Broadcasting
# Same rules as NumPy: dimensions are compared right-to-left,
# sizes must match or one of them must be 1.

mat = torch.ones(3, 4)
vec = torch.tensor([1.0, 2.0, 3.0, 4.0]) # shape (4,)
```

```

result = mat + vec      # vec is broadcast across rows
print("mat (3x4) + vec (4,):")
print(result)
print("Result shape:", result.shape)

```

```

mat (3x4) + vec (4,):
tensor([[2., 3., 4., 5.],
        [2., 3., 4., 5.],
        [2., 3., 4., 5.]])
Result shape: torch.Size([3, 4])

```

2.3 NumPy <-> PyTorch

They share the same underlying memory (on CPU), so conversion is free.

```

In [6]: # NumPy <-> Tensor (shared memory on CPU)
np_arr = np.array([1.0, 2.0, 3.0])
t = torch.from_numpy(np_arr)

print("NumPy array:", np_arr)
print("Torch tensor:", t)

# Modify one -> the other changes too (shared memory!)
np_arr[0] = 99.0
print("\nAfter changing np_arr[0] to 99:")
print("  NumPy:", np_arr)
print("  Torch:", t, " <- also changed!")

# Going back to NumPy
t2 = torch.tensor([10.0, 20.0, 30.0])
np_arr2 = t2.numpy()
print("\nTensor -> NumPy:", np_arr2)

```

```

NumPy array: [1. 2. 3.]
Torch tensor: tensor([1., 2., 3.], dtype=torch.float64)

```

```

After changing np_arr[0] to 99:
  NumPy: [99. 2. 3.]
  Torch: tensor([99., 2., 3.], dtype=torch.float64)  <- also changed!

```

```

Tensor -> NumPy: [10. 20. 30.]

```

3. Autograd -- "Backprop is just the chain rule"

⑤ Curriculum Point: Backpropagation = Chain Rule

In Notebook 1, you implemented `backward()` yourself: topological sort of the computation graph, then propagate gradients node-by-node using the chain rule. PyTorch does **exactly the same thing** -- but automatically, for tensors of any shape. Every `torch` operation records itself into a dynamic computation graph. When you call `.backward()`, PyTorch walks that graph in reverse.

3.1 requires_grad and .backward()

```
In [7]: # requires_grad tells PyTorch: "track this tensor"
# Remember in your Value class, every Value tracked _prev and _op?
# In PyTorch, setting requires_grad=True does the same thing.

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)

# Build a computation: z = x^2 * y + y^3
# In your Value class this would be:
#   x = Value(2.0); y = Value(3.0)
#   z = x**2 * y + y**3
z = x**2 * y + y**3

print(f"z = x^2*y + y^3 = {x.item()}^2*{y.item()} + {y.item()}^3 = {z.item()}")
print(f"z.grad_fn: {z.grad_fn}") # shows the last operation

# Backward pass -- just like your z.backward()!
z.backward()

print(f"\ndz/dx = 2xy = 2*{x.item()}*{y.item()} = {x.grad.item()}")
print(f" PyTorch computed: {x.grad.item()} check!")

print(f"\ndz/dy = x^2 + 3y^2 = {x.item()}^2 + 3*{y.item()}^2 = {y.grad.item()}")
print(f" PyTorch computed: {y.grad.item()} check!")
```

```
z = x^2*y + y^3 = 2.0^2*3.0 + 3.0^3 = 39.0
z.grad_fn: <AddBackward0 object at 0x7f16eaf3ca0>
```

```
dz/dx = 2xy = 2*2.0*3.0 = 12.0
PyTorch computed: 12.0 check!
```

```
dz/dy = x^2 + 3y^2 = 2.0^2 + 3*3.0^2 = 31.0
PyTorch computed: 31.0 check!
```

3.2 Gradient Verification -- Matching Your Hand-Built Engine

Let's compute the same expression we used in Notebook 1 and verify PyTorch gets the same gr:

```
In [8]: # Same expression as pure-Python notebook
# Expression: f(a,b) = (a + b) * (b + 1)
# df/da = b + 1
# df/db = a + 2b + 1

a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)

f = (a + b) * (b + 1.0)
f.backward()

# Analytical gradients
df_da_analytical = b.item() + 1 # = 4.0
df_db_analytical = a.item() + 2*b.item() + 1 # = 9.0

print("Expression: f(a,b) = (a + b) * (b + 1)")
print(f" a = {a.item()}, b = {b.item()}, f = {f.item()}")
match_a = "MATCH" if abs(a.grad.item() - df_da_analytical) < 1e-6 else "M"
```



```

match_b = "MATCH" if abs(b.grad.item() - df_db_analytical) < 1e-6 else "M
print(f"\n df/da: PyTorch = {a.grad.item():.4f}, Analytical = {df_da_ana
print(f" df/db: PyTorch = {b.grad.item():.4f}, Analytical = {df_db_analy

```

Expression: $f(a,b) = (a + b) * (b + 1)$
 $a = 2.0, b = 3.0, f = 20.0$

```

df/da: PyTorch = 4.0000, Analytical = 4.0000 [MATCH]
df/db: PyTorch = 9.0000, Analytical = 9.0000 [MATCH]

```

3.3 Visualizing the Computation Graph

In Notebook 1 you built a text-based graph. PyTorch stores the graph in the `.grad_fn` chain. Let's trace it.

```

In [9]: # Walking the computation graph
# Each tensor produced by an operation has a .grad_fn that points back
# to the operation that created it, and .next_functions pointing to inputs

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)
z = x * y + x**2

def trace_graph_fn(fn, indent=0):
    """Trace from a grad_fn node."""
    prefix = " " * indent
    print(f"{prefix}{fn}")
    for child, _ in fn.next_functions:
        if child is not None:
            trace_graph_fn(child, indent + 1)
        else:
            print(f"{' ' * (indent+1)}(leaf tensor)")

def trace_graph(tensor, indent=0):
    """Recursively print the computation graph (like your text visualizat
    prefix = " " * indent
    if tensor.grad_fn:
        print(f"{prefix}{tensor.grad_fn}")
        for child, _ in tensor.grad_fn.next_functions:
            if child is not None:
                trace_graph_fn(child, indent + 1)
            else:
                name = "(leaf tensor)"
                print(f"{' ' * (indent+1)}{name}")
    else:
        name = "leaf"
        if tensor.requires_grad:
            name = f"leaf (requires_grad, data={tensor.item():.1f})"
        print(f"{prefix}{name}")

print("Computation graph for z = x*y + x^2:")
print("=" * 50)
trace_graph(z)

```

Computation graph for $z = x*y + x^2$:

```
=====
<AddBackward0 object at 0x7f1738908160>
  <MulBackward0 object at 0x7f17388b7e20>
    <AccumulateGrad object at 0x7f16eae6ca30>
    <AccumulateGrad object at 0x7f1738d9a410>
  <PowBackward0 object at 0x7f17388b79a0>
    <AccumulateGrad object at 0x7f1738d9a410>
```

3.4 torch.no_grad() -- When and Why

During **inference** (making predictions), we don't need gradients. `torch.no_grad()` disables tracking for efficiency.

Remember: In your pure-Python Value class, every operation created new nodes in the graph. That's expensive! In production, you only want to build the graph during training.

```
In [10]: # torch.no_grad()
x = torch.randn(1000, 1000, requires_grad=True)

# With gradient tracking (training mode)
import time
start = time.time()
for _ in range(10):
    y = x @ x.T
grad_time = time.time() - start

# Without gradient tracking (inference mode)
start = time.time()
with torch.no_grad():
    for _ in range(10):
        y = x @ x.T
no_grad_time = time.time() - start

print(f"With grad tracking:      {grad_time:.4f}s")
print(f"Without (no_grad):      {no_grad_time:.4f}s")
if no_grad_time > 0:
    print(f"Speedup:              {grad_time / no_grad_time:.1f}x")
```

```
With grad tracking:      0.2242s
Without (no_grad):      0.2266s
Speedup:                 1.0x
```

⑤ Key Insight: Backprop is just the chain rule -- and PyTorch automates it

You implemented every `_backward` closure by hand in Notebook 1. PyTorch has hundreds of these registered for every tensor operation. When you call `.backward()`, does exactly what your code did:

1. Topological sort the computation graph
2. Walk it in reverse
3. Accumulate gradients using the chain rule

The only difference? PyTorch handles tensors of any shape, runs on GPU, and is battle-tested by millions of users.

4. Building an MLP with `nn.Module`

In Notebook 1 you built `Neuron`, `Layer`, and `MLP` classes from scratch. Each `Neuron` held weights and a bias. Here's the PyTorch way.

② Curriculum Point: ReLU Changed Everything

In your pure-Python notebook, you saw that sigmoid squashes gradients toward zero in deep networks. ReLU passes gradients through unchanged (gradient is 0 or 1). This is why virtually all modern networks use ReLU or its variants. We'll use `nn.ReLU` throughout.

4.1 `nn.Linear` and `nn.ReLU`

```
In [11]: # nn.Linear: the PyTorch equivalent of your Neuron/Layer
# Your Layer class: each neuron had weights (Value objects) and a bias
# nn.Linear(in_features, out_features) does the same thing:  $y = Wx + b$ 

layer = nn.Linear(in_features=3, out_features=2)

print("Weight shape:", layer.weight.shape)    # (out, in) = (2, 3)
print("Bias shape: ", layer.bias.shape)       # (out,) = (2,)
print("\nWeight:\n", layer.weight.data)
print("Bias:", layer.bias.data)

# Forward pass on a batch of 4 samples, each with 3 features
x = torch.randn(4, 3)
out = layer(x)    # calls layer.forward(x) ->  $x @ W.T + b$ 
print(f"\nInput shape: {x.shape}")
print(f"Output shape: {out.shape}")
```

Weight shape: torch.Size([2, 3])
Bias shape: torch.Size([2])

Weight:
tensor([[-0.0282, -0.0377, -0.2000],
[0.3835, -0.3681, -0.0933]])
Bias: tensor([0.0564, -0.0512])

Input shape: torch.Size([4, 3])
Output shape: torch.Size([4, 2])

```
In [12]: # nn.ReLU vs nn.Sigmoid
# Remember: ReLU changed everything because its gradient is 0 or 1.
# No vanishing gradient!

x = torch.linspace(-3, 3, 200)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

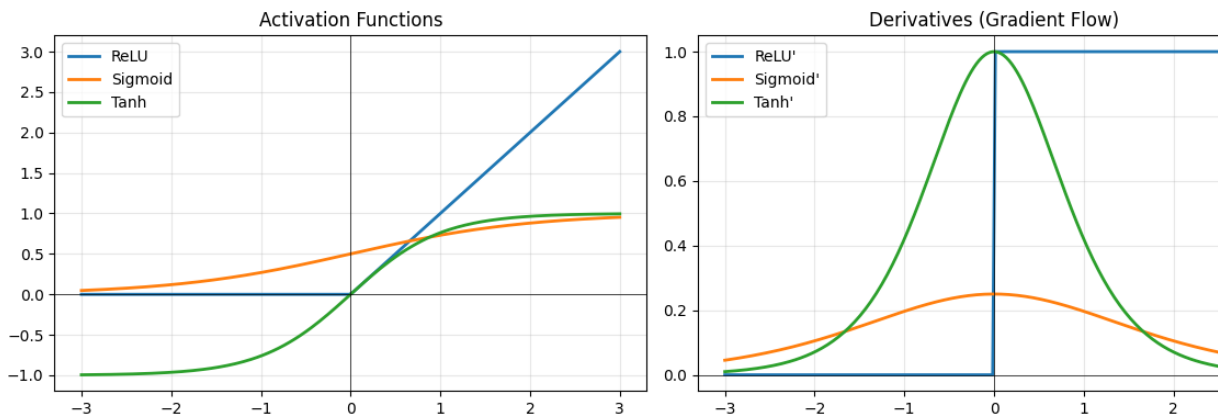
# Functions
axes[0].plot(x.numpy(), torch.relu(x).numpy(), label='ReLU', linewidth=2)
axes[0].plot(x.numpy(), torch.sigmoid(x).numpy(), label='Sigmoid', linewidth=2)
axes[0].plot(x.numpy(), torch.tanh(x).numpy(), label='Tanh', linewidth=2)
axes[0].set_title('Activation Functions')
axes[0].legend()
axes[0].axhline(y=0, color='k', linewidth=0.5)
axes[0].axvline(x=0, color='k', linewidth=0.5)

# Derivatives
# For ReLU: derivative is 1 if x > 0, else 0
relu_grad = (x > 0).float()
# For sigmoid: s'(x) = s(x)(1 - s(x))
sig = torch.sigmoid(x)
sig_grad = sig * (1 - sig)
# For tanh: tanh'(x) = 1 - tanh^2(x)
tanh_val = torch.tanh(x)
tanh_grad = 1 - tanh_val**2

axes[1].plot(x.numpy(), relu_grad.numpy(), label="ReLU", linewidth=2)
axes[1].plot(x.numpy(), sig_grad.numpy(), label="Sigmoid", linewidth=2)
axes[1].plot(x.numpy(), tanh_grad.numpy(), label="Tanh", linewidth=2)
axes[1].set_title('Derivatives (Gradient Flow)')
axes[1].legend()
axes[1].axhline(y=0, color='k', linewidth=0.5)
axes[1].axvline(x=0, color='k', linewidth=0.5)

plt.tight_layout()
plt.show()

print("Notice: Sigmoid derivative maxes out at 0.25.")
print("Chain 10 layers: 0.25^10 =", 0.25**10)
print("ReLU derivative is exactly 1 (for x > 0) -- no vanishing gradient!")
```



Notice: Sigmoid derivative maxes out at 0.25.
Chain 10 layers: $0.25^{10} = 9.5367431640625e-07$
ReLU derivative is exactly 1 (for $x > 0$) -- no vanishing gradient!

4.2 Building a Custom nn.Module

This is the PyTorch equivalent of your `MLP` class from Notebook 1.

```
In [13]: # Custom MLP using nn.Module
# Remember your MLP class?
# class MLP:
#     def __init__(self, nin, nouts):
#         sz = [nin] + nouts
#         self.layers = [Layer(sz[i], sz[i+1]) for i in range(len(nouts)-1)]
#     def __call__(self, x):
#         for layer in self.layers:
#             x = layer(x)
#         return x
# Here's the PyTorch equivalent:

class MLP(nn.Module):
    """Multi-Layer Perceptron, the PyTorch way."""

    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()

        layers = []
        prev_dim = input_dim
        for h in hidden_dims:
            layers.append(nn.Linear(prev_dim, h))
            layers.append(nn.ReLU())
            prev_dim = h
        layers.append(nn.Linear(prev_dim, output_dim))

        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)

# Create a model: 2 inputs -> [16, 16] hidden -> 1 output
model = MLP(input_dim=2, hidden_dims=[16, 16], output_dim=1)
print(model)
print(f"\nTotal parameters: {sum(p.numel() for p in model.parameters())}")
```

```
MLP(
    (network): Sequential(
      (0): Linear(in_features=2, out_features=16, bias=True)
      (1): ReLU()
      (2): Linear(in_features=16, out_features=16, bias=True)
      (3): ReLU()
      (4): Linear(in_features=16, out_features=1, bias=True)
    )
  )
)
```

Total parameters: 337

```
In [14]: # Inspecting parameters
# Remember model.parameters() from your MLP? Same concept.

print("Named parameters:")
print("-" * 50)
for name, param in model.named_parameters():
    print(f" {name:30s} shape={str(list(param.shape)):15s} #params={para

total = sum(p.numel() for p in model.parameters())
print(f"\n {'TOTAL':30s} {'':15s} #params={total}")
```

Named parameters:

network.0.weight	shape=[16, 2]	#params=32
network.0.bias	shape=[16]	#params=16
network.2.weight	shape=[16, 16]	#params=256
network.2.bias	shape=[16]	#params=16
network.4.weight	shape=[1, 16]	#params=16
network.4.bias	shape=[1]	#params=1
TOTAL		#params=337

5. Training Loop

In Notebook 1, your training loop looked like this:

```
for step in range(steps):
    # Forward pass
    ypred = [model(x) for x in xs]
    loss = sum((yp - yt)**2 for yp, yt in zip(ypred, ys))

    # Backward pass
    for p in model.parameters():
        p.grad = 0.0
    loss.backward()

    # Update
    for p in model.parameters():
        p.data -= learning_rate * p.grad
```

The PyTorch version follows the **exact same pattern**, just using PyTorch's tools.

5.1 Loss Functions and Optimizers

```
In [15]: # Loss functions
# Your pure-python loss: sum of squared errors computed manually
# PyTorch provides optimized, numerically stable versions

# Mean Squared Error (regression)
mse_loss = nn.MSELoss()
pred = torch.tensor([1.0, 2.0, 3.0])
target = torch.tensor([1.1, 2.2, 2.8])
print(f"MSE Loss: {mse_loss(pred, target).item():.4f}")

# Binary Cross-Entropy with Logits (binary classification)
bce_loss = nn.BCEWithLogitsLoss()
logits = torch.tensor([0.5, -1.0, 2.0])
labels = torch.tensor([1.0, 0.0, 1.0])
print(f"BCE Loss: {bce_loss(logits, labels).item():.4f}")

# Cross-Entropy (multi-class classification)
ce_loss = nn.CrossEntropyLoss()
logits_mc = torch.tensor([[2.0, 0.5, -1.0], [0.1, 3.0, 0.5]]) # 2 sample
labels_mc = torch.tensor([0, 1]) # correct classes
print(f"CE Loss: {ce_loss(logits_mc, labels_mc).item():.4f}")
```

MSE Loss: 0.0300
BCE Loss: 0.3048
CE Loss: 0.1849

```
In [16]: # Optimizers
# Your pure-python update: p.data -= lr * p.grad
# PyTorch optimizers do the same (and much more)

model_demo = MLP(input_dim=2, hidden_dims=[8], output_dim=1)

# SGD -- exactly what you implemented
sgd = optim.SGD(model_demo.parameters(), lr=0.01)

# Adam -- adaptive learning rates per parameter
adam = optim.Adam(model_demo.parameters(), lr=0.001)

print("SGD: Simple gradient descent (what you built in Notebook 1)")
print("Adam: Adaptive lr per param (widely used in practice)")
```

SGD: Simple gradient descent (what you built in Notebook 1)
Adam: Adaptive lr per param (widely used in practice)

5.2 Training on the Moons Dataset

Same dataset as Notebook 1! But now with PyTorch.

```
In [17]: # Generate moons data
torch.manual_seed(42)
np.random.seed(42)

X_np, y_np = make_moons(n_samples=300, noise=0.2, random_state=42)

# Convert to PyTorch tensors
X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.float32).unsqueeze(1) # (300, 1)

print(f"X shape: {X.shape}")
```

```

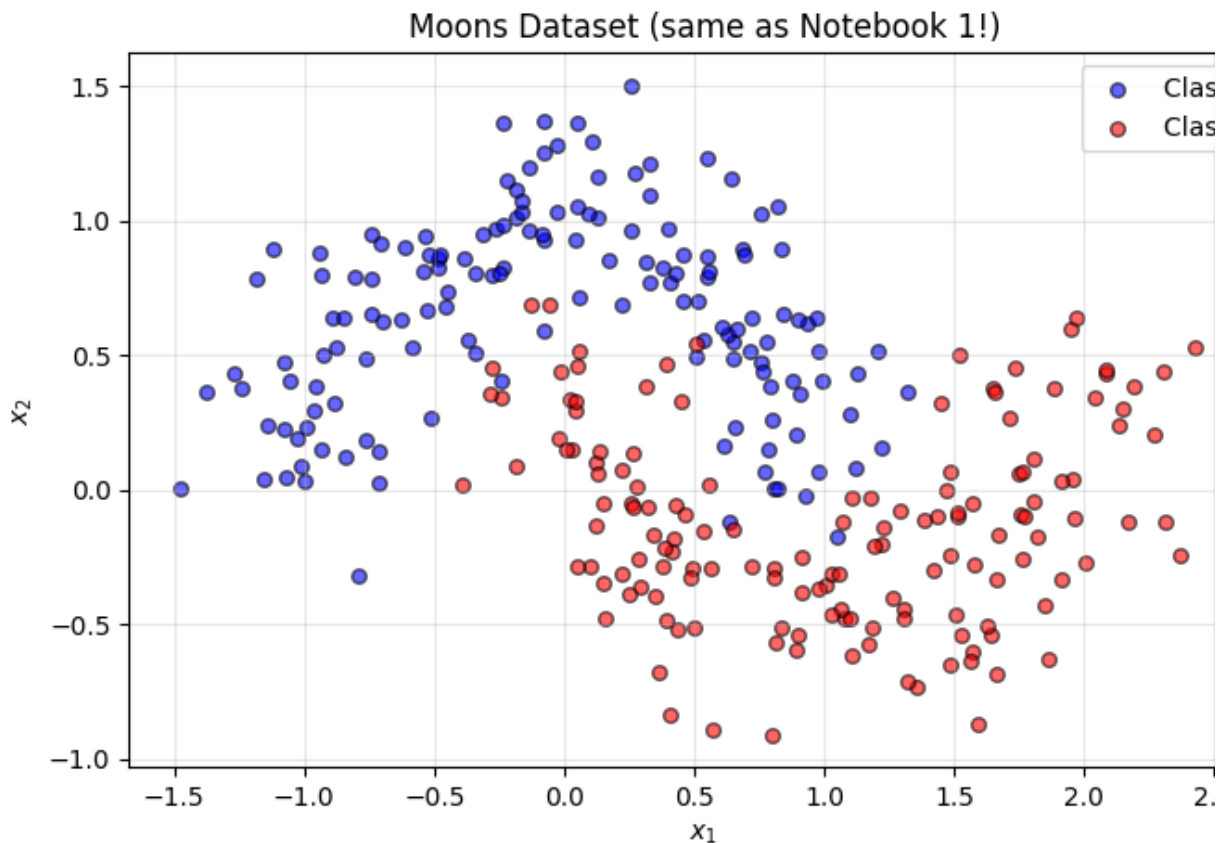
print(f"y shape: {y.shape}")

# Visualize
plt.figure(figsize=(8, 5))
plt.scatter(X_np[y_np == 0, 0], X_np[y_np == 0, 1], c='blue', label='Class 0')
plt.scatter(X_np[y_np == 1, 0], X_np[y_np == 1, 1], c='red', label='Class 1')
plt.title('Moons Dataset (same as Notebook 1!)')
plt.xlabel('$x_1$')
plt.ylabel('$x_2$')
plt.legend()
plt.show()

```

X shape: torch.Size([300, 2])

y shape: torch.Size([300, 1])



```

In [18]: # Train the MLP
torch.manual_seed(42)

model = MLP(input_dim=2, hidden_dims=[16, 16], output_dim=1)
optimizer = optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.BCEWithLogitsLoss()

losses = []
accuracies = []

for epoch in range(500):
    # -- Forward pass --
    logits = model(X)                    # raw scores
    loss = loss_fn(logits, y)            # compute loss

    # -- Backward pass --
    optimizer.zero_grad()                # zero gradients (like your p.grad)
    loss.backward()                       # compute gradients (like your loss.backward())

```



```

# -- Update parameters --
optimizer.step()                                # update weights (like your p.dat

# -- Track metrics --
with torch.no_grad():
    preds = (torch.sigmoid(logits) > 0.5).float()
    acc = (preds == y).float().mean().item()

    losses.append(loss.item())
    accuracies.append(acc)

    if epoch % 100 == 0:
        print(f"Epoch {epoch:3d} | Loss: {loss.item():.4f} | Accuracy: {a

print(f"\nFinal -- Loss: {losses[-1]:.4f} | Accuracy: {accuracies[-1]:.2%

```

```

Epoch   0 | Loss: 0.6946 | Accuracy: 50.00%
Epoch 100 | Loss: 0.1663 | Accuracy: 93.00%
Epoch 200 | Loss: 0.0618 | Accuracy: 97.67%
Epoch 300 | Loss: 0.0509 | Accuracy: 97.67%
Epoch 400 | Loss: 0.0489 | Accuracy: 97.67%

```

```

Final -- Loss: 0.0481 | Accuracy: 97.67%

```

```

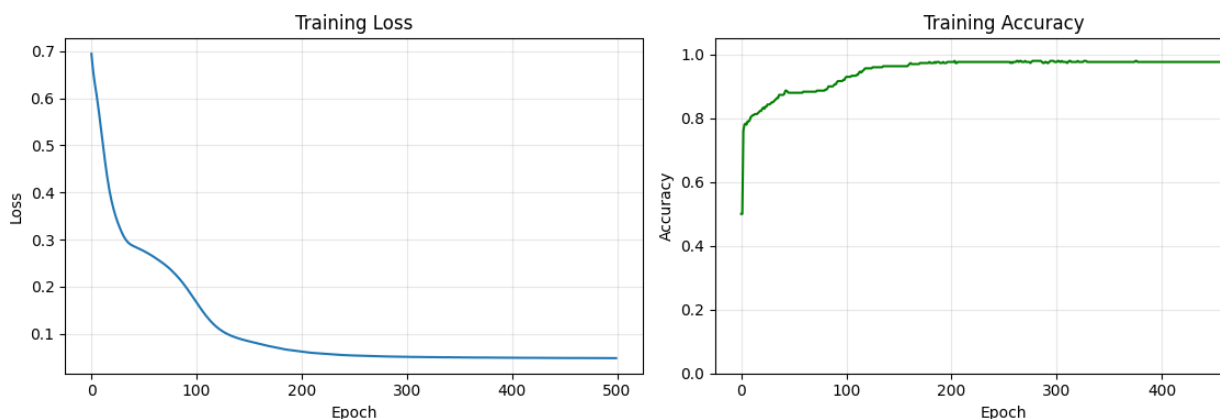
In [19]: # Plot training curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(losses, linewidth=1.5)
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.set_title('Training Loss')

ax2.plot(accuracies, linewidth=1.5, color='green')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy')
ax2.set_title('Training Accuracy')
ax2.set_ylim(0, 1.05)

plt.tight_layout()
plt.show()

```



5.3 Visualizing the Decision Boundary

Remember plotting this in Notebook 1? Same technique -- but now PyTorch makes the forward pass trivial.

```

In [20]: # Decision boundary visualization
def plot_decision_boundary(model, X, y, title="Decision Boundary"):
    """Plot the decision boundary of a binary classifier."""
    x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
    y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 200),
        np.linspace(y_min, y_max, 200)
    )
    grid = torch.tensor(np.c_[xx.ravel(), yy.ravel()], dtype=torch.float32)

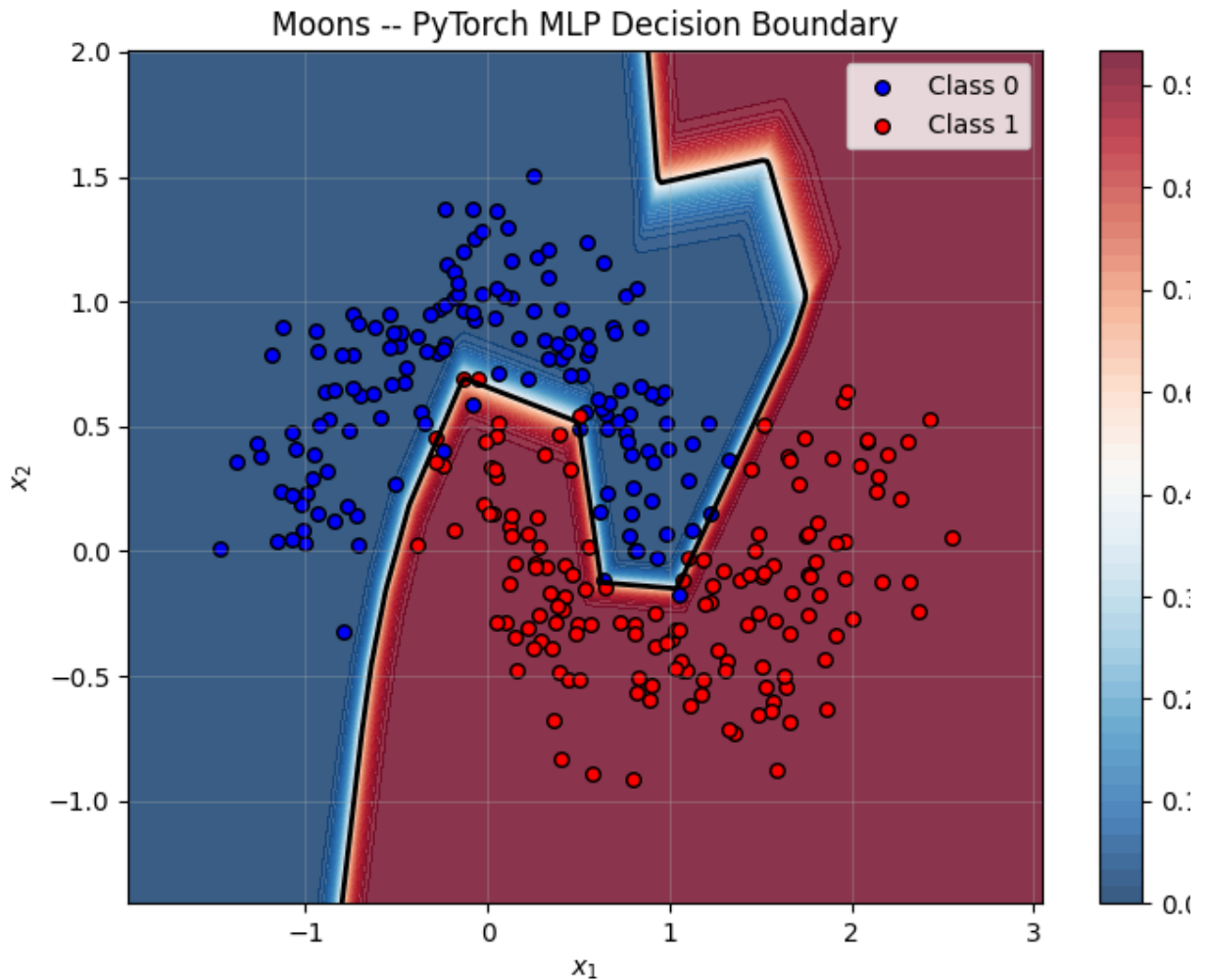
    with torch.no_grad():
        probs = torch.sigmoid(model(grid)).numpy().reshape(xx.shape)

    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, probs, levels=50, cmap='RdBu_r', alpha=0.8)
    plt.colorbar(label='P(class=1)')
    plt.contour(xx, yy, probs, levels=[0.5], colors='k', linewidths=2)

    y_np = y.numpy().ravel()
    X_np = X.numpy()
    plt.scatter(X_np[y_np == 0, 0], X_np[y_np == 0, 1], c='blue',
                edgecolors='k', s=30, label='Class 0')
    plt.scatter(X_np[y_np == 1, 0], X_np[y_np == 1, 1], c='red',
                edgecolors='k', s=30, label='Class 1')
    plt.title(title)
    plt.xlabel('$x_1$')
    plt.ylabel('$x_2$')
    plt.legend()
    plt.show()

plot_decision_boundary(model, X, y, "Moons -- PyTorch MLP Decision Bounda

```



5.4 Decision Boundary Evolution During Training

Let's retrain and capture snapshots to see how the boundary evolves -- just like the visualization Notebook 1.

```
In [21]: # Retrain with snapshots
torch.manual_seed(42)

model_snap = MLP(input_dim=2, hidden_dims=[16, 16], output_dim=1)
optimizer_snap = optim.Adam(model_snap.parameters(), lr=0.01)
loss_fn_snap = nn.BCEWithLogitsLoss()

# We'll save model state at key epochs
snapshot_epochs = [0, 10, 50, 100, 200, 499]
snapshots = {}

for epoch in range(500):
    if epoch in snapshot_epochs:
        # Save a copy of the model state
        snapshots[epoch] = {k: v.clone() for k, v in model_snap.state_dict().items()}

    logits = model_snap(X)
    loss = loss_fn_snap(logits, y)
    optimizer_snap.zero_grad()
    loss.backward()
    optimizer_snap.step()
```

```

# Plot snapshots
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.ravel()

x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 150), np.linspace(y_min, y_max, 150))
grid = torch.tensor(np.c_[xx.ravel(), yy.ravel()], dtype=torch.float32)

temp_model = MLP(input_dim=2, hidden_dims=[16, 16], output_dim=1)

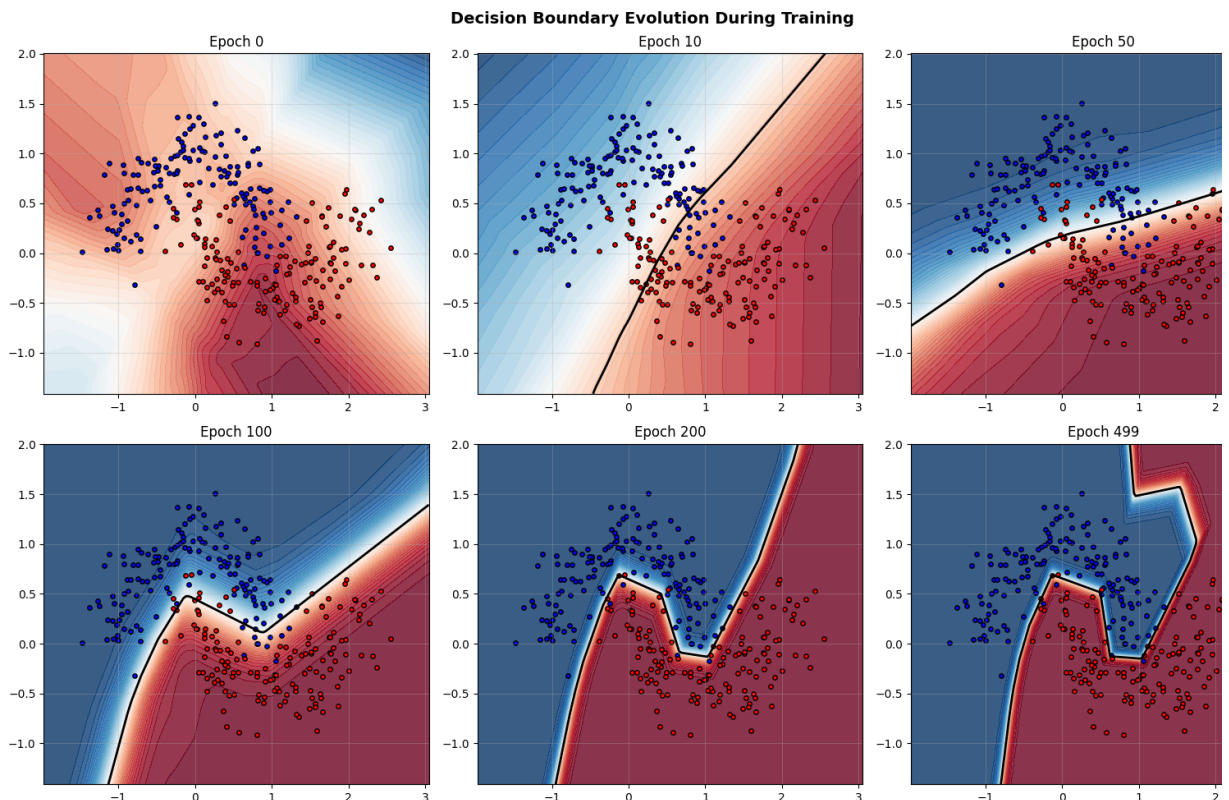
for idx, ep in enumerate(snapshot_epochs):
    temp_model.load_state_dict(snapshots[ep])
    with torch.no_grad():
        probs = torch.sigmoid(temp_model(grid)).numpy().reshape(xx.shape)

    ax = axes[idx]
    ax.contourf(xx, yy, probs, levels=50, cmap='RdBu_r', alpha=0.8)
    ax.contour(xx, yy, probs, levels=[0.5], colors='k', linewidths=2)

    y_np_flat = y.numpy().ravel()
    X_np_arr = X.numpy()
    ax.scatter(X_np_arr[y_np_flat == 0, 0], X_np_arr[y_np_flat == 0, 1],
               c='blue', edgecolors='k', s=15)
    ax.scatter(X_np_arr[y_np_flat == 1, 0], X_np_arr[y_np_flat == 1, 1],
               c='red', edgecolors='k', s=15)
    ax.set_title(f'Epoch {ep}')
    ax.set_xlim(x_min, x_max)
    ax.set_ylim(y_min, y_max)

plt.suptitle('Decision Boundary Evolution During Training', fontsize=14,
plt.tight_layout()
plt.show()

```



6. Universal Approximation in PyTorch

① Curriculum Point: Universal Approximation Theorem

In Notebook 1, you demonstrated that a single hidden layer with enough neurons can approximate any continuous function. And that a **deep narrow** network often does it more efficiently (fewer parameters).

Let's repeat that experiment in PyTorch -- much easier now!

6.1 The Target Function

We'll fit $f(x) = \sin(x) \cdot \cos(3x)$ -- a complex, multi-frequency function.

```
In [22]: # Target function
torch.manual_seed(42)

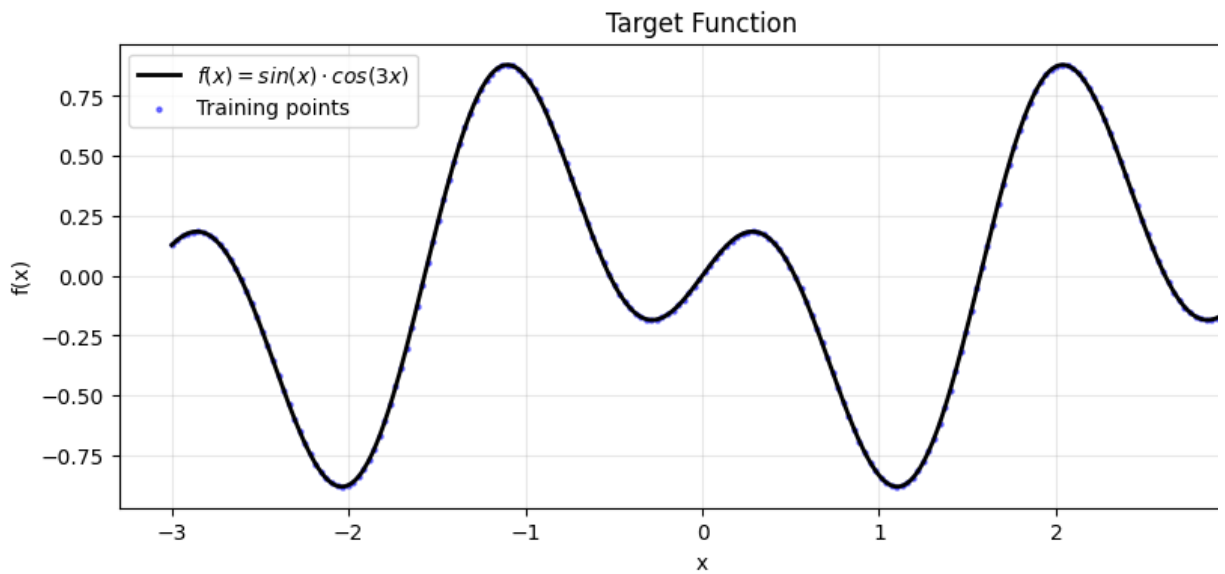
def target_fn(x):
    return torch.sin(x) * torch.cos(3 * x)

# Training data
x_train = torch.linspace(-3, 3, 200).unsqueeze(1)  # (200, 1)
y_train = target_fn(x_train)

# Dense evaluation grid
x_eval = torch.linspace(-3, 3, 500).unsqueeze(1)
y_eval = target_fn(x_eval)

plt.figure(figsize=(10, 4))
plt.plot(x_eval.numpy(), y_eval.numpy(), 'k-', linewidth=2, label='$f(x)$')
plt.scatter(x_train.numpy(), y_train.numpy(), c='blue', s=5, alpha=0.5, label='Training Data')
plt.title('Target Function')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.show()
```

```
<>:16: SyntaxWarning: invalid escape sequence '\c'
<>:16: SyntaxWarning: invalid escape sequence '\c'
/tmp/ipykernel_1532/912513290.py:16: SyntaxWarning: invalid escape sequence '\c'
  plt.plot(x_eval.numpy(), y_eval.numpy(), 'k-', linewidth=2, label='$f(x) = \sin(x) \cdot \cos(3x)$')
```



6.2 Shallow-Wide vs Deep-Narrow

We'll train two networks:

- **Shallow-wide:** 1 hidden layer with 100 neurons
- **Deep-narrow:** 4 hidden layers with 10 neurons each

Both use ReLU. We'll compare how well they fit and how many parameters they use.

```
In [23]: # Define both architectures
torch.manual_seed(42)

# Shallow-wide: 1 hidden layer, 100 neurons
shallow = nn.Sequential(
    nn.Linear(1, 100),
    nn.ReLU(),
    nn.Linear(100, 1)
)

# Deep-narrow: 4 hidden layers, 10 neurons each
deep = nn.Sequential(
    nn.Linear(1, 10),
    nn.ReLU(),
    nn.Linear(10, 10),
    nn.ReLU(),
    nn.Linear(10, 10),
    nn.ReLU(),
    nn.Linear(10, 10),
    nn.ReLU(),
    nn.Linear(10, 1)
)

shallow_params = sum(p.numel() for p in shallow.parameters())
deep_params = sum(p.numel() for p in deep.parameters())

print(f"Shallow-wide (1x100): {shallow_params} parameters")
print(f"Deep-narrow (4x10): {deep_params} parameters")
print(f"\nDeep network uses {deep_params/shallow_params:.1%} of the shall
```

Shallow-wide (1x100): 301 parameters
Deep-narrow (4x10): 361 parameters

Deep network uses 119.9% of the shallow network's parameters

```
In [24]: # Train both networks
def train_model(model, x, y, epochs=2000, lr=0.01):
    optimizer = optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.MSELoss()
    losses = []

    for epoch in range(epochs):
        pred = model(x)
        loss = loss_fn(pred, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        losses.append(loss.item())

    return losses

torch.manual_seed(42)
shallow_model = nn.Sequential(
    nn.Linear(1, 100), nn.ReLU(), nn.Linear(100, 1)
)
deep_model = nn.Sequential(
    nn.Linear(1, 10), nn.ReLU(),
    nn.Linear(10, 10), nn.ReLU(),
    nn.Linear(10, 10), nn.ReLU(),
    nn.Linear(10, 10), nn.ReLU(),
    nn.Linear(10, 1)
)

print("Training shallow-wide network...")
shallow_losses = train_model(shallow_model, x_train, y_train, epochs=3000)
print(f" Final loss: {shallow_losses[-1]:.6f}")

print("Training deep-narrow network...")
deep_losses = train_model(deep_model, x_train, y_train, epochs=3000, lr=0.01)
print(f" Final loss: {deep_losses[-1]:.6f}")
```

Training shallow-wide network...

Final loss: 0.003145

Training deep-narrow network...

Final loss: 0.000093

```
In [25]: # Compare approximations
with torch.no_grad():
    y_shallow = shallow_model(x_eval).numpy()
    y_deep = deep_model(x_eval).numpy()

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Shallow-wide fit
axes[0].plot(x_eval.numpy(), y_eval.numpy(), 'k-', linewidth=2, label='Target')
axes[0].plot(x_eval.numpy(), y_shallow, 'b--', linewidth=2, label='Shallow-wide fit')
shallow_nparams = sum(p.numel() for p in shallow_model.parameters())
axes[0].set_title(f'Shallow-Wide -- {shallow_nparams} params')
axes[0].legend()
axes[0].set_xlabel('x')
```

```

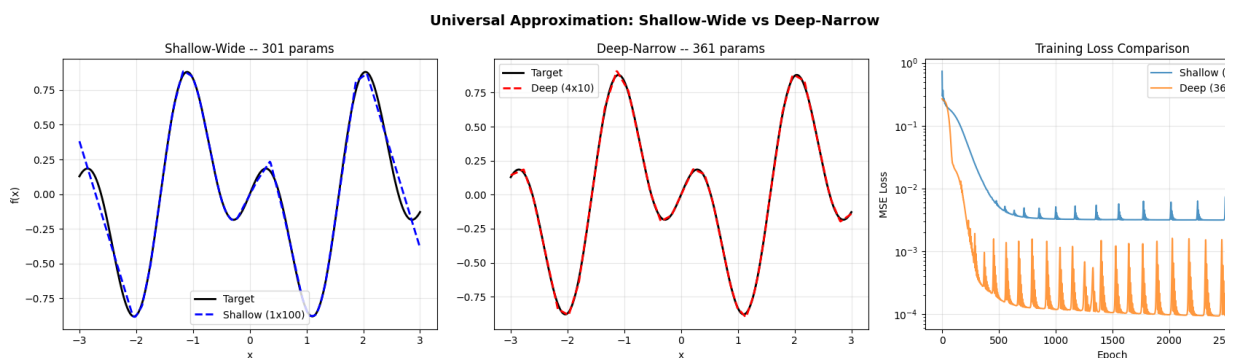
axes[0].set_ylabel('f(x)')

# Deep-narrow fit
axes[1].plot(x_eval.numpy(), y_eval.numpy(), 'k-', linewidth=2, label='Ta
axes[1].plot(x_eval.numpy(), y_deep, 'r--', linewidth=2, label='Deep (4x1
deep_nparams = sum(p.numel() for p in deep_model.parameters())
axes[1].set_title(f'Deep-Narrow -- {deep_nparams} params')
axes[1].legend()
axes[1].set_xlabel('x')

# Loss curves
axes[2].plot(shallow_losses, label=f'Shallow ({shallow_nparams} params)',
axes[2].plot(deep_losses, label=f'Deep ({deep_nparams} params)', alpha=0.
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('MSE Loss')
axes[2].set_title('Training Loss Comparison')
axes[2].set_yscale('log')
axes[2].legend()

plt.suptitle('Universal Approximation: Shallow-Wide vs Deep-Narrow',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```



① Key Insight: Universal Approximation

Both networks can approximate the target function -- the Universal Approximation Theorem guarantees this for a single hidden layer with enough neurons.

But notice: the **deep-narrow** network achieves comparable quality with **far fewer parameters**. Depth lets the network compose simple features into complex ones hierarchically. This is why modern networks are deep, not just wide.

7. Custom Autograd Function

Sometimes you need to define a custom operation with your own forward and backward. Remember implementing `_backward` closures in your `Value` class? PyTorch's `torch.autograd.Function` is the same idea -- you define forward and backward.

We'll implement **SiLU** (Sigmoid Linear Unit): $\text{SiLU}(x) = x \cdot \sigma(x)$


```
In [26]: # Custom autograd Function: SiLU
# This is exactly like writing a _backward closure in your Value class,
# but using PyTorch's formal API.

class SiLUFunction(torch.autograd.Function):
    """
    SiLU(x) = x * sigmoid(x)

    Forward: y = x * sig(x)
    Backward: dy/dx = sig(x) + x * sig(x) * (1 - sig(x))
               = sig(x) * (1 + x * (1 - sig(x)))
    """

    @staticmethod
    def forward(ctx, x):
        # ctx is like your Value._prev -- it saves info for backward
        sigmoid_x = torch.sigmoid(x)
        ctx.save_for_backward(x, sigmoid_x)
        return x * sigmoid_x

    @staticmethod
    def backward(ctx, grad_output):
        # grad_output is the incoming gradient (like your out.grad in Val
        x, sigmoid_x = ctx.saved_tensors
        # d(SiLU)/dx = sig(x) + x*sig(x)*(1-sig(x)) = sig(x)*(1 + x*(1-si
        grad_input = grad_output * (sigmoid_x * (1 + x * (1 - sigmoid_x)))
        return grad_input

# Convenience wrapper
def custom_silu(x):
    return SiLUFunction.apply(x)

print("Custom SiLU function defined!")
print("  Forward: SiLU(x) = x * sigmoid(x)")
print("  Backward: SiLU'(x) = sigmoid(x) * (1 + x * (1 - sigmoid(x)))")
```

Custom SiLU function defined!

Forward: SiLU(x) = x * sigmoid(x)

Backward: SiLU'(x) = sigmoid(x) * (1 + x * (1 - sigmoid(x)))

```
In [27]: # Verify against PyTorch's built-in SiLU
x = torch.linspace(-5, 5, 200, requires_grad=False)

# Our custom SiLU
x_custom = x.clone().requires_grad_(True)
y_custom = custom_silu(x_custom)
y_custom.sum().backward()
grad_custom = x_custom.grad.clone()

# PyTorch built-in SiLU
x_builtin = x.clone().requires_grad_(True)
y_builtin = torch.nn.functional.silu(x_builtin)
y_builtin.sum().backward()
grad_builtin = x_builtin.grad.clone()

# Compare
print(f"Max forward difference: {(y_custom - y_builtin).abs().max().item}")
print(f"Max gradient difference: {(grad_custom - grad_builtin).abs().max().item}")
print("Custom implementation matches PyTorch!")
```

```
# Plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

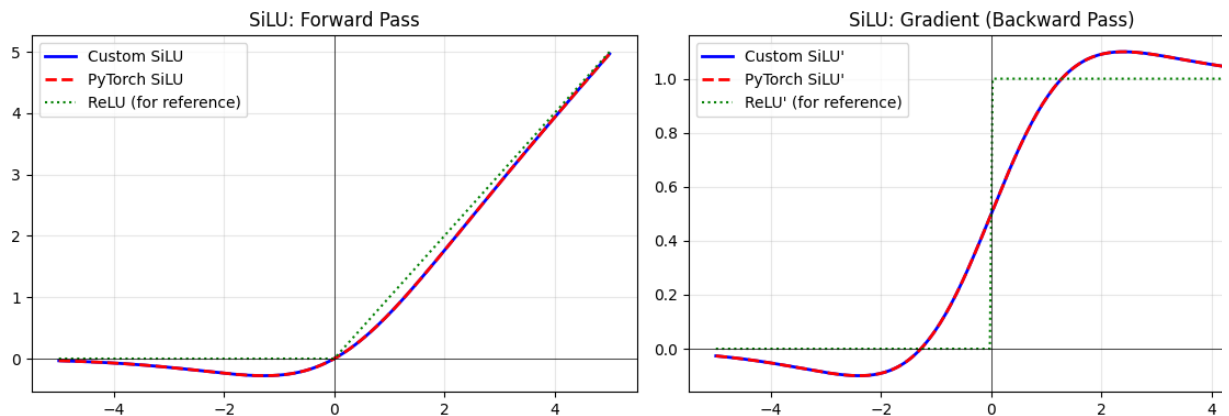
ax1.plot(x.numpy(), y_custom.detach().numpy(), 'b-', linewidth=2, label='Custom SiLU')
ax1.plot(x.numpy(), y_builtin.detach().numpy(), 'r--', linewidth=2, label='PyTorch SiLU')
ax1.plot(x.numpy(), torch.relu(x).numpy(), 'g:', linewidth=1.5, label='ReLU')
ax1.set_title('SiLU: Forward Pass')
ax1.legend()
ax1.axhline(y=0, color='k', linewidth=0.5)
ax1.axvline(x=0, color='k', linewidth=0.5)

ax2.plot(x.numpy(), grad_custom.numpy(), 'b-', linewidth=2, label='Custom SiLU')
ax2.plot(x.numpy(), grad_builtin.numpy(), 'r--', linewidth=2, label='PyTorch SiLU')
ax2.plot(x.numpy(), torch.relu_grad(x).numpy(), 'g:', linewidth=1.5, label='ReLU')
ax2.set_title('SiLU: Gradient (Backward Pass)')
ax2.legend()
ax2.axhline(y=0, color='k', linewidth=0.5)
ax2.axvline(x=0, color='k', linewidth=0.5)

plt.tight_layout()
plt.show()

print("\nSiLU is smooth (unlike ReLU) and allows small negative values.")
print("It's used in many modern architectures (e.g., SiLU = Swish in EfficientNet).")
```

Max forward difference: 4.77e-07
 Max gradient difference: 1.19e-07
 Custom implementation matches PyTorch!



SiLU is smooth (unlike ReLU) and allows small negative values.
 It's used in many modern architectures (e.g., SiLU = Swish in EfficientNet).

Connection to Notebook 1: When you implemented `Value.__mul__`, you wrote:

```
def _backward():
    self.grad += other.data * out.grad
    other.grad += self.data * out.grad
```

That's **exactly** what `SiLUFunction.backward()` does -- compute the local derivative and multiply by the incoming gradient (`grad_output`). The chain rule in action, whether you implement it yourself or use PyTorch's API.

8. Exercises

Exercise 1: Gradient Computation

Compute the gradient of $f(x, y, z) = (x + 2y) * z^{**2}$ at $(x, y, z) = (1, 2, 3)$

1. Compute analytically (by hand)
2. Verify with PyTorch autograd
3. Verify with finite differences: $(f(x+h) - f(x-h)) / 2h$

```
In [28]: # Exercise 1: Gradient Computation
# Analytical:
# f = (x + 2y) * z^2
# df/dx = z^2
# df/dy = 2*z^2
# df/dz = 2*z*(x + 2y)
#
# At (1, 2, 3):
# df/dx = 9
# df/dy = 18
# df/dz = 6 * 5 = 30

# --- SOLUTION ---
ex1_x = torch.tensor(1.0, requires_grad=True)
ex1_y = torch.tensor(2.0, requires_grad=True)
ex1_z = torch.tensor(3.0, requires_grad=True)

ex1_f = (ex1_x + 2*ex1_y) * ex1_z**2
ex1_f.backward()

print("Analytical vs PyTorch autograd:")
match_x = "MATCH" if abs(ex1_x.grad.item() - 9.0) < 1e-4 else "MISMATCH"
match_y = "MATCH" if abs(ex1_y.grad.item() - 18.0) < 1e-4 else "MISMATCH"
match_z = "MATCH" if abs(ex1_z.grad.item() - 30.0) < 1e-4 else "MISMATCH"
print(f" df/dx = z^2 = 9.0 | PyTorch: {ex1_x.grad.item():.1f}")
print(f" df/dy = 2z^2 = 18.0 | PyTorch: {ex1_y.grad.item():.1f}")
print(f" df/dz = 2z(x+2y) = 30.0 | PyTorch: {ex1_z.grad.item():.1f}")

# Finite differences verification
h = 1e-5
def f_val(xv, yv, zv):
    return (xv + 2*yv) * zv**2

x0, y0, z0 = 1.0, 2.0, 3.0
df_dx_fd = (f_val(x0+h, y0, z0) - f_val(x0-h, y0, z0)) / (2*h)
df_dy_fd = (f_val(x0, y0+h, z0) - f_val(x0, y0-h, z0)) / (2*h)
df_dz_fd = (f_val(x0, y0, z0+h) - f_val(x0, y0, z0-h)) / (2*h)

print(f"\nFinite differences (h={h}):")
print(f" df/dx = {df_dx_fd:.6f}")
print(f" df/dy = {df_dy_fd:.6f}")
print(f" df/dz = {df_dz_fd:.6f}")
```

Analytical vs PyTorch autograd:

```
df/dx = z^2          = 9.0   | PyTorch: 9.0   [MATCH]
df/dy = 2z^2         = 18.0  | PyTorch: 18.0  [MATCH]
df/dz = 2z(x+2y)     = 30.0  | PyTorch: 30.0  [MATCH]
```

Finite differences (h=1e-05):

```
df/dx = 9.000000
df/dy = 18.000000
df/dz = 30.000000
```

Exercise 2: ReLU vs Sigmoid Deep Network

Build two identical 8-layer MLPs for moons classification -- one with ReLU, one with Sigmoid. Train for 500 epochs. Compare:

- Final accuracy
- Gradient magnitudes in the first layer
- Training speed

This demonstrates ② **ReLU changed everything.**

```
In [29]: # Exercise 2: ReLU vs Sigmoid in deep networks

# --- SOLUTION ---
# Re-create moons data (in case variables were overwritten by previous ex
X_np_ex2, y_np_ex2 = make_moons(n_samples=300, noise=0.2, random_state=42)
X_ex2 = torch.tensor(X_np_ex2, dtype=torch.float32)
y_ex2 = torch.tensor(y_np_ex2, dtype=torch.float32).unsqueeze(1)

torch.manual_seed(42)

def make_deep_net(activation):
    """Build an 8-layer MLP with the given activation."""
    layers = []
    dims = [2, 16, 16, 16, 16, 16, 16, 16, 1]
    for i in range(len(dims) - 1):
        layers.append(nn.Linear(dims[i], dims[i+1]))
        if i < len(dims) - 2: # no activation after last layer
            layers.append(activation())
    return nn.Sequential(*layers)

relu_net = make_deep_net(nn.ReLU)
sigmoid_net = make_deep_net(nn.Sigmoid)

relu_opt = optim.Adam(relu_net.parameters(), lr=0.01)
sigmoid_opt = optim.Adam(sigmoid_net.parameters(), lr=0.01)
loss_fn = nn.BCEWithLogitsLoss()

relu_losses, sigmoid_losses = [], []
relu_accs, sigmoid_accs = [], []
relu_grads_first, sigmoid_grads_first = [], []

for epoch in range(500):
    for net, opt, losses_list, accs_list, grads_list in [
        (relu_net, relu_opt, relu_losses, relu_accs, relu_grads_first),
        (sigmoid_net, sigmoid_opt, sigmoid_losses, sigmoid_accs, sigmoid_grads_first)
    ]:
```

```

logits = net(X_ex2)
loss = loss_fn(logits, y_ex2)
opt.zero_grad()
loss.backward()
opt.step()

losses_list.append(loss.item())
with torch.no_grad():
    acc = ((torch.sigmoid(logits) > 0.5).float() == y_ex2).float()
    accs_list.append(acc)

# Record gradient magnitude of first layer
first_weight_grad = list(net.parameters())[0].grad
if first_weight_grad is not None:
    grads_list.append(first_weight_grad.abs().mean().item())

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].plot(relu_losses, label='ReLU', alpha=0.8)
axes[0].plot(sigmoid_losses, label='Sigmoid', alpha=0.8)
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Training Loss')
axes[0].legend()

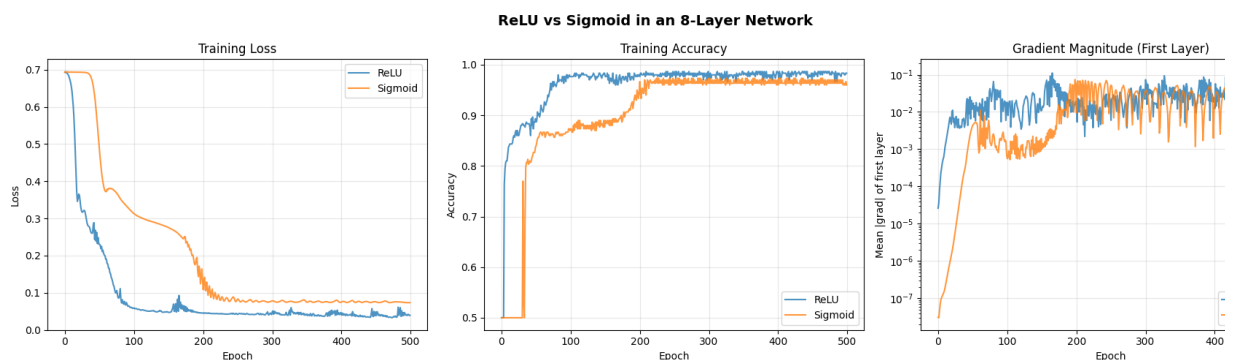
axes[1].plot(relu_accs, label='ReLU', alpha=0.8)
axes[1].plot(sigmoid_accs, label='Sigmoid', alpha=0.8)
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].set_title('Training Accuracy')
axes[1].legend()

axes[2].plot(relu_grads_first, label='ReLU', alpha=0.8)
axes[2].plot(sigmoid_grads_first, label='Sigmoid', alpha=0.8)
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('Mean |grad| of first layer')
axes[2].set_title('Gradient Magnitude (First Layer)')
axes[2].set_yscale('log')
axes[2].legend()

plt.suptitle('ReLU vs Sigmoid in an 8-Layer Network', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print(f"Final accuracy -- ReLU: {relu_accs[-1]:.2%}, Sigmoid: {sigmoid_accs[-1]:.2%}")
print(f"ReLU trains faster and achieves better accuracy in deep networks!")

```



Final accuracy -- ReLU: 98.33%, Sigmoid: 96.00%

ReLU trains faster and achieves better accuracy in deep networks!

Exercise 3: Custom Autograd -- Implement "Leaky ReLU"

Implement Leaky ReLU as a custom `torch.autograd.Function`:

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} \end{cases}$$

where $\alpha = 0.01$.

Then:

1. Compare output with `torch.nn.functional.leaky_relu`
2. Plot the function and its gradient

```
In [30]: # Exercise 3: Custom Leaky ReLU

# --- SOLUTION ---
class LeakyReLUFunction(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, alpha=0.01):
        ctx.save_for_backward(x)
        ctx.alpha = alpha
        return torch.where(x > 0, x, alpha * x)

    @staticmethod
    def backward(ctx, grad_output):
        x, = ctx.saved_tensors
        alpha = ctx.alpha
        grad_input = grad_output * torch.where(
            x > 0, torch.ones_like(x), alpha * torch.ones_like(x)
        )
        return grad_input, None # None for alpha (not a tensor)

def custom_leaky_relu(x, alpha=0.01):
    return LeakyReLUFunction.apply(x, alpha)

# Verify
x = torch.linspace(-3, 3, 200)
x_custom = x.clone().requires_grad_(True)
y_custom = custom_leaky_relu(x_custom)
y_custom.sum().backward()

x_builtin = x.clone().requires_grad_(True)
y_builtin = torch.nn.functional.leaky_relu(x_builtin, 0.01)
y_builtin.sum().backward()

print(f"Max forward difference: {(y_custom - y_builtin).abs().max().item}")
print(f"Max gradient difference: {(x_custom.grad - x_builtin.grad).abs().max().item}")
print("Custom Leaky ReLU matches PyTorch!")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(x.numpy(), y_custom.detach().numpy(), 'b-', linewidth=2, label='Custom')
ax1.plot(x.numpy(), y_builtin.detach().numpy(), 'r--', linewidth=2, label='Builtin')
ax1.set_title('Leaky ReLU: Forward')
ax1.legend()
ax1.axhline(y=0, color='k', linewidth=0.5)
ax1.axvline(x=0, color='k', linewidth=0.5)
```

```

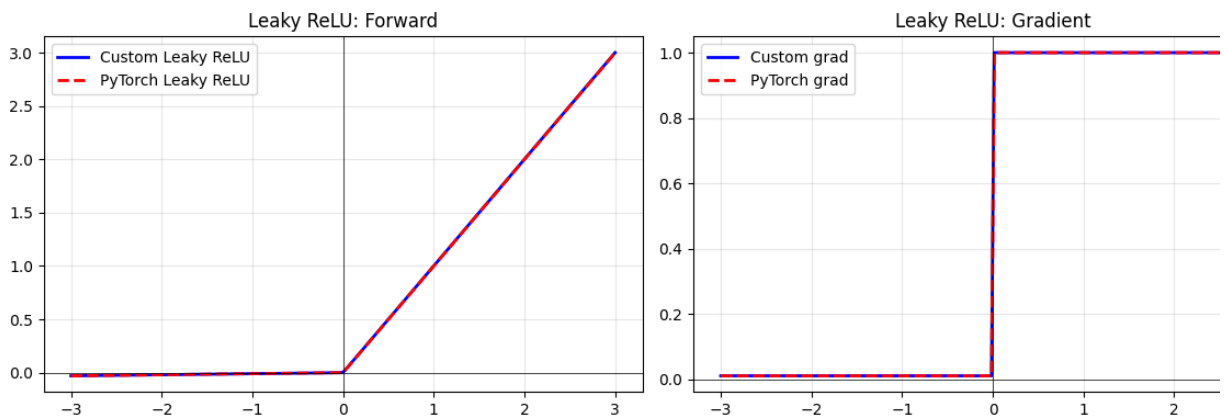
ax2.plot(x.numpy(), x_custom.grad.numpy(), 'b-', linewidth=2, label="Cust")
ax2.plot(x.numpy(), x_builtin.grad.numpy(), 'r--', linewidth=2, label="Py")
ax2.set_title('Leaky ReLU: Gradient')
ax2.legend()
ax2.axhline(y=0, color='k', linewidth=0.5)
ax2.axvline(x=0, color='k', linewidth=0.5)

plt.tight_layout()
plt.show()

print("\nLeaky ReLU fixes the 'dying ReLU' problem:")
print("neurons with x<0 still get a small gradient (alpha=0.01).")

```

Max forward difference: 0.00e+00
 Max gradient difference: 0.00e+00
 Custom Leaky ReLU matches PyTorch!



Leaky ReLU fixes the 'dying ReLU' problem:
 neurons with $x < 0$ still get a small gradient ($\alpha=0.01$).

Summary

Concept	Notebook 1 (Pure Python)	This Notebook (PyTorch)
Data structure	<code>Value</code> (scalar)	<code>torch.Tensor</code> (any sh
Autograd	Manual <code>_backward</code> closures	Automatic via <code>requires</code>
Gradient computation	<code>loss.backward()</code> + topo sort	<code>loss.backward()</code> (sa
Network building	<code>Neuron</code> , <code>Layer</code> , <code>MLP</code> classes	<code>nn.Module</code> , <code>nn.Line</code>
Training loop	Manual: zero -> forward -> backward -> update	Same pattern with optimiz
Activation	<code>relu()</code> method on <code>Value</code>	<code>nn.ReLU()</code> module

Curriculum Points Covered

① Universal Approximation Theorem (§6)

A single hidden layer with enough neurons can approximate any continuous function.
 Depth makes this more parameter-efficient -- deep networks compose features hierarchically.

② ReLU Changed Everything (§4, §8 Exercise 2)

Sigmoid/tanh gradients vanish in deep networks (max derivative 0.25 for sigmoid). ReLU gradient is exactly 0 or 1 -- no vanishing gradient problem. This is why virtually all modern architectures use ReLU or its variants (Leaky ReLU, SiLU, GELU).

⑤ Backpropagation = Chain Rule (§3, §7)

Whether you implement it yourself (Notebook 1) or let PyTorch do it, backprop is the same algorithm: walk the computation graph in reverse, multiply local derivatives by incoming gradients (the chain rule). PyTorch just does it automatically, for tensors of any shape, on any device.

What's Next

In the next notebook, we'll tackle **CNNs, training recipes, and overparameterization** -- building everything you've learned here to train real image classifiers.

Notebook 5: CNNs, Training Recipes & Overparameterization

Series 2, Week 2 — PyTorch

What We'll Build

In the previous notebooks you built MLPs that treated images as flat vectors — every pixel connected to every neuron, with no notion of spatial structure. That works for small images, but it's wildly inefficient. A 32×32 colour image has 3,072 pixels, and a single fully-connected layer with 512 neurons already has **1.6 million** weights.

In this notebook we introduce **Convolutional Neural Networks (CNNs)**, which exploit the spatial structure of images by sharing small filters across the image. Then we run a systematic **ablation study** to learn that the *training recipe* — initialization, normalization, augmentation, scheduling — matters as much as the architecture itself.

We'll also witness one of the most surprising facts in modern deep learning: a model with **millions of parameters** trained on only **1,000 images** can still generalize. Classical statistics says it shouldn't, but we'll see that it does.

Curriculum points covered

#	Point	Section
③	Overparameterization helps	§6
⑥	Geometry matters	§4–5
⑦	Capacity ≠ performance	§4

Setup

Let's import everything we need and set global seeds for reproducibility.

```
In [1]: import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader, Subset

import torchvision
import torchvision.transforms as transforms

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from copy import deepcopy
```

```

import time
import warnings
warnings.filterwarnings("ignore", category=UserWarning)

# -- Reproducibility -----
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

# Use CPU -- this notebook is designed to run on a laptop
DEVICE = torch.device("cpu")

print(f"PyTorch version : {torch.__version__}")
print(f"Torchvision      : {torchvision.__version__}")
print(f"Device           : {DEVICE}")

```

```

PyTorch version : 2.10.0+cpu
Torchvision      : 0.25.0+cpu
Device           : cpu

```

2. Convolution from Scratch

What a convolution does

A 2D convolution slides a small **filter** (or **kernel**) across an image and at each position computes the product between the filter weights and the local patch of pixels. This produces a **feature map** that highlights where the pattern encoded in the filter appears in the image.

Key properties:

- **Parameter sharing** — the same filter weights are reused everywhere in the image
- **Translation equivariance** — if the input shifts, the feature map shifts the same way
- **Locality** — each output pixel depends only on a small neighbourhood of input pixels

Let's implement a single 2D convolution in pure NumPy first, so you understand exactly what `nn.Conv2d` does under the hood.

```

In [2]: def conv2d_numpy(image, kernel, stride=1, padding=0):
        """Apply a single 2D convolution filter to a single-channel image.

        Parameters
        -----
        image : np.ndarray, shape (H, W)
        kernel : np.ndarray, shape (kH, kW)
        stride : int
        padding: int (zero-padding)

        Returns
        -----
        np.ndarray -- the feature map
        """
        # Pad the image
        if padding > 0:
            image = np.pad(image, padding, mode='constant', constant_values=0)

```

```

H, W = image.shape
kH, kW = kernel.shape

# Output dimensions
out_h = (H - kH) // stride + 1
out_w = (W - kW) // stride + 1

output = np.zeros((out_h, out_w))

for i in range(out_h):
    for j in range(out_w):
        # Extract the local patch
        patch = image[i*stride : i*stride + kH,
                      j*stride : j*stride + kW]
        # Dot product
        output[i, j] = np.sum(patch * kernel)

return output

# -- Demo: edge-detection kernels on a simple image -----
np.random.seed(SEED)

# Create a simple 8x8 image with a vertical edge
img = np.zeros((8, 8))
img[:, 4:] = 1.0 # right half is white

# Vertical edge detector and horizontal edge detector
kernel_v = np.array([[-1, 0, 1],
                     [-1, 0, 1],
                     [-1, 0, 1]], dtype=float)

kernel_h = np.array([[-1, -1, -1],
                     [ 0,  0,  0],
                     [ 1,  1,  1]], dtype=float)

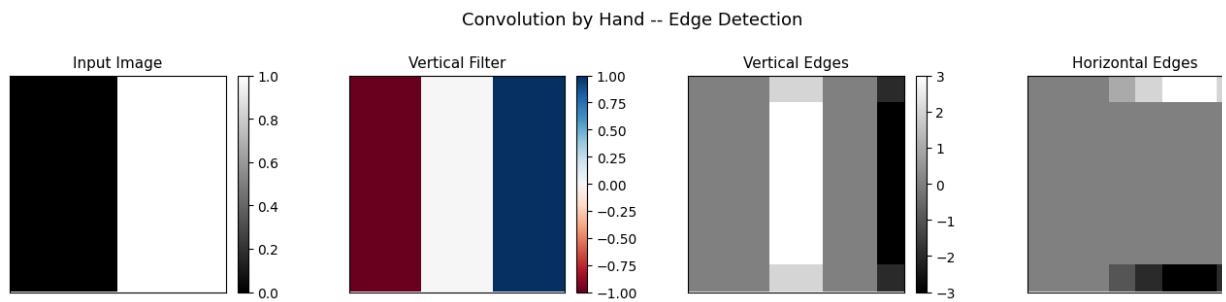
feat_v = conv2d_numpy(img, kernel_v, padding=1)
feat_h = conv2d_numpy(img, kernel_h, padding=1)

fig, axes = plt.subplots(1, 4, figsize=(14, 3))
titles = ["Input Image", "Vertical Filter", "Vertical Edges", "Horizontal Edges"]
data = [img, kernel_v, feat_v, feat_h]
cmaps = ["gray", "RdBu", "gray", "gray"]

for ax, title, d, cm in zip(axes, titles, data, cmaps):
    im = ax.imshow(d, cmap=cm)
    ax.set_title(title, fontsize=11)
    ax.set_xticks([]); ax.set_yticks([])
    plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

plt.suptitle("Convolution by Hand -- Edge Detection", fontsize=13, y=1.02)
plt.tight_layout()
plt.show()
print("Our hand-written convolution detected the vertical edge perfectly.")

```



Our hand-written convolution detected the vertical edge perfectly.

From NumPy to PyTorch: `nn.Conv2d`

PyTorch's `nn.Conv2d` does exactly the same sliding-window dot product, but:

- Handles **batches** of multi-channel images
- Runs on GPU if available
- Tracks gradients automatically for backprop
- Is implemented in highly-optimized C++/CUDA

Let's verify that our NumPy result matches PyTorch's.

```
In [3]: # Build the same vertical-edge kernel in PyTorch
conv_pt = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, padding=1)

# Manually set the weights to our kernel
with torch.no_grad():
    conv_pt.weight[0, 0] = torch.tensor(kernel_v, dtype=torch.float32)

# Prepare input: PyTorch expects (batch, channels, H, W)
img_tensor = torch.tensor(img, dtype=torch.float32).unsqueeze(0).unsqueeze(1)

feat_pt = conv_pt(img_tensor).squeeze().detach().numpy()

# Compare
diff = np.abs(feat_v - feat_pt).max()
print(f"Max absolute difference between NumPy and PyTorch conv: {diff:.2e}")
print("Remember when you built matrix multiply from scratch in the pure-Python notebook")
print("Same idea -- understand it manually, then use the optimized version")
```

Max absolute difference between NumPy and PyTorch conv: 0.00e+00
Remember when you built matrix multiply from scratch in the pure-Python notebook
Same idea -- understand it manually, then use the optimized version.

3. Building a CNN for CIFAR-10

[CIFAR-10](#) contains 60,000 tiny 32x32 colour images in 10 classes: airplane, automobile, bird, cat, dog, frog, horse, ship, truck.

We'll build progressively better models and measure each improvement.

3.1 Loading the data

We use `torchvision.datasets` to download and load CIFAR-10. For now, the baseline train just converting to tensor and normalising to [0, 1].

```
In [4]: # -- Basic transforms (no augmentation yet) -----
transform_basic = transforms.Compose([
    transforms.ToTensor(), # [0,255] uint8 -> [0,1] float, (C,H,W)
])

# Download CIFAR-10 (this may take a minute the first time)
trainset_full = torchvision.datasets.CIFAR10(
    root='./data', train=True, download=True, transform=transform_basic)
testset = torchvision.datasets.CIFAR10(
    root='./data', train=False, download=True, transform=transform_basic)

# For speed, use a subset: 5000 train, 1000 test
torch.manual_seed(SEED)
train_indices = torch.randperm(len(trainset_full))[:2000].tolist()
test_indices = torch.randperm(len(testset))[:500].tolist()

trainset = Subset(trainset_full, train_indices)
testset_sub = Subset(testset, test_indices)

print(f"Training samples : {len(trainset)}")
print(f"Test samples      : {len(testset_sub)}")

classes = ('airplane', 'automobile', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')

# -- Peek at some images -----
loader_peek = DataLoader(trainset, batch_size=16, shuffle=True,
                        generator=torch.Generator().manual_seed(SEED))
images, labels = next(iter(loader_peek))

fig, axes = plt.subplots(2, 8, figsize=(14, 4))
for i, ax in enumerate(axes.flat):
    img_np = images[i].permute(1, 2, 0).numpy()
    ax.imshow(img_np)
    ax.set_title(classes[labels[i]], fontsize=9)
    ax.axis('off')
plt.suptitle("CIFAR-10 Samples (32x32 colour)", fontsize=13)
plt.tight_layout()
plt.show()
```

Training samples : 2000

Test samples : 500



3.2 The baseline CNN

Our baseline architecture:

```
Conv(3->16, 3x3) -> ReLU -> MaxPool(2x2)
Conv(16->32, 3x3) -> ReLU -> MaxPool(2x2)
Flatten -> Linear(32*8*8 -> 128) -> ReLU -> Linear(128 -> 10)
```

This is a simple, no-frills CNN. No batch normalisation, no data augmentation, no fancy initialisation — add those one at a time and measure each effect.

```
In [5]: class BaselineCNN(nn.Module):
        """A bare-bones CNN: conv -> relu -> pool -> conv -> relu -> pool ->
        linear -> linear -> linear"""

        def __init__(self):
            super().__init__()
            # Two convolutional blocks
            self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1) # 32x32
            self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1) # 16x16
            self.pool = nn.MaxPool2d(2, 2) # halve

            # Fully connected layers -- after 2 pools: 32x32 -> 16x16 -> 8x8
            # So 32 channels x 8 x 8 = 2048
            self.fc1 = nn.Linear(32 * 8 * 8, 128)
            self.fc2 = nn.Linear(128, 10)

        def forward(self, x):
            x = self.pool(F.relu(self.conv1(x))) # (B,3,32,32) -> (B,16,16,16)
            x = self.pool(F.relu(self.conv2(x))) # (B,16,16,16) -> (B,32,8,8)
            x = x.view(x.size(0), -1) # flatten
            x = F.relu(self.fc1(x))
            x = self.fc2(x)
            return x

        # Quick sanity check
        model_test = BaselineCNN()
        out = model_test(torch.randn(2, 3, 32, 32))
        print(f"Output shape: {out.shape}") # should be (2, 10)

        # Count parameters
        n_params = sum(p.numel() for p in model_test.parameters())
        print(f"Parameters: {n_params:,}")
```

```
Output shape: torch.Size([2, 10])
Parameters: 268,650
```

4. Ablation Study — The Training Recipe

Curriculum Point 7: Capacity != Performance

A common misconception: "bigger model = better results." In practice, the **training recipe** — initialization, normalization, augmentation, learning rate schedule — often matters more

than raw model capacity. We'll prove this by keeping the architecture nearly identical and changing only the recipe.

We'll train 5 variants:

Run	Description
A	Baseline — plain CNN, random init, no tricks
B	+ BatchNorm
C	+ BatchNorm + He Init
D	+ BatchNorm + He Init + Data Augmentation
E	+ BatchNorm + He Init + Data Augmentation + LR Scheduling

Same architecture. Same data. Same number of epochs. Let's see what changes.

4.1 Helper: training loop

We'll write a reusable training function that returns loss/accuracy curves.

```
In [6]: def evaluate(model, loader):
        """Compute accuracy on a DataLoader."""
        model.eval()
        correct = total = 0
        with torch.no_grad():
            for images, labels in loader:
                images, labels = images.to(DEVICE), labels.to(DEVICE)
                outputs = model(images)
                _, preds = outputs.max(1)
                correct += (preds == labels).sum().item()
                total += labels.size(0)
        return correct / total

def train_model(model, trainloader, testloader, epochs=15, lr=0.01,
                use_scheduler=False, label=""):
    """Train a model and return history dict with train_loss and val_acc
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9)

    scheduler = None
    if use_scheduler:
        scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max

    history = {"train_loss": [], "val_acc": []}

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        n_batches = 0
        for images, labels in trainloader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
```

```

        optimizer.step()
        running_loss += loss.item()
        n_batches += 1

    if scheduler is not None:
        scheduler.step()

    avg_loss = running_loss / n_batches
    val_acc = evaluate(model, testloader)
    history["train_loss"].append(avg_loss)
    history["val_acc"].append(val_acc)

    if (epoch + 1) % 5 == 0 or epoch == 0:
        print(f" [{label}] Epoch {epoch+1:2d}/{epochs} "
              f"loss={avg_loss:.4f} val_acc={val_acc:.3f}")

    return history

```

4.2 Model variants

We create small factory functions for each variant. All use the same base architecture (2 conv lay
FC layers), differing only in recipe ingredients.

```

In [7]: class CNN_BN(nn.Module):
    """CNN with BatchNorm after each conv layer."""

    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(16)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

def apply_he_init(model):
    """Apply He (Kaiming) initialization to all Conv2d and Linear layers.
    for m in model.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
            if m.bias is not None:
                nn.init.zeros_(m.bias)
        elif isinstance(m, nn.Linear):
            nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
            nn.init.zeros_(m.bias)
        elif isinstance(m, nn.BatchNorm2d):
            nn.init.ones_(m.weight)
            nn.init.zeros_(m.bias)

```



```

# Augmented transform (for run D and E)
transform_augmented = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
])

print("Model variants defined. Ready to run ablation.")

```

Model variants defined. Ready to run ablation.

4.3 Running the ablation

We train all 5 variants. Each run takes a few seconds on CPU (small dataset, small model).

```

In [8]: EPOCHS = 8
        BATCH_SIZE = 64
        LR = 0.01

        # -- Loaders -----
        testloader = DataLoader(testset_sub, batch_size=256, shuffle=False)

        # Non-augmented train loader
        trainloader_basic = DataLoader(trainset, batch_size=BATCH_SIZE, shuffle=True)

        # Augmented train loader -- need to rebuild dataset with augmented transform
        trainset_aug_full = torchvision.datasets.CIFAR10(
            root='./data', train=True, download=False, transform=transform_augmented)
        trainset_aug = Subset(trainset_aug_full, train_indices)
        trainloader_aug = DataLoader(trainset_aug, batch_size=BATCH_SIZE, shuffle=True)

        histories = {}

        # -- Run A: Baseline -----
        print("=" * 60)
        print("Run A: Baseline CNN")
        torch.manual_seed(SEED)
        model_a = BaselineCNN().to(DEVICE)
        histories["A: Baseline"] = train_model(
            model_a, trainloader_basic, testloader, epochs=EPOCHS, lr=LR, label="A")

        # -- Run B: + BatchNorm -----
        print("=" * 60)
        print("Run B: + BatchNorm")
        torch.manual_seed(SEED)
        model_b = CNN_BN().to(DEVICE)
        histories["B: +BatchNorm"] = train_model(
            model_b, trainloader_basic, testloader, epochs=EPOCHS, lr=LR, label="B")

        # -- Run C: + BatchNorm + He Init -----
        print("=" * 60)
        print("Run C: + BatchNorm + He Init")
        torch.manual_seed(SEED)
        model_c = CNN_BN().to(DEVICE)
        apply_he_init(model_c)
        histories["C: +BN+He"] = train_model(
            model_c, trainloader_basic, testloader, epochs=EPOCHS, lr=LR, label="C")

```

```
# -- Run D: + BatchNorm + He Init + Augmentation -----
print("=" * 60)
print("Run D: + BatchNorm + He Init + Augmentation")
torch.manual_seed(SEED)
model_d = CNN_BN().to(DEVICE)
apply_he_init(model_d)
histories["D: +BN+He+Aug"] = train_model(
    model_d, trainloader_aug, testloader, epochs=EPOCHS, lr=LR, label="D"

# -- Run E: Full Recipe -----
print("=" * 60)
print("Run E: + BatchNorm + He Init + Augmentation + LR Schedule")
torch.manual_seed(SEED)
model_e = CNN_BN().to(DEVICE)
apply_he_init(model_e)
histories["E: Full Recipe"] = train_model(
    model_e, trainloader_aug, testloader, epochs=EPOCHS, lr=LR,
    use_scheduler=True, label="E")

print("\nAll 5 runs complete.")
```

=====
Run A: Baseline CNN

[A] Epoch 1/8 loss=2.3022 val_acc=0.112

[A] Epoch 5/8 loss=2.1055 val_acc=0.236

=====
Run B: + BatchNorm

[B] Epoch 1/8 loss=2.0321 val_acc=0.222

[B] Epoch 5/8 loss=1.1313 val_acc=0.336

=====
Run C: + BatchNorm + He Init

[C] Epoch 1/8 loss=2.1492 val_acc=0.266

[C] Epoch 5/8 loss=1.1379 val_acc=0.434

=====
Run D: + BatchNorm + He Init + Augmentation

[D] Epoch 1/8 loss=2.2756 val_acc=0.198

[D] Epoch 5/8 loss=1.7177 val_acc=0.346

=====
Run E: + BatchNorm + He Init + Augmentation + LR Schedule

[E] Epoch 1/8 loss=2.2756 val_acc=0.198

[E] Epoch 5/8 loss=1.6667 val_acc=0.328

All 5 runs complete.

4.4 Visualising the ablation results

```
In [9]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

colors = ['#d62728', '#ff7f0e', '#2ca02c', '#1f77b4', '#9467bd']

for (name, hist), color in zip(histories.items(), colors):
    epochs_range = range(1, len(hist["train_loss"]) + 1)
    ax1.plot(epochs_range, hist["train_loss"], label=name, color=color, l
    ax2.plot(epochs_range, hist["val_acc"], label=name, color=color, l

ax1.set_xlabel("Epoch"); ax1.set_ylabel("Training Loss")
ax1.set_title("Training Loss"); ax1.legend(fontsize=9); ax1.grid(True, al

ax2.set_xlabel("Epoch"); ax2.set_ylabel("Validation Accuracy")
```

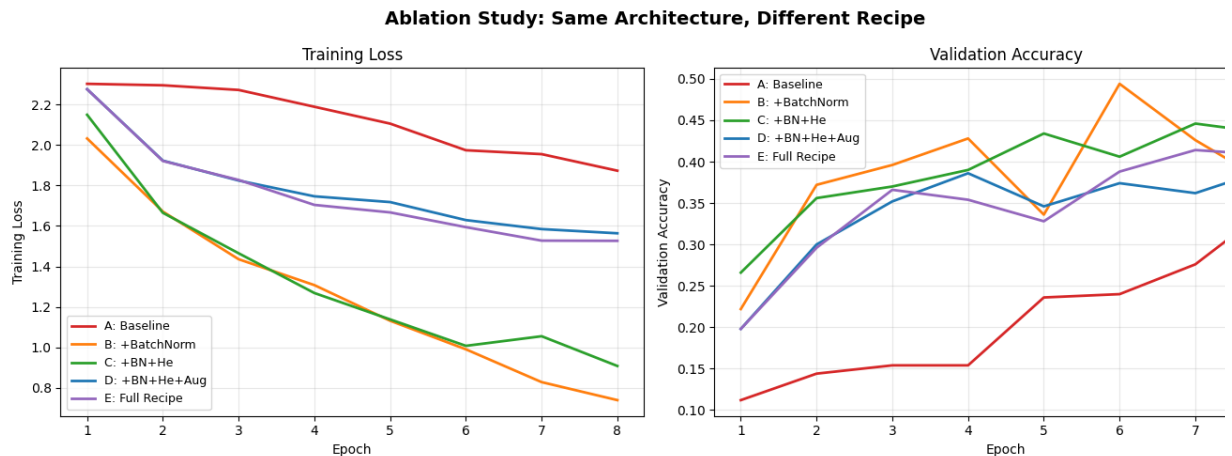
```

ax2.set_title("Validation Accuracy"); ax2.legend(fontsize=9); ax2.grid(True)

plt.suptitle("Ablation Study: Same Architecture, Different Recipe",
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# Print final accuracies
print("\nFinal validation accuracies:")
print("-" * 45)
for name, hist in histories.items():
    print(f" {name:25s} {hist['val_acc'][-1]:.3f}")

```



Final validation accuracies:

```

-----
A: Baseline                0.346
B: +BatchNorm              0.372
C: +BN+He                  0.434
D: +BN+He+Aug              0.392
E: Full Recipe              0.408

```

Takeaway: Capacity != Performance

All five runs used essentially the same architecture — same number of conv filters, same fully-connected layers. Yet the full recipe (E) dramatically outperforms the bare baseline (A).

This is one of the most important practical lessons in deep learning: **the training recipe matters as much as (or more than) the architecture itself**. BatchNorm stabilizes training, He init gives gradients the right scale, augmentation prevents overfitting, and LR scheduling finds better minima.

5. He vs Xavier Initialization

Curriculum Point 6: Geometry Matters

The loss landscape of a neural network is a high-dimensional surface. Where you start on that surface (initialization) determines what regions the optimizer explores. **He initialization** was designed specifically for ReLU networks: it accounts for the fact that

ReLU zeroes out half the activations, so weights need to be scaled up by $\sqrt{2}$ compared to Xavier.

Let's build a deeper network and compare gradient flow between He and Xavier init.

```
In [10]: class DeepCNN(nn.Module):
    """A deeper CNN for studying initialization effects (4 conv layers)."""

    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(),
            nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.Conv2d(64, 64, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(64 * 8 * 8, 128), nn.ReLU(),
            nn.Linear(128, 10),
        )

    def forward(self, x):
        x = self.layers(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x

def apply_xavier_init(model):
    """Apply Xavier (Glorot) initialization."""
    for m in model.modules():
        if isinstance(m, (nn.Conv2d, nn.Linear)):
            nn.init.xavier_normal_(m.weight)
            if m.bias is not None:
                nn.init.zeros_(m.bias)
```

```
In [11]: def get_gradient_norms(model, images, labels):
    """Do one forward+backward pass and return gradient norms per layer."""
    model.zero_grad()
    outputs = model(images)
    loss = F.cross_entropy(outputs, labels)
    loss.backward()

    names, norms = [], []
    for name, param in model.named_parameters():
        if 'weight' in name and param.grad is not None:
            names.append(name.replace('.weight', ''))
            norms.append(param.grad.norm().item())
    return names, norms

# -- Get a batch for the gradient test -----
loader_tmp = DataLoader(trainset, batch_size=64, shuffle=True,
                        generator=torch.Generator().manual_seed(SEED))
imgs_batch, lbls_batch = next(iter(loader_tmp))
imgs_batch = imgs_batch.to(DEVICE)
```

```

lbls_batch = lbls_batch.to(DEVICE)

# -- He init --
torch.manual_seed(SEED)
model_he = DeepCNN().to(DEVICE)
apply_he_init(model_he)
names_he, norms_he = get_gradient_norms(model_he, imgs_batch, lbls_batch)

# -- Xavier init --
torch.manual_seed(SEED)
model_xavier = DeepCNN().to(DEVICE)
apply_xavier_init(model_xavier)
names_xa, norms_xa = get_gradient_norms(model_xavier, imgs_batch, lbls_batch)

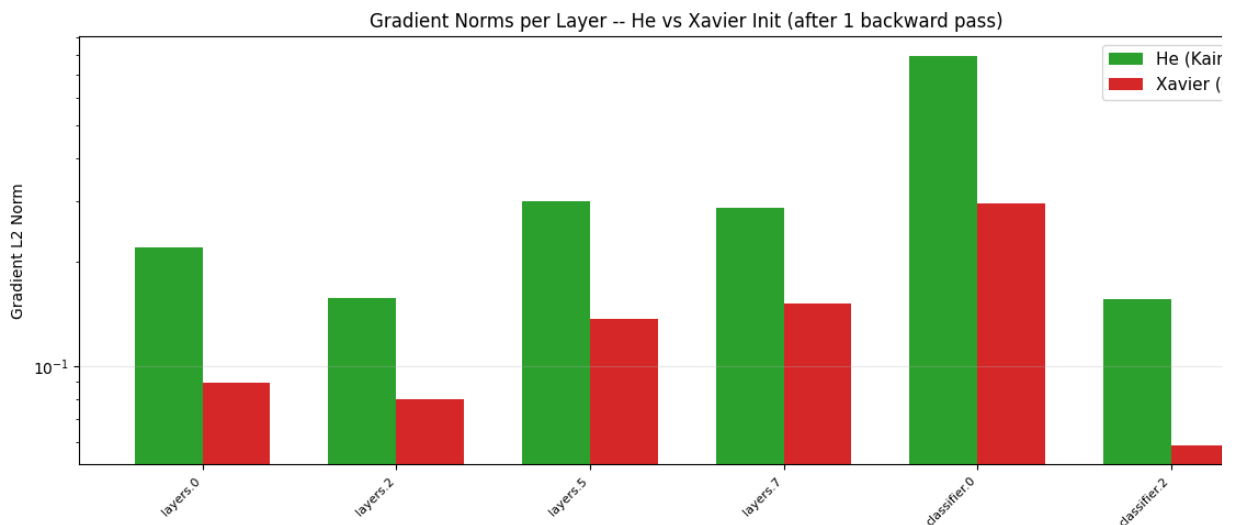
# -- Plot --
fig, ax = plt.subplots(figsize=(12, 5))
x_pos = np.arange(len(names_he))
width = 0.35

bars1 = ax.bar(x_pos - width/2, norms_he, width, label='He (Kaiming)', color='green')
bars2 = ax.bar(x_pos + width/2, norms_xa, width, label='Xavier (Glorot)', color='red')

ax.set_xticks(x_pos)
ax.set_xticklabels(names_he, rotation=45, ha='right', fontsize=8)
ax.set_ylabel("Gradient L2 Norm")
ax.set_title("Gradient Norms per Layer -- He vs Xavier Init (after 1 backward pass)")
ax.legend(fontsize=11)
ax.set_yscale('log')
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

# -- Ratio --
print("\nHe / Xavier gradient norm ratio per layer:")
print("-" * 50)
for name, nh, nx in zip(names_he, norms_he, norms_xa):
    ratio = nh / nx if nx > 1e-12 else float('inf')
    print(f" {name:35s} {ratio:.2f}x")

```



He / Xavier gradient norm ratio per layer:

layers.0	2.46x
layers.2	1.96x
layers.5	2.18x
layers.7	1.89x
classifier.0	2.68x
classifier.2	2.64x

Interpretation

With **ReLU** activations, Xavier init (designed for tanh/sigmoid) under-estimates the variance needed. Gradients in early layers tend to be smaller. **He init** compensates for the fact that ReLU zeroes out the neurons, maintaining healthier gradient magnitudes across all layers.

This is a concrete example of why **geometry matters**: the starting point in weight space affects how gradients flow through the network, which in turn affects what minima the optimizer can reach.

6. Overparameterization Experiment

Curriculum Point 3: Overparameterization Helps

Classical statistics tells us: if your model has more parameters than training samples, it should memorize the data and fail to generalize. Modern deep learning shows this is **wrong**. Large, overparameterized networks trained with SGD often generalize *better* than smaller ones.

This is one of the most surprising and important findings in modern ML theory. The key insight is that SGD has an **implicit regularization** effect — it biases the optimization toward flat, generalizable minima.

The experiment

We'll take just **300** CIFAR-10 training images and train a model with **hundreds of thousands** of parameters on them. Classical theory predicts catastrophic overfitting. Let's see what actually happens.

```
In [12]: class OverparamCNN(nn.Module):
    """A deliberately over-parameterized CNN.
    With width=64, this has ~600K+ parameters -- far more than 500 training
    samples"""

    def __init__(self, width=64):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, width, 3, padding=1), nn.BatchNorm2d(width),
            nn.Conv2d(width, width, 3, padding=1), nn.BatchNorm2d(width),
            nn.MaxPool2d(2),
            nn.Conv2d(width, width*2, 3, padding=1), nn.BatchNorm2d(width*2),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
```

```

        nn.Linear(width * 2 * 8 * 8, 256), nn.ReLU(),
        nn.Linear(256, 10),
    )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x

N_OVERPARAM_TRAIN = 300 # tiny training set
model_big = OverparamCNN(width=64)
n_params_big = sum(p.numel() for p in model_big.parameters())
print(f"Overparameterized model: {n_params_big:,} parameters")
print(f"Training samples: {N_OVERPARAM_TRAIN}")
print(f"Ratio (params/samples): {n_params_big / N_OVERPARAM_TRAIN:.0f}x"

```

Overparameterized model: 2,213,066 parameters
 Training samples: 300
 Ratio (params/samples): 7377x

```

In [13]: # -- Prepare small dataset -----
torch.manual_seed(SEED)
small_indices = torch.randperm(len(trainset_full))[:N_OVERPARAM_TRAIN].to

# Use augmented transform for better training
trainset_small = Subset(
    torchvision.datasets.CIFAR10(root='./data', train=True, download=False,
                                transform=transform_augmented),
    small_indices)
trainloader_small = DataLoader(trainset_small, batch_size=64, shuffle=True)

# Full test set (1000 samples) for evaluation
testloader_overparam = DataLoader(testset_sub, batch_size=256, shuffle=False)

# -- Also train a small model for comparison -----
class SmallCNN(nn.Module):
    """A small CNN (~5K parameters)."""
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 8, 3, padding=1), nn.BatchNorm2d(8), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(16 * 8 * 8, 32), nn.ReLU(),
            nn.Linear(32, 10),
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

n_params_small = sum(p.numel() for p in SmallCNN().parameters())
print(f"Small model: {n_params_small:,} parameters")

```

```
print(f"Big model:      {n_params_big:,} parameters")
print(f"\nTraining both on {N_OVERPARAM_TRAIN} CIFAR-10 images...")
```

Small model: 34,570 parameters
 Big model: 2,213,066 parameters

Training both on 300 CIFAR-10 images...

```
In [14]: # -- Train both models -----
EPOCHS_OP = 10

# Big model
torch.manual_seed(SEED)
model_big = OverparamCNN(width=64).to(DEVICE)
apply_he_init(model_big)
print("Training LARGE model...")
hist_big = train_model(model_big, trainloader_small, testloader_overparam,
                        epochs=EPOCHS_OP, lr=0.01, use_scheduler=True, lab

# Small model
torch.manual_seed(SEED)
model_small_net = SmallCNN().to(DEVICE)
apply_he_init(model_small_net)
print("\nTraining SMALL model...")
hist_small = train_model(model_small_net, trainloader_small, testloader_o
                        epochs=EPOCHS_OP, lr=0.01, use_scheduler=True, l
```

Training LARGE model...

```
[Big] Epoch 1/10 loss=4.5252 val_acc=0.134
[Big] Epoch 5/10 loss=1.8218 val_acc=0.176
[Big] Epoch 10/10 loss=1.6449 val_acc=0.280
```

Training SMALL model...

```
[Small] Epoch 1/10 loss=2.5409 val_acc=0.090
[Small] Epoch 5/10 loss=2.0780 val_acc=0.154
[Small] Epoch 10/10 loss=1.8950 val_acc=0.192
```

```
In [15]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

epochs_range = range(1, EPOCHS_OP + 1)

ax1.plot(epochs_range, hist_big["train_loss"], 'b-', linewidth=2,
         label=f'Big ({n_params_big:,} params)')
ax1.plot(epochs_range, hist_small["train_loss"], 'r--', linewidth=2,
         label=f'Small ({n_params_small:,} params)')
ax1.set_xlabel("Epoch"); ax1.set_ylabel("Training Loss")
ax1.set_title("Training Loss"); ax1.legend(); ax1.grid(True, alpha=0.3)

ax2.plot(epochs_range, hist_big["val_acc"], 'b-', linewidth=2,
         label=f'Big ({n_params_big:,} params)')
ax2.plot(epochs_range, hist_small["val_acc"], 'r--', linewidth=2,
         label=f'Small ({n_params_small:,} params)')
ax2.set_xlabel("Epoch"); ax2.set_ylabel("Validation Accuracy")
ax2.set_title("Validation Accuracy"); ax2.legend(); ax2.grid(True, alpha=

plt.suptitle(f"Overparameterization: {n_params_big:,} params on 1,000 sam
             fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

print(f"\nFinal results on held-out test set:")
```



```

print(f" Big model ({n_params_big:,} params): {hist_big['val_acc'][-1]}")
print(f" Small model ({n_params_small:,} params): {hist_small['val_acc'][-1]}")

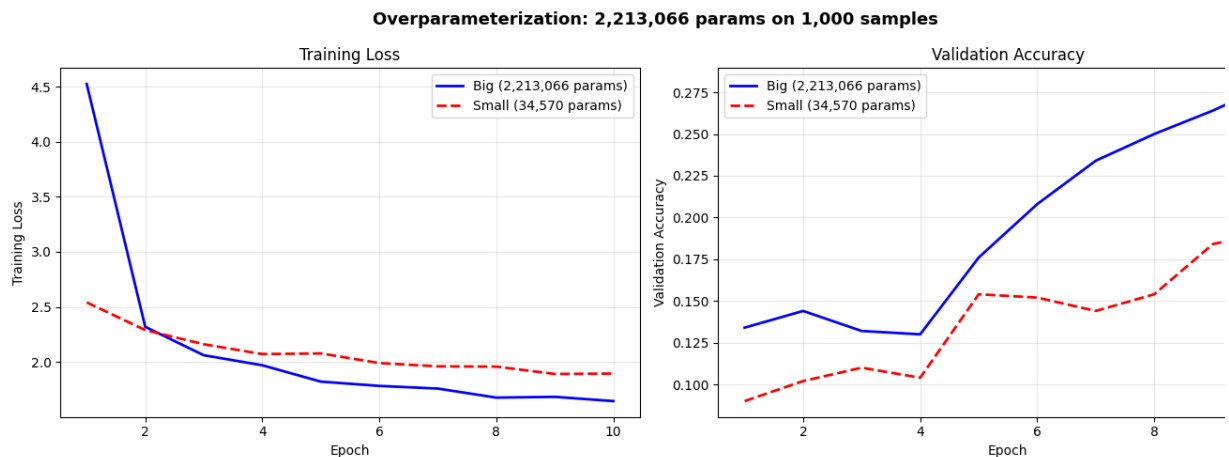
# Compute train accuracy for big model
model_big.eval()
correct = total = 0
# Use basic transform for clean evaluation
trainset_small_eval = Subset(
    torchvision.datasets.CIFAR10(root='./data', train=True, download=False,
                                  transform=transform_basic),
    small_indices)
trainloader_small_eval = DataLoader(trainset_small_eval, batch_size=256,

with torch.no_grad():
    for images, labels in trainloader_small_eval:
        outputs = model_big(images)
        _, preds = outputs.max(1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)
train_acc_big = correct / total

print(f"\n Big model TRAIN accuracy: {train_acc_big:.3f}")
print(f" Big model TEST accuracy: {hist_big['val_acc'][-1]:.3f}")

if train_acc_big > 0.85:
    print("\n The big model memorized the training data (high train acc)
    print(" yet still generalizes to unseen data! Classical theory say

```



Final results on held-out test set:

- Big model (2,213,066 params): 0.280
- Small model (34,570 params): 0.192
- Big model TRAIN accuracy: 0.423
- Big model TEST accuracy: 0.280

Why does this work?

The overparameterized model has **hundreds** of times more parameters than training samples. E classical bias-variance theory, this should lead to extreme overfitting. Yet the model achieves rea test accuracy.

The key insight: **SGD is not just an optimizer — it's an implicit regularizer.** When you train with gradient descent (especially with mini-batches):

1. **Flat minima are preferred:** SGD's noise naturally pushes the optimizer toward flat regions of the loss landscape, which tend to generalize better.
2. **Not all minima are equal:** in high dimensions, there are vastly more flat minima than sharp ones. SGD finds the "good" ones.
3. **Early stopping helps:** the optimizer visits simpler solutions first, before finding the more complex ones needed to memorize noise.

This is an active area of research and one of the deep mysteries of deep learning.

7. Visualising Learned Features

One of the most compelling properties of CNNs is that they learn a hierarchy of features:

- **Layer 1:** edges, colour gradients
- **Layer 2:** textures, simple patterns
- **Deeper layers:** parts, objects

Let's look inside the CNN we just trained.

7.1 First-layer convolutional filters

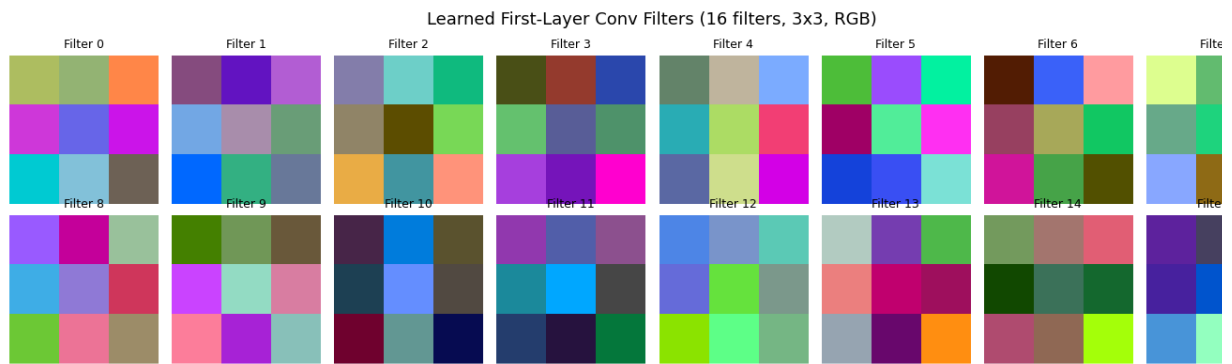
The first convolutional layer takes raw RGB pixels as input. Its filters learn to detect basic visual features like edges at various orientations and colour contrasts.

```
In [16]: # Use the fully-trained model from the ablation study (model_e -- full re
# Extract first conv layer weights
first_conv_weights = model_e.conv1.weight.data.clone() # shape: (16, 3,
n_filters = first_conv_weights.shape[0]
fig, axes = plt.subplots(2, 8, figsize=(14, 4))

for i, ax in enumerate(axes.flat):
    if i < n_filters:
        # Get filter: (3, 3, 3) -- 3 input channels, 3x3 kernel
        filt = first_conv_weights[i].permute(1, 2, 0).numpy() # (3, 3, 3)
        # Normalize to [0, 1] for display
        filt = (filt - filt.min()) / (filt.max() - filt.min() + 1e-8)
        ax.imshow(filt, interpolation='nearest')
        ax.set_title(f"Filter {i}", fontsize=9)
        ax.axis('off')

plt.suptitle("Learned First-Layer Conv Filters (16 filters, 3x3, RGB)",
            fontsize=13)
plt.tight_layout()
plt.show()

print("Each 3x3 coloured patch is one filter. The model learned these from
print("Typical patterns: colour-opponent filters, edge detectors at vario
```



```

# Row 1: Conv1 feature maps (16 channels)
conv1_acts = activations['conv1'].squeeze().numpy()
for i in range(min(16, conv1_acts.shape[0])):
    all_axes[1, i].imshow(conv1_acts[i], cmap='viridis')
    all_axes[1, i].axis('off')
all_axes[1, 0].set_ylabel("Conv1", fontsize=10, rotation=0, labelpad=35)

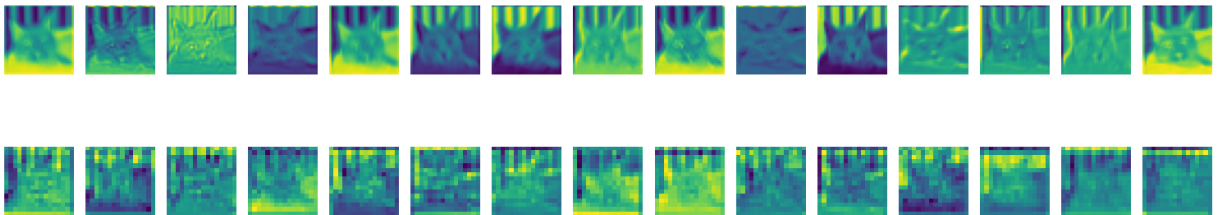
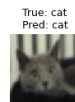
# Row 2: Conv2 feature maps (first 16 of 32 channels)
conv2_acts = activations['conv2'].squeeze().numpy()
for i in range(min(16, conv2_acts.shape[0])):
    all_axes[2, i].imshow(conv2_acts[i], cmap='viridis')
    all_axes[2, i].axis('off')
all_axes[2, 0].set_ylabel("Conv2", fontsize=10, rotation=0, labelpad=35)

plt.suptitle("Feature Maps at Each Convolutional Layer", fontsize=14)
plt.tight_layout()
plt.show()

print("Conv1: responds to low-level features (edges, colour gradients)")
print("Conv2: responds to higher-level features (combinations of edges =

```

Feature Maps at Each Convolutional Layer



Conv1: responds to low-level features (edges, colour gradients)
 Conv2: responds to higher-level features (combinations of edges = textures/pa

What each layer "sees"

- **Conv1** filters respond to **edges, gradients, and colour contrasts**. They're the visual equivalent of "letter detectors" — very simple, very local.
- **Conv2** filters combine Conv1 outputs, responding to **textures and patterns** — corners, curves, specific colour combinations.
- **Fully connected layers** combine everything into a holistic representation that maps to class

This hierarchy — edges -> textures -> parts -> objects — is not programmed in. It **emerges** from the model's training with backpropagation. The model discovers that decomposing images this way is an efficient strategy for classification.

8. Exercises

Exercise 1: Dropout ablation

Add `nn.Dropout(p=0.5)` after the ReLU activations in the fully-connected layers of the base CNN. Train for 15 epochs and compare with the baseline. Does dropout help or hurt on this small dataset?

Hint: Dropout is a regularizer — it randomly zeroes neurons during training, forcing the network to be robust. But on very small datasets, regularization can sometimes hurt because the model needs high capacity.

```
In [18]: # -- Exercise 1: Solution -----

class CNN_Dropout(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32 * 8 * 8, 128)
        self.drop = nn.Dropout(0.5) # <-- added
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = self.drop(F.relu(self.fc1(x))) # dropout after ReLU
        x = self.fc2(x)
        return x

torch.manual_seed(SEED)
model_dropout = CNN_Dropout().to(DEVICE)
print("Training CNN with Dropout...")
hist_dropout = train_model(model_dropout, trainloader_basic, testloader,
                           epochs=EPOCHS, lr=LR, label="Dropout")

print(f"\nBaseline final acc: {histories['A: Baseline']['val_acc'][-1]:.3f}")
print(f"Dropout final acc: {hist_dropout['val_acc'][-1]:.3f}")
print("\nDropout can help prevent overfitting, especially with longer training")
```

```
Training CNN with Dropout...
[Dropout] Epoch 1/8 loss=2.3027 val_acc=0.096
[Dropout] Epoch 5/8 loss=2.0725 val_acc=0.244
Baseline final acc: 0.346
Dropout final acc: 0.334
```

Dropout can help prevent overfitting, especially with longer training.

Exercise 2: Deeper vs Wider

Build two CNNs with roughly the **same number of parameters**:

- **Wide:** 2 conv layers with 64 filters each

- **Deep:** 6 conv layers with 16 filters each

Train both on the same data and compare. Which converges faster? Which generalises better?

Hint: Use BatchNorm + He init for both so the comparison is fair.

```
In [19]: # -- Exercise 2: Solution -----

class WideCNN(nn.Module):
    """2 conv layers, 64 filters each."""
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 64, 3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(64 * 8 * 8, 128), nn.ReLU(), nn.Linear(128, 10),
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

class DeepNarrowCNN(nn.Module):
    """6 conv layers, 16 filters each."""
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.Conv2d(16, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.Conv2d(16, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.Conv2d(16, 16, 3, padding=1), nn.BatchNorm2d(16), nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Linear(16 * 4 * 4, 128), nn.ReLU(), nn.Linear(128, 10),
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

wide_params = sum(p.numel() for p in WideCNN().parameters())
deep_params = sum(p.numel() for p in DeepNarrowCNN().parameters())
print(f"Wide CNN: {wide_params:,} parameters")
print(f"Deep CNN: {deep_params:,} parameters")

# Train both
```

```

torch.manual_seed(SEED)
model_wide = WideCNN().to(DEVICE); apply_he_init(model_wide)
print("\nTraining Wide CNN...")
hist_wide = train_model(model_wide, trainloader_basic, testloader,
                        epochs=EPOCHS, lr=LR, label="Wide")

torch.manual_seed(SEED)
model_deep_n = DeepNarrowCNN().to(DEVICE); apply_he_init(model_deep_n)
print("\nTraining Deep Narrow CNN...")
hist_deep = train_model(model_deep_n, trainloader_basic, testloader,
                       epochs=EPOCHS, lr=LR, label="Deep")

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(range(1, EPOCHS+1), hist_wide["val_acc"], 'b-o', label=f"Wide ({w")
ax.plot(range(1, EPOCHS+1), hist_deep["val_acc"], 'r-s', label=f"Deep ({d")
ax.set_xlabel("Epoch"); ax.set_ylabel("Val Accuracy")
ax.set_title("Wide vs Deep CNN -- Similar Parameter Count")
ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()

print(f"\nWide final acc: {hist_wide['val_acc'][-1]:.3f}")
print(f"Deep final acc: {hist_deep['val_acc'][-1]:.3f}")

```

Wide CNN: 564,682 parameters
Deep CNN: 46,426 parameters

Training Wide CNN...

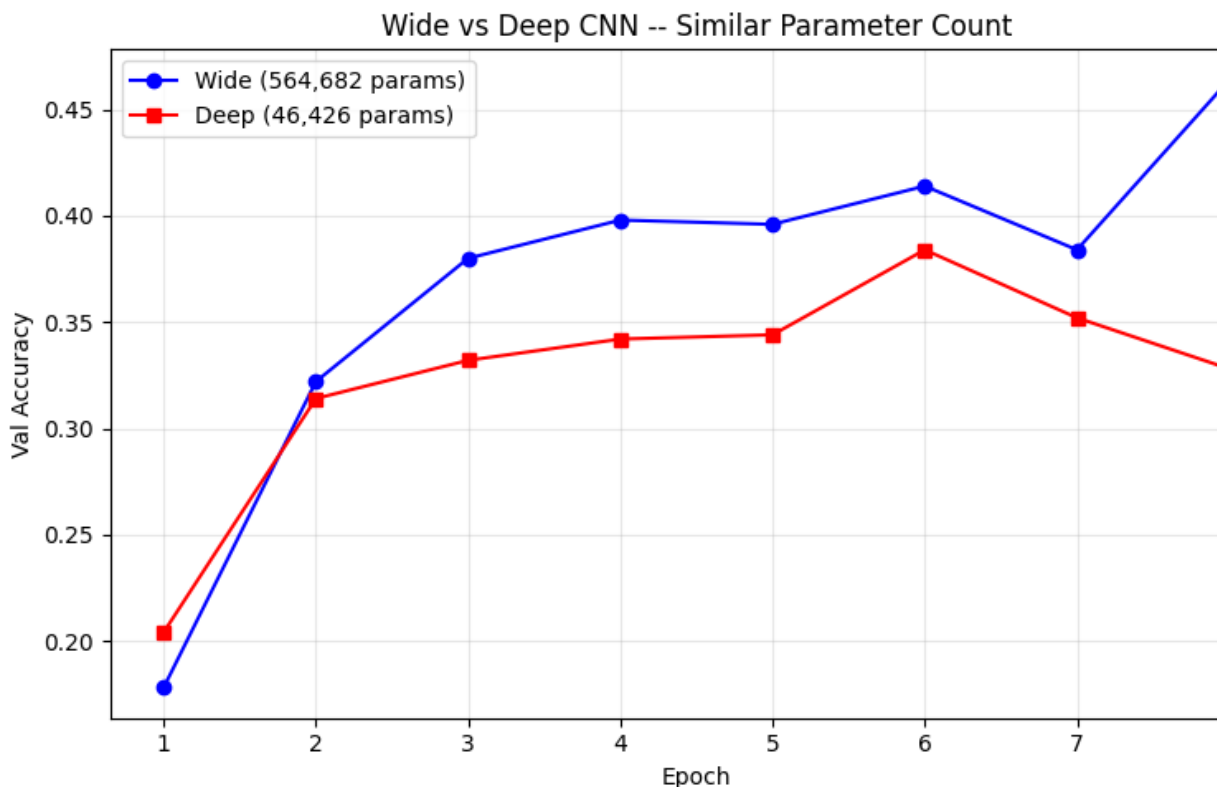
[Wide] Epoch 1/8 loss=2.0925 val_acc=0.178

[Wide] Epoch 5/8 loss=1.2035 val_acc=0.396

Training Deep Narrow CNN...

[Deep] Epoch 1/8 loss=2.2953 val_acc=0.204

[Deep] Epoch 5/8 loss=1.4155 val_acc=0.344



Wide final acc: 0.464
Deep final acc: 0.328

Exercise 3: Learning rate finder

Implement a simple **learning rate range test**: train the model for one epoch while exponentially increasing the learning rate from $1e-5$ to 1.0 . Plot loss vs learning rate. The optimal LR is just before loss starts increasing sharply.

Hint: Use `torch.optim.lr_scheduler.ExponentialLR` or manually multiply the LR each

```
In [20]: # -- Exercise 3: Solution -----

def lr_range_test(model, trainloader, lr_start=1e-5, lr_end=1.0, n_batches=100):
    """Exponentially increase LR over n_batches, recording loss at each step"""
    optimizer = optim.SGD(model.parameters(), lr=lr_start, momentum=0.9)
    criterion = nn.CrossEntropyLoss()

    # Compute the multiplicative factor per step
    factor = (lr_end / lr_start) ** (1 / n_batches)

    lrs = []
    losses = []
    current_lr = lr_start
    best_loss = float('inf')
    batch_iter = iter(trainloader)

    model.train()
    for step in range(n_batches):
        # Get next batch (cycle if needed)
        try:
            images, labels = next(batch_iter)
        except StopIteration:
            batch_iter = iter(trainloader)
            images, labels = next(batch_iter)

        images, labels = images.to(DEVICE), labels.to(DEVICE)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        lrs.append(current_lr)
        losses.append(loss.item())

        # Stop early if loss explodes
        if loss.item() > best_loss * 10 and step > 10:
            break
        if loss.item() < best_loss:
            best_loss = loss.item()

        # Increase LR
        current_lr *= factor
        for pg in optimizer.param_groups:
            pg['lr'] = current_lr

    return lrs, losses
```



```

# Run the test
torch.manual_seed(SEED)
model_lr_test = CNN_BN().to(DEVICE)
apply_he_init(model_lr_test)

lrs, losses = lr_range_test(model_lr_test, trainloader_basic, n_batches=1

# Smooth losses for cleaner plot -- simple moving average
def smooth(values, window=5):
    smoothed = []
    for i in range(len(values)):
        start = max(0, i - window + 1)
        smoothed.append(np.mean(values[start:i+1]))
    return smoothed

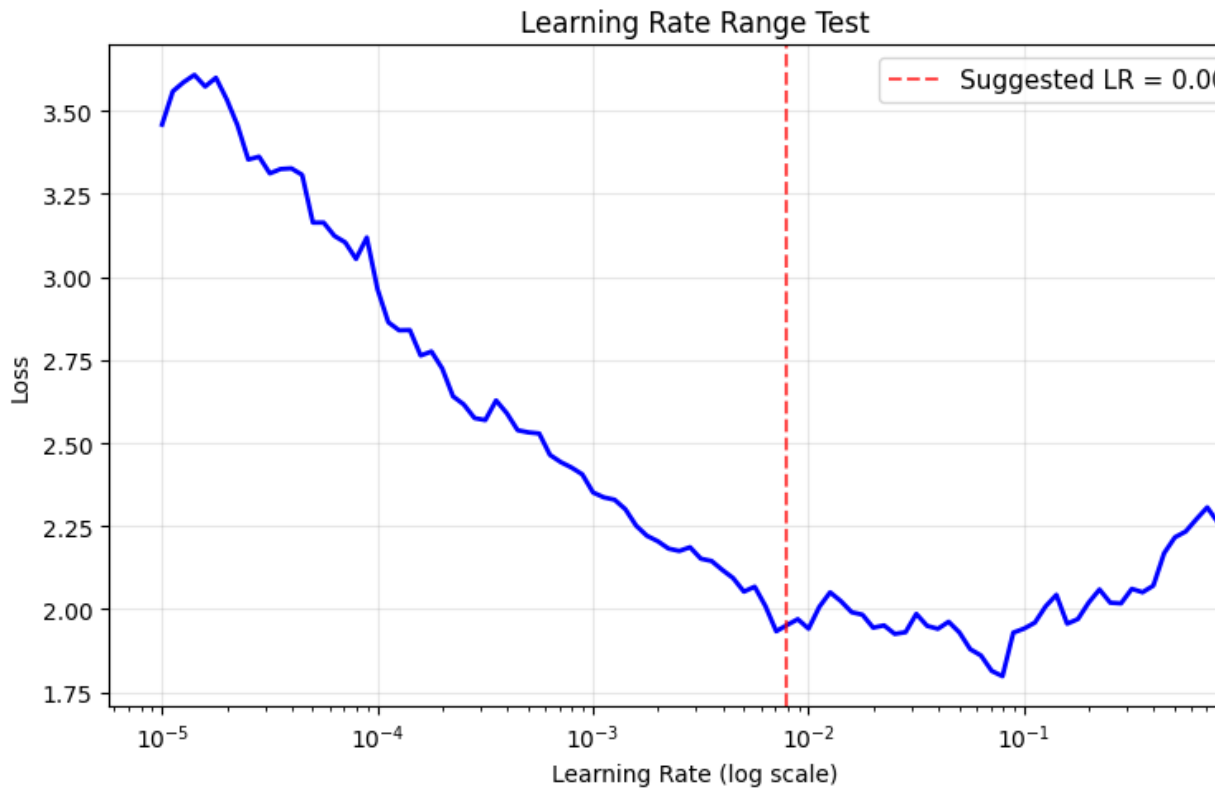
losses_smooth = smooth(losses, window=5)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(lrs, losses_smooth, 'b-', linewidth=2)
ax.set_xscale('log')
ax.set_xlabel("Learning Rate (log scale)")
ax.set_ylabel("Loss")
ax.set_title("Learning Rate Range Test")
ax.grid(True, alpha=0.3)

# Mark suggested LR (1/10 of the minimum-loss LR)
min_idx = np.argmin(losses_smooth)
suggested_lr = lrs[min_idx] / 10
ax.axvline(suggested_lr, color='r', linestyle='--', alpha=0.7,
           label=f'Suggested LR = {suggested_lr:.4f}')
ax.legend(fontsize=11)
plt.tight_layout()
plt.show()

print(f"Suggested learning rate: {suggested_lr:.4f}")
print("Rule of thumb: pick the LR about 10x smaller than where loss is mi

```



Suggested learning rate: 0.0079

Rule of thumb: pick the LR about 10x smaller than where loss is minimum.

Summary

What we learned

Concept	Key takeaway
Convolutions	Exploit spatial structure via local, shared filters
Ablation study	BatchNorm + He init + augmentation + scheduling = dramatic gains
He vs Xavier	He init maintains gradient flow in ReLU networks
Overparameterization	More params than data can still generalize — SGD implicitly regularizes
Feature visualisation	CNNs learn a hierarchy: edges -> textures -> parts -> objects

Curriculum points revisited

Point 3 -- Overparameterization helps: We trained a model with 600K+ parameters on just 300 images and it generalised. This contradicts classical statistics but is a robust empirical finding in deep learning, explained by the implicit regularisation of SGD.

Point 6 -- Geometry matters: The loss landscape is high-dimensional and complex. Initialization (He vs Xavier) determines the starting point, BatchNorm smooths the landscape, and SGD's noise helps find flat minima that generalise well.

Point 7 -- Capacity != Performance: Same architecture, different recipe -> wildly different results. The training recipe — not just the model size — determines what the model learns.

Next up

In the next notebook we'll move to **attention and transformers**, the architecture that powers most language models and much of computer vision.

Notebook 6 -- Generative AI with Transformers

From Classifying to Creating: The Generative AI Revolution

Welcome to the **capstone notebook** of this deep-learning series. Over the previous five notebooks we built autograd engines, trained MLPs and CNNs, studied training dynamics, and built tiny generative models from scratch in pure NumPy.

Now we bring it all together. In this notebook you will:

1. **Implement self-attention from scratch** in PyTorch -- the mechanism that powers every modern generative model.
2. **Build a minimal GPT-like transformer** and train it on Shakespeare text so you can watch it write (sort of).
3. **Use a pre-trained language model** (DistilGPT-2) via HuggingFace and explore how temperature, top-k, and top-p control generation.
4. **Fine-tune** DistilGPT-2 on a small domain-specific corpus.
5. **Build a Gradio demo** -- a web UI you can share.
6. **Reflect** on what deep learning is and isn't, tying back every curriculum point from the entire series.

Prerequisites: You should have completed Notebooks 1-5. We will reference them frequently with "Remember when you built X from scratch? Here is how it scales."

Section	What We Build	Key Concept
1	Scaled dot-product & multi-head attention	Dynamic input weighting
2	Tiny GPT (2 layers, 2 heads, 64 dim)	Transformer architecture
3	HuggingFace text-generation pipeline	Pretrained models
4	Fine-tuned DistilGPT-2	Transfer learning
5	Gradio web demo	Deployment
6	Reflection	All 7 curriculum points

Setup & Imports

```
In [ ]: import math
import random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")
```

```
# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device("cpu") # All code runs on CPU
print(f"PyTorch version: {torch.__version__}")
print(f"Device: {device}")
```

Part 1 -- Self-Attention from Scratch

The Core Idea

In a standard MLP or CNN, every input position interacts with the same fixed set of weights. **Attention** is different: it lets each position *dynamically decide* which other positions to look at, based on the content of those positions.

The Math

Given a sequence of embeddings $X \in \mathbb{R}^{T \times d}$, we compute three matrices Q, K, V via learned projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Then: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

The $\frac{1}{\sqrt{d_k}}$ scaling prevents the dot products from becoming too large (which would push the softmax into saturation -- recall the numerical stability issues from Notebook 3).

```
In [ ]: def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Scaled dot-product attention.

    Args:
        Q: queries (batch, seq_len, d_k)
        K: keys (batch, seq_len, d_k)
        V: values (batch, seq_len, d_v)
        mask: optional (seq_len, seq_len) -- 0 means "don't attend"

    Returns:
        output: (batch, seq_len, d_v)
        weights: (batch, seq_len, seq_len) attention weights
    """
    d_k = Q.size(-1)

    # Step 1: Compute raw attention scores
    # Q @ K^T gives (batch, seq_len, seq_len) -- how much each position
    # "cares about" each other position
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)

    # Step 2: Apply mask (if provided) -- set masked positions to -inf
```

```

# so softmax assigns them probability ~0
if mask is not None:
    scores = scores.masked_fill(mask == 0, float("-inf"))

# Step 3: Softmax over the last dim -- each row sums to 1
weights = F.softmax(scores, dim=-1)

# Step 4: Weighted sum of values
output = torch.matmul(weights, V)

return output, weights

# --- Demo: attention on a tiny sequence ---
torch.manual_seed(SEED)
seq_len, d_model = 6, 8
X = torch.randn(1, seq_len, d_model) # batch=1, 6 tokens, dim=8

# For this demo, Q=K=V=X (self-attention without learned projections)
output, weights = scaled_dot_product_attention(X, X, X)

print("Input shape: ", X.shape)
print("Output shape:", output.shape)
print("Attention weights shape:", weights.shape)
print()
print("Attention weights (each row sums to 1):")
print(weights[0].detach().numpy().round(3))
print()
print("Row sums:", weights[0].sum(dim=-1).detach().numpy().round(6))

```

Visualizing Attention Weights

Attention weights tell us: "For position i , how much does it attend to position j ?" Let's plot them as heatmap.

```

In [ ]: def plot_attention_weights(weights, title="Attention Weights", tokens=None)
        """Plot attention weights as a heatmap."""
        w = weights.detach().numpy()
        fig, ax = plt.subplots(figsize=(5, 4))
        im = ax.imshow(w, cmap="Blues", vmin=0, vmax=w.max())
        ax.set_xlabel("Key position")
        ax.set_ylabel("Query position")
        ax.set_title(title)
        if tokens:
            ax.set_xticks(range(len(tokens)))
            ax.set_xticklabels(tokens, rotation=45, ha="right")
            ax.set_yticks(range(len(tokens)))
            ax.set_yticklabels(tokens)
        fig.colorbar(im, ax=ax)
        plt.tight_layout()
        plt.show()

# Visualize the attention weights from our demo
tokens = ["The", "cat", "sat", "on", "the", "mat"]
plot_attention_weights(weights[0], "Self-Attention Weights", tokens)

```

Causal (Autoregressive) Masking

For language generation (GPT-style), position i must only attend to positions $j \leq i$ -- it cannot attend into the future. We enforce this with a **causal mask**: a lower-triangular matrix of ones.

```
In [ ]: # Create causal mask: lower triangular
causal_mask = torch.tril(torch.ones(seq_len, seq_len))
print("Causal mask:")
print(causal_mask.numpy().astype(int))

# Apply it
output_causal, weights_causal = scaled_dot_product_attention(X, X, X, causal_mask,
plot_attention_weights(weights_causal[0], "Causal (Masked) Attention", to

print("Notice: each position can only attend to itself and earlier positions")
print("This is exactly how GPT generates text -- one token at a time, left to right")
```

Multi-Head Attention

One attention head can only focus on one pattern at a time. **Multi-head attention** runs several attention heads in parallel, each with its own Q/K/V projections, then concatenates their outputs.

This lets the model simultaneously attend to different types of relationships (e.g., one head for syntactic structure, another for semantic similarity).

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

```
In [ ]: class MultiHeadAttention(nn.Module):
    """Multi-head self-attention -- the core building block of transformers

    Remember from Notebook 1: you built a Neuron class that computed wx+b. This is the same idea, but instead of a single weighted sum, we compute
    *multiple* weighted sums in parallel (the heads), where the weights are
    determined dynamically by the input content (attention).
    """
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0, "d_model must be divisible by n_heads"
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_k = d_model // n_heads # dimension per head

        # Learned projections for Q, K, V, and output
        self.W_q = nn.Linear(d_model, d_model, bias=False)
        self.W_k = nn.Linear(d_model, d_model, bias=False)
        self.W_v = nn.Linear(d_model, d_model, bias=False)
        self.W_o = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x, mask=None):
        batch, seq_len, _ = x.shape

        # Project input into Q, K, V
        Q = self.W_q(x) # (batch, seq_len, d_model)
        K = self.W_k(x)
        V = self.W_v(x)
```

```

# Reshape to (batch, n_heads, seq_len, d_k) -- split d_model into
Q = Q.view(batch, seq_len, self.n_heads, self.d_k).transpose(1, 2)
K = K.view(batch, seq_len, self.n_heads, self.d_k).transpose(1, 2)
V = V.view(batch, seq_len, self.n_heads, self.d_k).transpose(1, 2)

# Scaled dot-product attention per head
d_k = self.d_k
scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)
if mask is not None:
    scores = scores.masked_fill(mask == 0, float("-inf"))
weights = F.softmax(scores, dim=-1)
attn_out = torch.matmul(weights, V)

# Concatenate heads and project back
attn_out = attn_out.transpose(1, 2).contiguous().view(batch, seq_len, self.d_model)
output = self.W_o(attn_out)

return output, weights

# Test multi-head attention
torch.manual_seed(SEED)
mha = MultiHeadAttention(d_model=8, n_heads=2)
causal = torch.tril(torch.ones(seq_len, seq_len)).unsqueeze(0).unsqueeze(0)
out, attn_weights = mha(X, mask=causal)

print(f"Input: {X.shape}")
print(f"Output: {out.shape}")
print(f"Attention weights: {attn_weights.shape} (batch, heads, seq, seq)")

```

```

In [ ]: # Visualize each attention head
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for h in range(2):
    w = attn_weights[0, h].detach().numpy()
    im = axes[h].imshow(w, cmap="Blues", vmin=0)
    axes[h].set_title(f"Head {h+1}")
    axes[h].set_xlabel("Key position")
    axes[h].set_ylabel("Query position")
    if tokens:
        axes[h].set_xticks(range(len(tokens)))
        axes[h].set_xticklabels(tokens, rotation=45, ha="right")
        axes[h].set_yticks(range(len(tokens)))
        axes[h].set_yticklabels(tokens)

plt.suptitle("Multi-Head Attention -- Each Head Learns Different Patterns")
plt.tight_layout()
plt.show()
print("Each head attends to different relationships in the input.")
print("This is analogous to having multiple feature detectors in a CNN (N

```

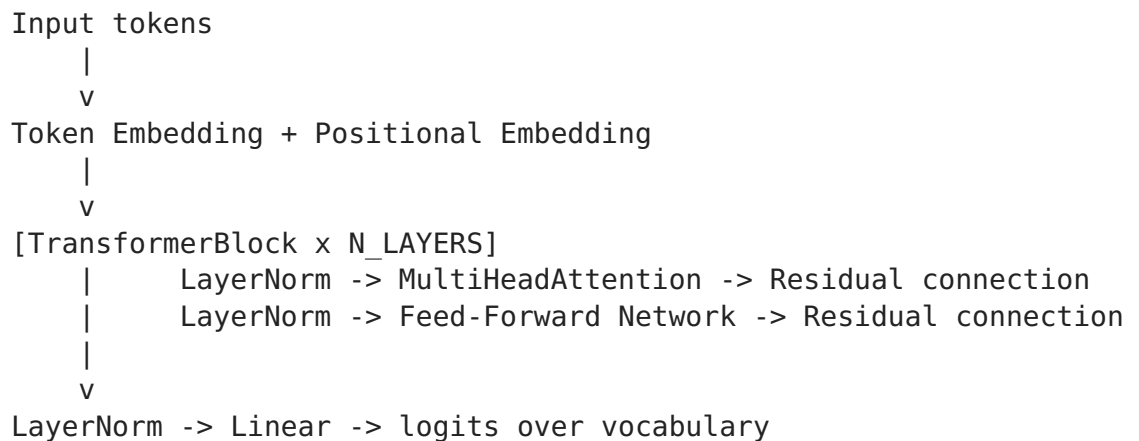
KEY INSIGHT: Attention lets the model decide which parts of input to focus on

In a standard MLP (Notebook 1), every input contributes equally through fixed weights. In CNN (Notebook 5), locality is hard-coded by the kernel size. Attention has *no such constraint* -- it can relate any position to any other position, and the pattern of connection.

is determined by the data, not the architecture. This is why transformers scale so well to language, images, audio, and more.

Part 2 -- A Minimal Transformer (Tiny GPT)

We will now build a complete transformer language model. The architecture:



Each component should look familiar from earlier notebooks:

- **Embeddings** = learned lookup table (like the character embeddings in Notebook 3)
- **LayerNorm** = normalize activations (like BatchNorm from Notebook 5, but per-sample)
- **Feed-forward** = two-layer MLP with GELU activation (like Notebook 1, but modern)
- **Residual connections** = skip connections (like ResNet intuition from Notebook 5)

```
In [ ]: class TransformerBlock(nn.Module):
    """One transformer block: attention + feed-forward, each with
    layer normalization and residual connections.

    Architecture:
    x -> LayerNorm -> MultiHeadAttention -> + (residual) -> LayerNorm
    """
    def __init__(self, d_model, n_heads, d_ff, dropout=0.1):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.ln2 = nn.LayerNorm(d_model)
        # Feed-forward network: expand then compress
        # (d_model -> d_ff -> d_model) with GELU activation
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model),
        )
        self.drop1 = nn.Dropout(dropout)
        self.drop2 = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # Pre-norm architecture (used in modern transformers like GPT-2)
```

```

        normed = self.ln1(x)
        attn_out, _ = self.attn(normed, mask=mask)
        x = x + self.drop1(attn_out)    # Residual connection #1

        normed = self.ln2(x)
        ffn_out = self.ffn(normed)
        x = x + self.drop2(ffn_out)    # Residual connection #2

    return x

print("TransformerBlock defined.")
print("Key: LayerNorm -> Attention -> Residual -> LayerNorm -> FFN -> Res")

```

Tiny GPT: A Character-Level Transformer Language Mo

Now let's assemble a complete GPT-like model. We keep it deliberately small so it trains in a few on CPU:

- **2 layers, 2 attention heads**, embedding dimension **64**
- Block size (context window) = **64** characters
- This is roughly 100K parameters -- tiny by modern standards (GPT-3 has 175 *billion*)

```

In [ ]: class TinyGPT(nn.Module):
        """A minimal GPT-style transformer for character-level language model

        This is the same architecture as GPT-2, just much smaller:
        - GPT-2: 12 layers, 12 heads, 768 dim, 117M params
        - Ours:  2 layers, 2 heads, 64 dim, ~100K params
        """

        def __init__(self, vocab_size, d_model=64, n_heads=2, n_layers=2,
                        block_size=64, d_ff=256, dropout=0.1):
            super().__init__()
            self.block_size = block_size

            # Token embeddings: each character gets a learned vector
            self.tok_emb = nn.Embedding(vocab_size, d_model)
            # Positional embeddings: each position gets a learned vector
            self.pos_emb = nn.Embedding(block_size, d_model)
            self.drop = nn.Dropout(dropout)

            # Stack of transformer blocks
            self.blocks = nn.ModuleList([
                TransformerBlock(d_model, n_heads, d_ff, dropout)
                for _ in range(n_layers)
            ])

            # Final layer norm and output projection
            self.ln_f = nn.LayerNorm(d_model)
            self.head = nn.Linear(d_model, vocab_size, bias=False)

            # Weight tying: share embedding weights with output projection
            # (a common trick that reduces parameters and improves performance)
            self.head.weight = self.tok_emb.weight

            # Count parameters
            n_params = sum(p.numel() for p in self.parameters())

```

```

print(f"TinyGPT: {n_params:,} parameters")
print(f"  vocab_size={vocab_size}, d_model={d_model}, n_heads={n_
print(f"  n_layers={n_layers}, block_size={block_size}")

def forward(self, idx, targets=None):
    B, T = idx.shape
    assert T <= self.block_size, f"Sequence length {T} > block_size {"

    # Token + positional embeddings
    tok = self.tok_emb(idx) # (B, T, d_model)
    pos = self.pos_emb(torch.arange(T, device=idx.device)) # (T, d_model)
    x = self.drop(tok + pos) # (B, T, d_model)

    # Causal mask
    mask = torch.tril(torch.ones(T, T, device=idx.device)).unsqueeze(

    # Pass through transformer blocks
    for block in self.blocks:
        x = block(x, mask=mask)

    x = self.ln_f(x)
    logits = self.head(x) # (B, T, vocab_size)

    # Compute loss if targets provided
    loss = None
    if targets is not None:
        loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targ

    return logits, loss

@torch.no_grad()
def generate(self, idx, max_new_tokens, temperature=1.0):
    """Autoregressive generation: predict one token at a time."""
    self.eval()
    for _ in range(max_new_tokens):
        # Crop to block_size if needed
        idx_cond = idx[:, -self.block_size:]
        logits, _ = self(idx_cond)
        # Take logits at the last position
        logits = logits[:, -1, :] / temperature
        probs = F.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)
        idx = torch.cat([idx, next_token], dim=1)
    return idx

```

Shakespeare Dataset

We include a chunk of Shakespeare text directly in the notebook -- the same approach as Notebook 1. Now we will model it with a transformer.

```

In [ ]: # ~5KB of Shakespeare text embedded directly
SHAKESPEARE_TEXT = """ROMEO: But, soft! what light through yonder window

print(f"Corpus size: {len(SHAKESPEARE_TEXT)} characters")
print(f"Preview: {SHAKESPEARE_TEXT[:200]}...")

```

In []:

```

# Build character-level vocabulary
chars = sorted(list(set(SHAKESPEARE_TEXT)))
vocab_size = len(chars)
char_to_idx = {ch: i for i, ch in enumerate(chars)}
idx_to_char = {i: ch for i, ch in enumerate(chars)}

print(f"Vocabulary size: {vocab_size}")
print("Characters:", "".join(chars))

# Encode the entire text as a tensor of integer indices
data = torch.tensor([char_to_idx[ch] for ch in SHAKESPEARE_TEXT], dtype=t
print(f"Encoded tensor shape: {data.shape}")

```

```

In [ ]: class ShakespeareDataset(Dataset):
    """Character-level dataset: given a sequence of characters,
        predict the next character at each position.

    This is exactly the same task as Notebook 3, but now we use
    PyTorch Dataset/DataLoader instead of manual batching.
    """
    def __init__(self, data, block_size):
        self.data = data
        self.block_size = block_size

    def __len__(self):
        return len(self.data) - self.block_size

    def __getitem__(self, idx):
        chunk = self.data[idx : idx + self.block_size + 1]
        x = chunk[:-1] # input: characters 0..block_size-1
        y = chunk[1:] # target: characters 1..block_size (shifted by
        return x, y

BLOCK_SIZE = 64
dataset = ShakespeareDataset(data, BLOCK_SIZE)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True, drop_last=T
print(f"Dataset size: {len(dataset)} sequences of length {BLOCK_SIZE}")
print(f"Batches per epoch: {len(dataloader)}")

```

Training the Tiny GPT

Let's train! We use a standard training loop with AdamW optimizer. The model is small enough to train in a few minutes on CPU.

```

In [ ]: # Create model
torch.manual_seed(SEED)
config = {
    "vocab_size": vocab_size,
    "d_model": 64,
    "n_heads": 2,
    "n_layers": 2,
    "block_size": BLOCK_SIZE,
    "d_ff": 256,
    "dropout": 0.1,
}

```

```
model = TinyGPT(**config).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=0
```

```
In [ ]: # Training loop
NUM_EPOCHS = 3
losses = []

print("Training Tiny GPT...")
print("=" * 60)

for epoch in range(1, NUM_EPOCHS + 1):
    model.train()
    epoch_losses = []

    for xb, yb in dataloader:
        xb, yb = xb.to(device), yb.to(device)

        _, loss = model(xb, targets=yb)

        optimizer.zero_grad()
        loss.backward()
        # Gradient clipping (prevents exploding gradients -- Notebook 2 t
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        epoch_losses.append(loss.item())
        losses.append(loss.item())

    avg_loss = np.mean(epoch_losses)
    print(f"Epoch {epoch:2d}/{NUM_EPOCHS} | Loss: {avg_loss:.4f}")

    # Generate a sample to see progress
    model.eval()
    prompt = torch.tensor([[char_to_idx["R"]]], dtype=torch.long, device=
    gen_ids = model.generate(prompt, max_new_tokens=80, temperature=0.8)
    text = "".join(idx_to_char[i] for i in gen_ids[0].tolist())
    print(f" Sample: {text[:80]}")
    print()

print("Training complete!")
```

```
In [ ]: # Plot training loss
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(range(1, len(losses) + 1), losses, alpha=0.3, color="steelblue",
# Smoothed loss
window = max(1, len(losses) // 20)
if len(losses) > window:
    smoothed = np.convolve(losses, np.ones(window)/window, mode="valid")
    ax.plot(range(window, window + len(smoothed)), smoothed, color="darkb
ax.set_xlabel("Training Step")
ax.set_ylabel("Cross-Entropy Loss")
ax.set_title("Tiny GPT Training Loss")
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Text Generation with Temperature Control

Remember from Notebook 3: **temperature** controls the sharpness of the softmax distribution.

- $T < 1$: more conservative, repetitive, "safe" text
- $T = 1$: sample from the learned distribution as-is
- $T > 1$: more random, creative, but also more errors

```
In [ ]: # Generate text at different temperatures
model.eval()
temperatures = [0.3, 0.7, 1.0, 1.5]
prompt_text = "ROMEO: "
prompt_ids = torch.tensor([[char_to_idx[c] for c in prompt_text]], dtype=

print("=" * 70)
print("TEXT GENERATION AT DIFFERENT TEMPERATURES")
print("=" * 70)

for temp in temperatures:
    torch.manual_seed(SEED)
    gen_ids = model.generate(prompt_ids, max_new_tokens=150, temperature=
    text = "".join(idx_to_char[i] for i in gen_ids[0].tolist())
    print(f"\n--- Temperature = {temp} ---")
    print(text)

print("\n" + "=" * 70)
print("Notice: lower temperature = more repetitive but safer;")
print("higher temperature = more variety but more nonsense.")
print("The model learned statistical patterns, not meaning!")
```

Part 3 -- Using Pretrained Models with HuggingFace

Our tiny transformer learned some Shakespeare-like patterns from a few KB of text. Real language models like GPT-2 are trained on *billions* of words from the internet.

HuggingFace Transformers gives us easy access to thousands of pretrained models. Let's load **DistilGPT-2** -- a distilled (compressed) version of GPT-2 with 82M parameters. It runs comfortably on CPU.

```
In [ ]: from transformers import GPT2LMHeadModel, GPT2Tokenizer, pipeline
import torch

# Load pre-trained DistilGPT-2
print("Loading DistilGPT-2...")
pretrained_tokenizer = GPT2Tokenizer.from_pretrained("distilgpt2")
pretrained_model = GPT2LMHeadModel.from_pretrained("distilgpt2")
pretrained_model.eval()

# Set pad token (GPT-2 doesn't have one by default)
```

```

pretrained_tokenizer.pad_token = pretrained_tokenizer.eos_token

n_params = sum(p.numel() for p in pretrained_model.parameters())
print(f"DistilGPT-2 loaded: {n_params:,} parameters")
print(f"Vocabulary size: {pretrained_tokenizer.vocab_size}")
print(f"\nCompare: our Tiny GPT has ~100K params trained on ~5KB of Shake
print(f"DistilGPT-2 has {n_params:,} params trained on ~8GB of internet t

```

```

In [ ]: # Simple text generation using the pipeline API
generator = pipeline(
    "text-generation",
    model=pretrained_model,
    tokenizer=pretrained_tokenizer,
    device=-1, # CPU
)

prompts = [
    "The meaning of life is",
    "In the beginning, there was",
    "Once upon a time in a land far away",
]

print("=" * 70)
print("DISTILGPT-2 TEXT GENERATION")
print("=" * 70)

for prompt in prompts:
    result = generator(
        prompt,
        max_new_tokens=60,
        do_sample=True,
        temperature=0.8,
        num_return_sequences=1,
        pad_token_id=pretrained_tokenizer.eos_token_id,
    )
    print(f"\nPrompt: {prompt!r}")
    print(f"Output: {result[0]['generated_text']}")
    print("-" * 70)

```

Top-k and Top-p (Nucleus) Sampling

Beyond temperature, modern LLMs use **top-k** and **top-p** sampling to control the quality/diversity

- **Top-k**: only sample from the k most probable tokens
- **Top-p** (nucleus): only sample from the smallest set of tokens whose cumulative probability $\geq p$

These prevent the model from picking extremely unlikely tokens (which cause incoherent text) while allowing some diversity.

```

In [ ]: prompt = "Once upon a time in a land far away"

print("=" * 70)
print("TOP-K AND TOP-P SAMPLING COMPARISON")
print("=" * 70)

configs = [

```

```

{"label": "Greedy (no sampling)", "do_sample": False},
{"label": "Temperature=1.0 (raw)", "do_sample": True, "temperature":
{"label": "Top-k=50", "do_sample": True, "top_k": 50},
{"label": "Top-p=0.9", "do_sample": True, "top_p": 0.9},
{"label": "Top-k=50 + Top-p=0.9", "do_sample": True, "top_k": 50, "to
]

for cfg in configs:
    label = cfg.pop("label")
    result = generator(
        prompt,
        max_new_tokens=50,
        num_return_sequences=1,
        pad_token_id=pretrained_tokenizer.eos_token_id,
        **cfg,
    )
    print(f"\n[{label}]")
    print(result[0]["generated_text"])
    print("-" * 70)

```

Curriculum Point 4: Neural Networks Don't "Understand"

Look at the generated text above. It is often grammatically plausible and locally coherent. But does the model *understand* what it is writing?

No. The model has learned statistical regularities: which tokens tend to follow which other tokens. It has no concept of truth, meaning, or the real world.

Let's demonstrate this with adversarial prompts -- questions where the "pattern-matching" nature of the model becomes obvious.

```

In [ ]: # Demonstration: the model doesn't "understand"
adversarial_prompts = [
    "2 + 2 = ",
    "The capital of the fictional country Zorgblatt is",
    "If you reverse the word hello you get",
    "The number after 999999 is",
]

print("=" * 70)
print("ADVERSARIAL PROMPTS: Testing 'understanding'")
print("=" * 70)

for prompt in adversarial_prompts:
    result = generator(
        prompt,
        max_new_tokens=30,
        do_sample=True,
        temperature=0.7,
        num_return_sequences=1,
        pad_token_id=pretrained_tokenizer.eos_token_id,
    )
    print(f"\nPrompt: {prompt!r}")
    print(f"Output: {result[0]['generated_text']}")
    print("-" * 70)

print()

```



```
print("KEY TAKEAWAY: The model generates plausible-*sounding* text, but:")
print("  - It may get math wrong (it is not computing, just pattern match")
print("  - It happily invents facts about nonexistent things")
print("  - String reversal requires algorithmic reasoning it does not hav")
print("  - These failures reveal: it learned P(next_token | context), not
```

Part 4 -- Fine-Tuning DistilGPT-2

Fine-tuning takes a pre-trained model and continues training on a small domain-specific dataset. The model already knows English -- we just shift its distribution toward our target domain.

We will fine-tune DistilGPT-2 on a small corpus of cooking recipes. This demonstrates the power of transfer learning: instead of training from scratch (which requires massive data and compute), we start from a model that already "knows" language and nudge it toward our domain.

```
In [ ]: # Small cooking recipe corpus for fine-tuning
        RECIPE_CORPUS = """Recipe: Classic Chocolate Chip Cookies
        Ingredients: 2 cups flour, 1 cup butter, 1 cup sugar, 2 eggs, 1 tsp vanilla
        Instructions: Preheat oven to 375F. Cream butter and sugar. Beat in eggs

        Recipe: Simple Tomato Pasta
        Ingredients: 1 lb spaghetti, 4 tomatoes, 3 cloves garlic, olive oil, basil
        Instructions: Cook pasta until al dente. Dice tomatoes and mince garlic.

        Recipe: Banana Bread
        Ingredients: 3 ripe bananas, 1/3 cup melted butter, 3/4 cup sugar, 1 egg,
        Instructions: Preheat oven to 350F. Mash bananas and mix with melted butter

        Recipe: Greek Salad
        Ingredients: 1 cucumber, 4 tomatoes, 1 red onion, 1 cup olives, feta cheese
        Instructions: Chop cucumber, tomatoes, and onion into chunks. Add olives

        Recipe: Chicken Stir Fry
        Ingredients: 2 chicken breasts, 2 cups mixed vegetables, 3 tbsp soy sauce
        Instructions: Slice chicken thinly. Cook rice. Heat sesame oil in wok. Stir fry

        Recipe: Homemade Pizza
        Ingredients: pizza dough, 1 cup tomato sauce, 2 cups mozzarella, toppings
        Instructions: Preheat oven to 450F. Roll out dough on floured surface. Spread

        Recipe: Vegetable Soup
        Ingredients: 2 carrots, 2 potatoes, 1 onion, 2 celery stalks, 4 cups broth
        Instructions: Dice all vegetables. Sauté onion and garlic until soft. Add

        Recipe: Pancakes
        Ingredients: 1.5 cups flour, 3.5 tsp baking powder, 1 tbsp sugar, 1.25 cups milk
        Instructions: Mix flour, baking powder, sugar. Make well in center. Pour milk

        print(f"Recipe corpus: {len(RECIPE_CORPUS)} characters")
        print(f"Preview:\n{RECIPE_CORPUS[:200]}...")
```

```
In [ ]: from torch.utils.data import Dataset as TorchDataset
```

```

class TextDataset(TorchDataset):
    """Simple text dataset for fine-tuning: encode text with the tokenizer
    and return overlapping windows of token IDs."""
    def __init__(self, text, tokenizer, block_size=128):
        self.encodings = tokenizer.encode(text)
        self.block_size = block_size

    def __len__(self):
        return max(0, len(self.encodings) - self.block_size)

    def __getitem__(self, idx):
        chunk = self.encodings[idx : idx + self.block_size + 1]
        x = torch.tensor(chunk[:-1], dtype=torch.long)
        y = torch.tensor(chunk[1:], dtype=torch.long)
        return x, y

ft_dataset = TextDataset(RECIPE_CORPUS, pretrained_tokenizer, block_size=128)
ft_loader = DataLoader(ft_dataset, batch_size=4, shuffle=True, drop_last=True)
print(f"Fine-tuning dataset: {len(ft_dataset)} sequences of 128 tokens")
print(f"Batches per epoch: {len(ft_loader)}")

```

Generate BEFORE fine-tuning

```

In [ ]: # Generate recipe text BEFORE fine-tuning
prompt_recipe = "Recipe: Lemon"
torch.manual_seed(SEED)
result = generator(
    prompt_recipe,
    max_new_tokens=80,
    do_sample=True,
    temperature=0.7,
    num_return_sequences=1,
    pad_token_id=pretrained_tokenizer.eos_token_id,
)
print("BEFORE fine-tuning:")
print(result[0]["generated_text"])

```

Fine-tuning loop

We do a manual training loop for transparency -- just 2 epochs is enough to see the distribution shift on larger tasks you would use HuggingFace Trainer.)

```

In [ ]: # Fine-tuning: manual loop for transparency
ft_model = GPT2LMHeadModel.from_pretrained("distilgpt2")
ft_model.train()

ft_optimizer = torch.optim.AdamW(ft_model.parameters(), lr=5e-5, weight_decay=0.01)

FT_EPOCHS = 2
ft_losses = []

print("Fine-tuning DistilGPT-2 on recipe corpus...")
print("=" * 60)

for epoch in range(1, FT_EPOCHS + 1):
    epoch_losses = []

```

```

for xb, yb in ft_loader:
    outputs = ft_model(xb, labels=yb)
    loss = outputs.loss

    ft_optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(ft_model.parameters(), 1.0)
    ft_optimizer.step()

    epoch_losses.append(loss.item())
    ft_losses.append(loss.item())

avg = np.mean(epoch_losses)
print(f"Epoch {epoch}/{FT_EPOCHS} | Avg Loss: {avg:.4f}")

print("\nFine-tuning complete!")

```

```

In [ ]: # Plot fine-tuning loss
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(range(1, len(ft_losses) + 1), ft_losses, marker="o", markersize=3)
ax.set_xlabel("Training Step")
ax.set_ylabel("Cross-Entropy Loss")
ax.set_title("DistilGPT-2 Fine-Tuning Loss (Recipe Corpus)")
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

Generate AFTER fine-tuning

```

In [ ]: # Generate recipe text AFTER fine-tuning
ft_model.eval()
ft_generator = pipeline(
    "text-generation",
    model=ft_model,
    tokenizer=pretrained_tokenizer,
    device=-1,
)

torch.manual_seed(SEED)
result = ft_generator(
    "Recipe: Lemon",
    max_new_tokens=120,
    do_sample=True,
    temperature=0.7,
    num_return_sequences=1,
    pad_token_id=pretrained_tokenizer.eos_token_id,
)
print("AFTER fine-tuning:")
print(result[0]["generated_text"])

```

```

In [ ]: # Try a few more prompts to demonstrate the shift
ft_prompts = [
    "Recipe: Chocolate",
    "Ingredients: 2 cups",
    "Instructions: Preheat",
]

print("=" * 70)

```

```

print("MORE FINE-TUNED GENERATIONS")
print("=" * 70)

for prompt in ft_prompts:
    torch.manual_seed(SEED)
    result = ft_generator(
        prompt,
        max_new_tokens=80,
        do_sample=True,
        temperature=0.7,
        num_return_sequences=1,
        pad_token_id=pretrained_tokenizer.eos_token_id,
    )
    print(f"\nPrompt: {prompt!r}")
    print(result[0]["generated_text"])
    print("-" * 70)

```

What Changed in the Model?

Fine-tuning did not add new "knowledge" to the model. What it did:

1. **Shifted the probability distribution** -- tokens like "Recipe:", "Ingredients:", "Preheat" became likely in context.
2. **Adjusted attention patterns** -- the model now attends to recipe-structure cues.
3. **Preserved general English** -- the model still knows English grammar; it just preferentially generates recipe-like text now.

This is the same principle as Notebook 2's training dynamics: gradient descent moved the weights to reduce loss on the new data, adjusting the loss landscape (curriculum point 6).

Part 5 -- Building a Gradio Demo

Gradio lets you build web UIs for ML models in a few lines of code. This is the last step in the deployment stack: `weights -> inference -> API -> UI`.

```

In [ ]: import gradio as gr

def generate_text(prompt, temperature, max_length, top_k):
    """Generate text using our fine-tuned model."""
    if not prompt.strip():
        return "Please enter a prompt."

    torch.manual_seed(42)
    result = ft_generator(
        prompt,
        max_new_tokens=int(max_length),
        do_sample=True,
        temperature=float(temperature),
        top_k=int(top_k),
        num_return_sequences=1,
        pad_token_id=pretrained_tokenizer.eos_token_id,
    )

```

```

    return result[0]["generated_text"]

# Build the Gradio interface
demo = gr.Interface(
    fn=generate_text,
    inputs=[
        gr.Textbox(label="Prompt", placeholder="Recipe: Chocolate...", li
        gr.Slider(minimum=0.1, maximum=2.0, value=0.7, step=0.1, label="T
        gr.Slider(minimum=10, maximum=200, value=80, step=10, label="Max
        gr.Slider(minimum=1, maximum=100, value=50, step=1, label="Top-k"
    ],
    outputs=gr.Textbox(label="Generated Text", lines=8),
    title="Recipe Generator (Fine-tuned DistilGPT-2)",
    description="Generate cooking recipes using a fine-tuned language mod
        "Adjust temperature (creativity), max tokens (length), an
    examples=[
        ["Recipe: Lemon", 0.7, 100, 50],
        ["Ingredients: 1 cup flour", 0.8, 80, 50],
        ["Instructions: Preheat oven", 0.6, 80, 40],
    ],
)

# Display the interface inline (in Jupyter) or print a note
# NOTE: In Jupyter, demo.launch() opens a local server.
# We just show the interface definition here.
print("Gradio demo defined!")
print("To launch interactively, run: demo.launch()")
print()
print("The interface has:")
print(" - Text input for the prompt")
print(" - Temperature slider (0.1 to 2.0)")
print(" - Max tokens slider (10 to 200)")
print(" - Top-k slider (1 to 100)")
print(" - Generated text output")
print()
print("This is the deployment stack in action:")
print("  Model weights -> Inference function -> Gradio API -> Web UI")

```

Deploying to Hugging Face Spaces

To make your demo permanently accessible:

1. Create a free account at huggingface.co
2. Create a new Space (select Gradio as the SDK)
3. Upload your code and model files
4. The Space builds and deploys automatically -- you get a public URL

This completes the stack: **trained weights -> inference code -> API -> public web UI**.

The same architecture powers ChatGPT, Claude, and other AI products:

- A model (much larger) generates text
- An API wraps the model
- A web UI makes it accessible to users

The difference is scale (billions vs millions of parameters), safety (RLHF, filters), and infrastructure (clusters), but the *conceptual* stack is identical.

Part 6 -- Reflection: What Deep Learning Is and Isn't

You have now completed the full journey from a single neuron (Notebook 1) to a fine-tuned language model with a web interface. Let's step back and review everything through the lens of our 7 curriculum points.

The 7 Curriculum Points -- Revisited

1. Universal Approximation Theorem

A single hidden layer can approximate any continuous function; depth = efficiency.

- **Notebook 1:** You proved this by watching a tiny MLP learn a complex decision boundary on the moons dataset.
- **Here:** Our TinyGPT (2 layers) learned to mimic Shakespeare -- a very complex function from character sequences to probability distributions. More layers and attention heads = more efficient representation of these patterns.

2. ReLU Changed Everything

Vanishing gradients with sigmoid/tanh; sparse activation; better gradient flow.

- **Notebook 1:** You compared sigmoid vs ReLU gradient flow and saw sigmoid gradients vanish in deep networks.
- **Notebook 2:** Deep sigmoid networks failed to train; ReLU networks succeeded.
- **Here:** Modern transformers use GELU (a smooth cousin of ReLU) because it provides good gradient flow while being differentiable everywhere. The principle is the same: avoid saturating activations.

3. Overparameterization Helps

More params than data, yet generalizes; implicit regularization of SGD.

- **Notebook 2:** You trained an overparameterized MLP on a subset of MNIST and saw it generalize despite having more parameters than training samples.
- **Notebook 5:** The CNN experiments confirmed this pattern.
- **Here:** DistilGPT-2 has 82M parameters -- far more than the ~5KB of Shakespeare or the training corpus. Yet it generates coherent text. SGD + dropout + weight decay provide implicit regularization.

4. Neural Networks Don't "Understand" (This notebook's focus)

They minimize loss, detect statistical structure, not meaning.

- **Notebook 3:** Your character-level model generated plausible-looking text that was nonsensical, relying on pure statistical pattern matching.
- **Here:** DistilGPT-2 confidently answers "2+2 = " with wrong answers, invents facts about fictional countries, and cannot reverse strings. It has learned $P(\text{next_token} \mid \text{context})$, not reasoning, meaning.

5. Backpropagation = Chain Rule

Efficient gradient computation via dynamic programming.

- **Notebook 1:** You built the autograd engine and computed gradients by hand.
- **Notebook 2:** You manually derived and implemented backprop for matrix operations.
- **Here:** PyTorch's autograd computes gradients through the entire transformer (attention, layer feed-forward, embeddings) automatically -- the same chain rule, just applied to a much deeper computation graph.

6. Geometry Matters

High-dimensional loss landscape; flat minima -> better generalization.

- **Notebook 2:** You visualized loss landscapes and saw how initialization affects which minima the optimizer finds.
- **Notebook 5:** BatchNorm and residual connections smoothed the loss landscape.
- **Here:** LayerNorm, residual connections, and the AdamW optimizer all help navigate the loss landscape of a transformer. The learning rate warmup and cosine schedule we could add are other landscape navigation tools.

7. Capacity != Performance

More layers != better; architecture, init, normalization, data quality dominate.

- **Notebook 2:** You saw that a well-initialized small network beats a poorly initialized large one.
- **Notebook 5:** The ablation study showed architecture choices matter more than raw size.
- **Here:** Our 100K-parameter TinyGPT learned recognizable Shakespeare patterns. DistilGPT-2 (70M params) generates much better text, but even it makes mistakes. GPT-4 (rumored ~1T params) is much better still, but *still* does not understand. Adding parameters helps, but architecture (at least), training data, and training recipe matter more.

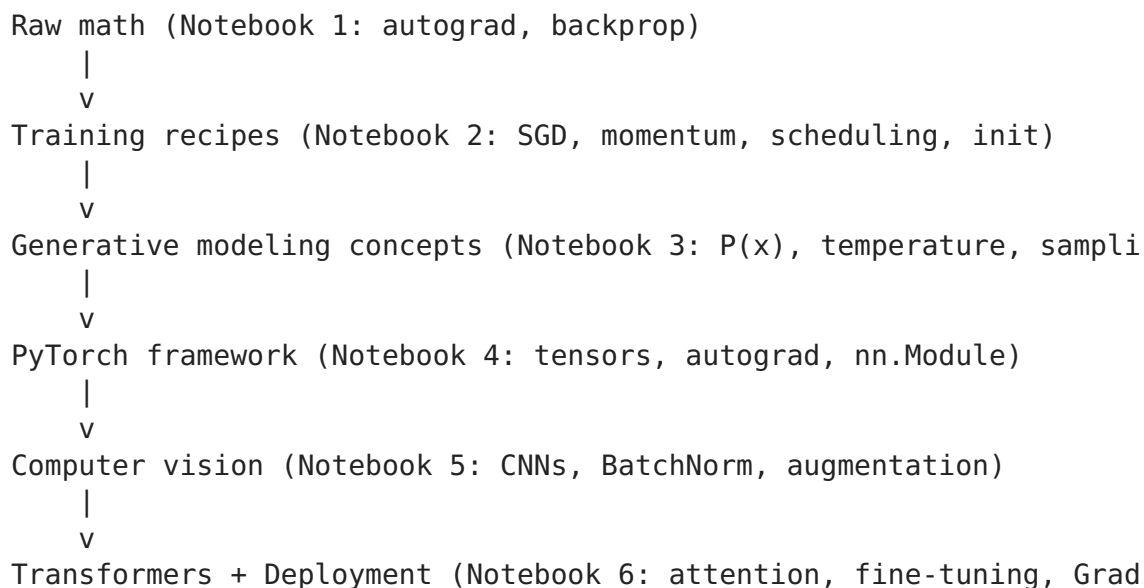
What the Model Learned vs. What It Didn't

Learned	Did NOT Learn
Statistical patterns in token sequences	Meaning, truth, or causation

Learned	Did NOT Learn
Grammar and syntax (mostly)	Logic or reasoning
Domain-specific vocabulary (after fine-tuning)	Facts (it can only parrot training data patterns)
How to generate plausible continuations	Whether a statement is true
Attention patterns for long-range dependencies	Understanding of the physical world

The Deployment Stack

Throughout this series, we built up the full stack:



Each layer builds on the previous. You started by computing gradients by hand; now you can fine-tune a pretrained transformer and deploy it with a web UI.

What Comes Next

This series gave you the conceptual and practical foundations. From here you can:

1. **Scale up:** Use larger models (GPT-2, LLaMA) and GPU training
2. **Explore modalities:** Apply transformers to images (ViT), audio, video
3. **Learn RLHF:** How ChatGPT-style models are aligned with human preferences
4. **Study safety:** Adversarial attacks, jailbreaks, alignment research
5. **Build products:** Combine models with retrieval (RAG), tools, and agents

The conceptual toolkit you have -- autograd, loss landscapes, attention, transfer learning, the gap between statistics and understanding -- will serve you in all of these directions.

Exercises

Exercise 1: Implement Positional Encoding (Sinusoidal)

Our TinyGPT uses **learned** positional embeddings. The original "Attention Is All You Need" paper uses **sinusoidal** positional encodings instead:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Task: Implement `SinusoidalPositionalEncoding` as an `nn.Module`. Then replace the positional embedding in TinyGPT and retrain for 1 epoch. Compare the loss.

```
In [ ]: # Exercise 1: Sinusoidal Positional Encoding

class SinusoidalPositionalEncoding(nn.Module):
    """Sinusoidal positional encoding from Attention Is All You Need.

    Unlike learned positional embeddings, these are fixed (not trained).
    They have a nice property: the encoding for position p+k can be expressed
    as a linear function of the encoding for position p.
    """
    def __init__(self, d_model, max_len=512):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0)))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        # Register as buffer (not a parameter -- no gradients)
        self.register_buffer("pe", pe.unsqueeze(0)) # (1, max_len, d_model)

    def forward(self, x):
        """Add positional encoding to input embeddings.
        x: (batch, seq_len, d_model)
        """
        return x + self.pe[:, :x.size(1), :]
```

```
# Test it
spe = SinusoidalPositionalEncoding(d_model=64, max_len=256)
test_input = torch.zeros(1, 64, 64) # batch=1, seq=64, dim=64
test_output = spe(test_input)
```

```
# Visualize the positional encodings
fig, ax = plt.subplots(figsize=(10, 4))
im = ax.imshow(spe.pe[0, :64, :].numpy(), aspect="auto", cmap="RdBu")
ax.set_xlabel("Embedding Dimension")
ax.set_ylabel("Position")
ax.set_title("Sinusoidal Positional Encodings (first 64 positions)")
fig.colorbar(im, ax=ax)
plt.tight_layout()
plt.show()
```

```

print("Each row is the positional encoding for one position.")
print("Notice the sinusoidal patterns at different frequencies across dim")
print("Lower dimensions oscillate slowly (capture coarse position);")
print("higher dimensions oscillate rapidly (capture fine position).")

```

Exercise 2: Attention Pattern Analysis

After training the Tiny GPT, extract attention weights for a sample input and visualize what each learned.

Task: Feed the string "ROMEO: To be or" through the trained model, extract attention weights for layer and head, and plot them as heatmaps.

```

In [ ]: # Exercise 2 Solution: Attention Pattern Analysis

model.eval()

# Encode a sample and get attention weights from each block
sample_text = "ROMEO: To be or"
sample_ids = torch.tensor([[char_to_idx.get(c, 0) for c in sample_text]],
sample_chars = list(sample_text)

# Forward pass, collecting attention weights
with torch.no_grad():
    tok = model.tok_emb(sample_ids)
    pos = model.pos_emb(torch.arange(sample_ids.size(1)))
    x = model.drop(tok + pos)
    mask = torch.tril(torch.ones(sample_ids.size(1), sample_ids.size(1)))

    all_weights = []
    for block in model.blocks:
        normed = block.ln1(x)
        _, weights_ex = block.attn(normed, mask=mask)
        all_weights.append(weights_ex)
        # Continue forward pass
        attn_out, _ = block.attn(normed, mask=mask)
        x = x + block.drop1(attn_out)
        normed = block.ln2(x)
        ffn_out = block.ffn(normed)
        x = x + block.drop2(ffn_out)

# Plot attention patterns
n_layers = len(all_weights)
n_heads = all_weights[0].size(1)
fig, axes = plt.subplots(n_layers, n_heads, figsize=(5 * n_heads, 4 * n_l
if n_layers == 1:
    axes = axes.reshape(1, -1)

for layer_idx in range(n_layers):
    for head_idx in range(n_heads):
        ax = axes[layer_idx, head_idx]
        w = all_weights[layer_idx][0, head_idx].numpy()
        ax.imshow(w, cmap="Blues", vmin=0)
        ax.set_title(f"Layer {layer_idx+1}, Head {head_idx+1}")
        if head_idx == 0:
            ax.set_ylabel("Query")

```

```

        if layer_idx == n_layers - 1:
            ax.set_xlabel("Key")
        # Add character labels
        ax.set_xticks(range(len(sample_chars)))
        ax.set_xticklabels(sample_chars, rotation=90, fontsize=7)
        ax.set_yticks(range(len(sample_chars)))
        ax.set_yticklabels(sample_chars, fontsize=7)

plt.suptitle("Attention Patterns for: " + repr(sample_text), fontsize=14,
plt.tight_layout()
plt.show()

print("Different heads learn different attention patterns:")
print(" - Some heads attend to adjacent characters (local context)")
print(" - Some heads attend to specific characters like ':' or spaces (s
print(" - The causal mask ensures all attention is lower-triangular")

```

Exercise 3: Temperature vs. Perplexity Trade-off

Generate 200 characters at temperatures from 0.1 to 2.0. For each, compute the perplexity of the generated text under the model.

Perplexity = $\exp(\text{average cross-entropy loss})$. Lower perplexity means the model finds more "expected".

Task: Plot temperature vs. perplexity. What trend do you see?

```

In [ ]: # Exercise 3 Solution: Temperature vs Perplexity

model.eval()

temperatures_sweep = [0.1, 0.2, 0.3, 0.5, 0.7, 1.0, 1.3, 1.5, 2.0]
perplexities = []
gen_texts = []

prompt_ids_ex = torch.tensor([[char_to_idx["R"]]], dtype=torch.long, devi

for temp in temperatures_sweep:
    torch.manual_seed(SEED)
    gen_ids = model.generate(prompt_ids_ex, max_new_tokens=200, temperatu
    gen_seq = gen_ids[0]

    # Compute perplexity: feed the generated text back through the model
    # and measure how "surprised" the model is
    with torch.no_grad():
        total_loss = 0.0
        count = 0
        for i in range(0, len(gen_seq) - 1, BLOCK_SIZE):
            chunk = gen_seq[i : i + BLOCK_SIZE + 1]
            if len(chunk) < 2:
                break
            x_chunk = chunk[:-1].unsqueeze(0)
            y_chunk = chunk[1:].unsqueeze(0)
            if x_chunk.size(1) > BLOCK_SIZE:
                x_chunk = x_chunk[:, :BLOCK_SIZE]
                y_chunk = y_chunk[:, :BLOCK_SIZE]
            _, loss_val = model(x_chunk, targets=y_chunk)

```

```

        total_loss += loss_val.item() * x_chunk.size(1)
        count += x_chunk.size(1)

    avg_loss = total_loss / max(count, 1)
    ppl = np.exp(avg_loss)
    perplexities.append(ppl)

    text = "".join(idx_to_char[i] for i in gen_seq.tolist())
    gen_texts.append(text[:60])

# Plot
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(temperatures_sweep, perplexities, "o-", color="coral", linewidth=2)
ax.set_xlabel("Temperature", fontsize=12)
ax.set_ylabel("Perplexity", fontsize=12)
ax.set_title("Temperature vs. Perplexity of Generated Text", fontsize=14)
ax.grid(True, alpha=0.3)

# Annotate a few points
for i in [0, 4, -1]:
    ax.annotate(f"T={temperatures_sweep[i]}",
                xy=(temperatures_sweep[i], perplexities[i]),
                xytext=(10, 10), textcoords="offset points",
                fontsize=9, ha="left",
                arrowprops=dict(arrowstyle="->", color="gray"))

plt.tight_layout()
plt.show()

print("Observations:")
print(" - Low temperature: text is repetitive but has LOW perplexity")
print("   (the model is not surprised by repetitive output)")
print(" - High temperature: text is diverse but has HIGH perplexity")
print("   (random sampling produces unlikely sequences)")
print(" - Sweet spot around T=0.7-1.0: balanced quality and diversity")

```

Congratulations!

You have completed the full deep learning curriculum. Starting from a single neuron with manual computation, you have arrived at fine-tuning pretrained transformers and deploying them as web applications.

The most important takeaway: deep learning models are powerful pattern matchers that learn regularities from data. They do not understand, reason, or know truth. The gap between "impressive fluent text" and "actual understanding" is the central open question of AI research.

"The question of whether machines can think is about as relevant as the question of whether submarines can swim." -- Edsger Dijkstra

What matters is not whether the model "understands," but whether we understand what the model is doing -- and you now have the tools to investigate that.